Amazon & Airbnb: ML Coding and Practical ML System Design

Amazon L5 Applied Scientist and Airbnb Senior MLE onsite question banks

01. Amazon L5 Applied Scientist: ML Coding Round

- How this round works
- Top 15 to practice first
- A. Classical ML from scratch
- B. Deep learning from scratch (NumPy / PyTorch)
- C. NLP and LLM components
- D. Metrics and evaluation code
- E. Probability, statistics and sampling code
- F. Data wrangling (pandas / SQL / pipelines)
- G. DSA problems seen in AS loops
- H. Level calibration: L5 probes
- References

02. Amazon L5 Applied Scientist: Practical ML System Design

- How this round works
- Top 15 to practice first
- 1. Design an agentic recommendation system
- 2. Team-specific ML system design: from business problem to science problem
- 3. Design a multimodal translation pipeline for catalog objects
- 4. Design Alexa from scratch as an ML system
- 5. Summarize product reviews into key concepts
- 6. Design an end-to-end product recommendation engine
- 7. Frame an open-ended ML product problem
- 8. Design a spam or fake-review detection system
- 9. Classify short, noisy text (review language, tweets)
- 10. Design a GitHub Copilot-style coding agent
- 11. Design a system for global request detection and labeling
- 12. Design Amazon product search ranking
- 13. Design a Rufus-style LLM shopping assistant
- 14. Design CTR/CVR prediction and ad placement for Sponsored Products
- 15. Detect invalid or bot traffic on Amazon Ads
- 16. Design a fraud detection system for payments or orders
- 17. Extract structured product attributes for catalog quality
- 18. Detect duplicate listings in the catalog
- 19. Design a demand forecasting system for millions of ASINs
- 20. Predict delivery ETA (promise date)
- 21. Estimate price elasticity for pricing
- 22. Diagnose and redeploy a degrading production model
- 23. Design Prime Video homepage recommendations with cold start
- 24. Design GenAI ad creative generation for sellers
- 25. Cross-cutting probes asked across any prompt
- Level calibration: harder L5 probes
- References

03. Airbnb Senior MLE: ML Coding and Practical Coding

- How this round works
- Top 15 to practice first
- A. Airbnb-tagged practical coding problems (most reported first)
- B. ML-specific coding
- C. Data, pandas and SQL coding
- D. Experimentation and statistics in code
- E. Debugging and production-readiness probes
- F. Level calibration (Senior bar)
- References

04. Airbnb Senior MLE: Practical ML System Design

- How this round works

Top 15 to practice first
1. Search ranking without a text query
2. Pricing prediction and Smart Pricing
3. Retrieval-assisted (RAG) customer support
4. User embeddings and two-stage semantic retrieval and ranking
5. Offline wins, online flat: diagnose and decide
6. Guest or listing lifetime value (LTV)
7. Photo and listing quality models
8. Property damage claims
9. "Family friendly" listing classifier
10. Threshold choice and model choice under cost constraints
11. Real-time content personalization (homepage and categories)
12. Sensitive-information detection and redaction
13. Listing recommendation and similar listings
14. Relisting detection
15. ML across the booking funnel
16. Two-sided marketplace A/B test for a ranking or pricing launch
17. Pricing experiments on expiring inventory
18. Fraud and trust: fake bookings, fake listings, account takeover and review fraud
19. Multi-task ranking that balances guests and hosts
20. Diversity and map ranking
21. Cold start for new hosts, listings and guests
22. Natural-language query understanding and AI search
23. Monitoring, training-serving skew and the feature store
24. Distributed recommendation system at Airbnb scale
Level calibration: senior probes
References

01. Amazon L5 Applied Scientist: ML Coding Round

> Scope: the coding round(s) in the Amazon L5 Applied Scientist (Applied Scientist II) phone screen and onsite loop, with the focus on **ML coding**: implementing algorithms, losses, metrics, sampling and data pipelines from scratch in Python, NumPy or PyTorch. A short DSA section at the end covers the LeetCode-style problems reported in AS loops. This chapter merges and dedupes four independent search passes with the earlier chapters `amazon-l5-applied-scientist/chapters/05-ml-coding.md` and `04-coding-dsa.md`. **Questions only. No answers or solutions.**
>
> Tags: **[Reported]** means a candidate report, or an aggregator of candidate reports, ties the coding task to an Amazon science loop (citation [n]). Where the report is from a junior, intern, Senior, Data Scientist, Research Scientist or MLE loop, the line says so. **[Likely]** means the task is inferred: the concept was asked verbally in a breadth or depth round, or the prompt comes from a practice site or prep guide and no candidate report says it was coded. Many Glassdoor sources were read only as search-result extracts, and several extracts could not be tied to one exact review ID; the reference list flags these. Treat each report as the candidate's account.
>
> Ordering: within each section, questions are ordered by how many independent reports name them, then by importance for an L5 loop. Numbering runs across the whole chapter.

How this round works

- **Where it sits.** Amazon's AS prep page describes the onsite as 55-minute interviews with scientists [1]. AS II candidates in 2025-2026 report five rounds: ML breadth, ML depth, team-specific system design, coding, and a Bar Raiser [6]. A 2026 loop was "1 phone screen, 1 coding, 2 ML design and 2 LP rounds" [23]. A London 2026 loop had live coding plus ML depth and breadth rounds, 60 minutes each [32]. L5 phone screens mix ML concepts, statistics, LPs and a short coding task [9][74].
- **What the coding task looks like.** It depends on who interviews you [73]. An SDE-style interviewer gives "leetcode at L4 SWE bar. Correctness is top priority, explain your answer, give complexities" [17]. A scientist interviewer "tends to ask ML questions such as PCA and least squares" [11]; Blind commenters call OLS, gradient descent and batch normalization fair game, but not bagging, boosting or a vanilla recommender [10][14]. Reported mixes: "two coding questions, one ML related and the other an easy to medium leetcode question" (Melbourne, Nov 2025) [23]; "two easy Leetcode problems, some ML coding problems" (Santa Monica AS II, 2026) [3]; an ML coding round separate from ML design and LP rounds in a two-day onsite [51]; "most questions were non-leetcode questions more related to day to day ml implementations" (2026) [23]. LLM teams ask decoding code (beam search) in the

screen [8]; AGI teams report debugging a broken Transformer, training a small classifier live, and RL coding [56].

- **Duration and pacing.** 45 to 60 minutes [28][32]. LP questions often take the first 20 to 30 minutes, and several reports describe only about 15 minutes left for code [53][54][70]. Practice each core primitive so it fits in 15 minutes.
- **Language and tools.** Python. NumPy for from-scratch numerics, PyTorch on deep-learning and LLM teams, and no scikit-learn estimators unless the task is a dataset round, where pandas, NumPy and sklearn are allowed [13]. You write in Amazon's shared editor (Livecode), with no IDE, and the code is usually not run, so you trace it by hand [1][2][71]. Amazon asks for syntactically correct code, not pseudo code [2]; a Berlin candidate who wrote k-means in "a mix of Python and pseudo-code" shows how this gets tested [7].
- **What is evaluated at L5.** Correct, readable, vectorized code; complexity stated without prompting; edge cases (empty input, ties, a single class, all-zero weights) raised before you are asked; numerical stability (max-subtraction, log-space, stable variance); and an answer to the scale-up follow-up (streaming, mini-batch, distributed) [17][53][54]. Preparation matters: an LLM-team candidate could not deliver working beam search "as I hadn't prepared" [8], while a Senior AS candidate passed with an unfinished graph problem by narrating the approach clearly [49].

Top 15 to practice first

1. ROC AUC from scores and labels, with ties, in O(n log n) (Q44) [Reported]
2. k-means with k-means++ and empty-cluster handling (Q1) [Reported]
3. Logistic regression or "a traditional ML classifier" from scratch (Q2) [Reported]
4. Feed-forward neural network with backprop, extended to any number of layers (Q16) [Reported]
5. Gradient descent, then mini-batch SGD with momentum (Q3) [Reported]
6. OLS linear regression: closed form vs gradient descent (Q4) [Reported]
7. PCA from scratch with explained variance (Q5) [Reported]
8. Numerically stable softmax + cross-entropy and its gradient (Q17) [Reported]
9. Scaled dot-product attention with a causal mask (Q27) [Reported]
10. Beam search decoding for an LLM (Q29) [Reported]
11. Law-of-large-numbers simulation (Q55) [Reported]
12. Weighted random pick (Q56) [Reported]
13. Bootstrap confidence interval for a statistic (Q58) [Reported]
14. Dataset round: pandas cleaning, encoding, split, train, metrics (Q68) [Reported]
15. String compression, then number of islands, as the DSA warm-ups (Q79, Q80) [Reported]

A. Classical ML from scratch

1. **k-means.** Implement `kmeans(X: np.ndarray, k: int, max_iters: int = 300, tol: float = 1e-4, init: str = "k-means++", seed: int | None = None) -> tuple[np.ndarray, np.ndarray]` returning `centroids` (k, d) and `labels` (n,). Alternate assignment and update steps, stop when the centroid shift is below `tol` or at `max_iters`. Handle: empty

clusters, k > n, duplicate points, k = 1. No sklearn. [Reported] [7][15][39][52] (Berlin AS, Feb 2025, coded in Python [7]; a 2024 phone screen and a distance-metric follow-up [39]; Amazon aggregator sighting Dec 2025 [52]; also on Amazon-tagged practice lists [58][69])

- What is the cost per iteration, and can you compute all point-to-centroid distances with broadcasting and no Python loop over points?
- Can you implement k-means++ seeding as its own function, and why does it help?
- What do you do when a cluster goes empty?
- Which distance metric would you use, and what goes wrong when features have very different dynamic ranges (price vs rating)? [39]
- How would you choose k?
- How would you run it on data that arrives as a generator of (b, d) batches, or on 1 billion product embeddings (mini-batch, distributed)?
- How would you port it to PyTorch tensors on a GPU? [68]
- Does it always converge, and to what?

2. **Logistic regression / "a traditional ML classifier" from scratch.** Implement `class LogisticRegression` with `fit(X, y, lr=0.1, epochs=1000, l2=0.0)`, `predict_proba(X)` and `predict(X, threshold=0.5)`, trained with batch or mini-batch gradient descent, with a numerically stable sigmoid and log-loss. Then train on a train split, predict on a test split and report accuracy, precision and recall. Handle: a constant feature, a single class in `y`, separable data. [Reported] [13][15][22][24][56] (phone-screen "traditional ML classifier" [22][24]; "code a ML model from scratch", junior AS [13]; Blind loop summary names LR with gradient descent [15]; Amazon AGI "build/train a small classifier live" [56]; the conceptual loss and MLE were asked in [4]; also a practice prompt [60])

   - Can you derive the gradient of the log loss before you code it?
   - Why log loss and not MSE for this model? What are the model's assumptions? [38]
   - How do you avoid overflow in the sigmoid and `log(0)` in the loss?
   - Should the bias be regularized? How would you add L1 instead of L2?
   - The classes are 1:100. How do you add class weights to the loss and gradient, and which metric do you report (AUC vs PR-AUC)? [26]
   - How do you show it is learning when you cannot plot?
   - How would you check convergence, and what happens on perfectly separable data?

3. **Gradient descent, then SGD with momentum.** Implement `gradient_descent(X, y, lr, epochs) -> (w, b)` for linear regression with MSE, with the gradient written by hand and vectorized. Then generalize to `sgd(params, grad_fn, data, batch_size, lr, momentum, epochs)` that shuffles each epoch and applies momentum. Handle: a last batch smaller than `batch_size`, divergence (loss becomes NaN). [Reported] [19][21][52][70] ("implement gradient descent from scratch", AS intern [19]; "implement SGD for a problem (derive gradients, update, momentum etc.)" [21]; "write and explain gradient descent" [52][70])

- Why does the learning rate that works for batch GD fail for SGD?
- What does momentum change about the trajectory on an ill-conditioned problem?
- Why does feature scaling matter here?
- How would you add a learning-rate schedule and early stopping?

4. **Ordinary least squares, two ways.** Implement `fit_ols_closed_form(X, y)` with a bias column, and `fit_ols_gd(X, y, lr, epochs)`, and compare their weights on the same data. [Reported] [11][14][27] (Blind: scientist interviewers ask least squares [11][14]; an older review asked for the closed form, when GD is preferable, and the cost of the inversion [27])

    - What is the complexity of the closed form, and when does gradient descent win?
    - When is `X^T X` singular (collinearity, d > n), and what do you use instead (`lstsq`, pseudo-inverse, ridge)?
    - Why should you avoid calling `np.linalg.inv` directly?
    - How would you fit this when X has 1 billion rows and does not fit in memory?

5. **PCA.** Implement `pca(X: np.ndarray, k: int) -> tuple[components (k, d), projected (n, k), explained_variance_ratio (k,)]`, with centering and either the covariance eigendecomposition or the SVD. Handle: k > d, n < d, a zero-variance feature. [Reported] [11] (Blind: AS interviewers "tend to ask ML questions such as PCA and least squares")

    - SVD or eigendecomposition: which is more numerically stable, and which is cheaper when d is much larger than n?
    - Why are component signs arbitrary, and how would you make the output deterministic?
    - Should you standardize features first?
    - How would you get only the top component with power iteration?
    - How would you do PCA on data that streams in (incremental PCA)?

6. **Naive Bayes.** Implement multinomial Naive Bayes for text: `fit(docs: list[list[str]], labels)` estimates class priors and per-class word likelihoods with Laplace smoothing; `predict(docs)` uses log-probabilities. Then do the Gaussian variant for numeric features. Handle: unseen words, a class with no examples of a word, zero variance. [Reported] [53][54] (a Bayesian classifier from scratch, applied to interviewer scenarios, in Amazon Research Scientist and MLE loops; practice prompt [59])

    - Why compute in log space?
    - What does the smoothing parameter do, and how would you pick it?
    - What does the independence assumption break, and when does Naive Bayes still rank well?

7. **Decision tree (CART).** Implement `DecisionTreeClassifier(max_depth, min_samples_split)` with `fit(X, y)` and `predict(X)`: search every feature and candidate threshold for the best Gini or entropy split, then recurse. Handle: all labels equal, no valid split, duplicate feature values. [Likely] [57] (Amazon-tagged MLE onsite on a practice site, not an AS candidate report; how splits are chosen is asked verbally)

- What is the cost per node, and how does pre-sorting each feature help?
- How do you handle categorical features and missing values?
- How would you store the trained tree as flat arrays for fast prediction?
- How do you turn it into a random forest, and how do you compute out-of-bag error?

8. **k-nearest neighbors.** Implement `knn_predict(X_train, y_train, X_query, k) -> labels` with fully vectorized pairwise Euclidean distances (no Python loop over n x m pairs), majority vote, and distance-weighted tie-breaking. [Likely] [65]

    - The expansion `||a||^2 - 2ab + ||b||^2` can return small negative values. Why, and how do you fix it?
    - How do you pick k with cross-validation, and why must scaling be fit on training folds only?
    - What is the query cost, and what changes at millions of items (KD-tree, approximate nearest neighbors such as FAISS/HNSW)?

9. **Gradient boosting with stumps.** Implement `fit_gbm(X, y, n_estimators, learning_rate)` that fits depth-1 regression trees to the residuals of squared loss, plus `predict`. [Likely] (bagging vs boosting asked verbally in AS II loops [6][25]; Blind says boosting is a stretch for live coding [10])

    - What changes for log loss, and what are the pseudo-residuals?
    - How does this differ from bagging in what it does to bias and variance?

10. **Matrix-factorization recommender.** Implement `train_mf(ratings: list[tuple[user, item, rating]], k, lr, reg, epochs)` with SGD on regularized MSE, then `predict(u, i)` and `recommend(u, n)` that excludes items the user has already seen. [Likely] [38] (a recommendation-system question followed logistic regression)

    - How do you handle a cold-start user or item?
    - How would you adapt it to implicit feedback (clicks, no ratings)?

11. **Cross-validation and grid search from scratch.** Implement `stratified_kfold(y, k, seed)` and `grid_search(model_factory, param_grid, X, y, k) -> (best_params, results)` reporting mean and std per config, with preprocessing fit inside each training fold only. [Likely] [22]

    - Where can leakage creep in, and how does your code prevent it?
    - How do you split when rows from the same user must stay together?

12. **L1-regularized regression.** Implement `lasso(X: np.ndarray (n, d), y: np.ndarray (n,), alpha: float, max_iters: int = 1000, tol: float = 1e-6) -> np.ndarray (d,)` with cyclic coordinate descent and soft-thresholding (unpenalized intercept, objective `(1/2n)||y - Xw||^2 + alpha ||w||_1`). Stop when the max coordinate change is below `tol`. Handle: an all-zero column, alpha large enough that every weight is zero. [Likely]

    - Why does L1 produce exact zeros and L2 does not?
    - How do you keep the residual updated in O(n) per coordinate instead of recomputing `Xw`, and how would you compute a warm-started regularization

path?

13. **EM for a Gaussian mixture.** Implement `gmm_em(x: np.ndarray (n,), k: int = 2, max_iters: int = 200, tol: float = 1e-6, seed: int | None = None) -> (means (k,), variances (k,), weights (k,), log_likelihood: float)`. Compute responsibilities in log space, stop when the log-likelihood gain is below `tol`, and floor variances at a small epsilon. Handle: a component that collapses onto one point, identical data points. [Likely]
    - How is this related to k-means, and how do you keep the likelihood computation stable?
14. **Linear SVM.** Implement `fit_linear_svm(X (n, d), y in {-1, +1} (n,), lam: float, epochs: int, batch_size: int, seed) -> (w (d,), b)` minimizing `lam/2 ||w||^2 + mean(max(0, 1 - y(Xw + b)))` by mini-batch subgradient descent with step size `1 / (lam * t)` (Pegasos). Handle: labels given as {0, 1}, a batch with no margin violations. [Likely]
    - What does the subgradient do at the margin?
15. **Item-to-item collaborative filtering.** Given `interactions: scipy.sparse.csr_matrix` of shape (n_users, n_items) with implicit counts, implement `top_n_similar(interactions, n: int) -> np.ndarray (n_items, n)` returning, for each item, the indices of the n most cosine-similar other items (exclude the item itself; items with no interactions return -1 padding). [Likely] (Amazon's item-to-item recommendations)
    - How would you scale this to millions of items without materializing the full similarity matrix?

B. Deep learning from scratch (NumPy / PyTorch)

16. **Feed-forward network with backpropagation, then any number of layers.** In NumPy only, implement a network of `Linear` and `ReLU` layers with a softmax output: forward pass, cross-entropy loss, backward pass for every weight and bias, and SGD updates. Then refactor so the user can stack an arbitrary number of layers (`Layer.forward`, `Layer.backward`, `Network.add`). Input: `X` (N, D), `y` (N,), layer sizes, lr, epochs. Output: trained weights, the loss curve and final accuracy. [Reported] [12][13] [20] ("write a neural network and add layers" [12]; "implement back propagation without using any additional libraries" [20]; "code a ML model from scratch", junior AS [13]; prep-thread variants [61][62])
    - What is the shape of every gradient?
    - How do you verify your gradients with finite differences, and which epsilon do you choose?
    - How do you initialize weights, and what goes wrong with all-zero initialization?
    - What changes if you swap ReLU for sigmoid in a deep stack?
    - How would you add L2 weight decay and mini-batching to your abstraction?

17. **Stable softmax and cross-entropy with gradient.** Implement `softmax(z)` with no third-party libraries for inputs in [-1000, 1000], then `cross_entropy(logits: (N, C), labels: (N,)) -> (loss, dlogits)` using log-sum-exp. [Reported] [52][54][55] (Amazon online assessment requiring the max-subtraction form [55]; hand-written softmax in applied-science loops [54]; the cross-entropy formula was asked verbally [37])

    - Why does the combined gradient simplify to `(softmax - onehot) / N`?
    - How would you support an ignore index for padding, and label smoothing?
    - What happens to the loss if a predicted probability is exactly zero?

18. **Activations by hand.** Implement elementwise `sigmoid(x: np.ndarray) -> np.ndarray` with no overflow warnings for x in [-1000, 1000], then `relu`, `tanh` and `gelu` (tanh approximation), each with a `*_grad(x)` derivative checked against finite differences. [Reported] (sigmoid [52][54]); the other activations are [Likely]

    - Why is the naive `1 / (1 + exp(-x))` unsafe, and for which inputs?

19. **Training and evaluation loop.** Given a PyTorch model, a `DataLoader`, a loss function and an optimizer, implement `train_one_epoch(...)` and `evaluate(...) -> dict` (loss, accuracy, AUC), with early stopping on validation loss and best-checkpoint saving. [Reported] [52][54] (PyTorch training loop listed for Amazon; "practice a training loop from memory"); framed as "day to day ML implementation" in [23][7]

    - Where do `zero_grad`, `model.train()` and `model.eval()` go, and what breaks if you forget each?
    - Add gradient accumulation and gradient clipping by global norm.
    - Add a warmup-plus-cosine learning-rate schedule.
    - How do you make the run reproducible?

20. **2D convolution layer.** Implement the forward pass of `conv2d(x: (N, C_in, H, W), w: (C_out, C_in, kH, kW), b, stride, padding)` first with loops, then vectorized (im2col). [Reported] [18] (Blind: some AS teams ask for "a convolutional layer in a CNN")

    - What is the output shape formula?
    - What does im2col cost in memory?
    - How would you write the backward pass for the weights?

21. **Batch normalization.** Implement `batchnorm_forward(x: (N, D), gamma, beta, eps, momentum, running_mean, running_var, training: bool) -> (out, cache)` and `batchnorm_backward(dout, cache) -> (dx, dgamma, dbeta)`. [Likely] [10][14] (Blind commenters call it fair game for Amazon AS; not a first-person report)

    - How does it behave at train time vs inference time?
    - What happens with batch size 1?
    - How does it differ from layer normalization, and why do Transformers use the latter?

22. **Optimizers.** Implement plain gradient descent, momentum and Adam as `step(w, grad, state)` functions, run them on an ill-conditioned quadratic, and return each trajectory. [Likely] [32] (GD vs Adam asked verbally in a June 2026 London AS loop)

- Why does Adam need bias correction in the early steps?
- Change Adam to AdamW. Where does decoupled weight decay enter, and why is it not the same as L2 in the loss?

23. **Dropout.** Implement `dropout_forward(x: np.ndarray, p: float, training: bool, rng) -> (out, mask)` and `dropout_backward(dout, mask) -> dx` using inverted dropout, where `p` is the drop probability. Handle: `p = 0`, `p = 1`. [Likely]
    - Why scale at training time and not at inference?
24. **Recurrent cell.** Implement `lstm_step(x_t (B, D), h_prev (B, H), c_prev (B, H), W (D + H, 4H), b (4H,)) -> (h_t, c_t)` with one fused matmul for the four gates, then `lstm_forward(x (B, T, D), h0, c0, W, b) -> (h_all (B, T, H), (h_T, c_T))` that respects a per-sequence `lengths` vector. [Likely]
    - Where do vanishing gradients come from in backpropagation through time?
25. **Tiny autograd.** Build a scalar `Value(data: float)` class supporting `+`, `*`, `-`, `**` (constant exponent), `tanh` and `backward()` that fills `.grad` for every ancestor via topological sort. Test it by fitting a 2-layer MLP on four points and comparing gradients against finite differences. [Likely]
    - How do you handle a node that is used twice in the graph?
26. **Losses for imbalance.** Implement `focal_loss(logits: (N,), y: (N,), alpha: float = 0.25, gamma: float = 2.0) -> (loss, dlogits)` and `weighted_bce(logits, y, pos_weight) -> (loss, dlogits)`, computed from logits with log-sigmoid for stability (no `log(sigmoid(x))`). [Likely] [26] (imbalanced binary classification asked verbally)
    - How do alpha and gamma change the loss on easy vs hard examples?

C. NLP and LLM components

27. **Scaled dot-product attention.** Implement `attention(Q, K, V, mask=None) -> (out, weights)` for shapes (B, T, d) in NumPy or PyTorch, with a stable softmax and an optional causal or padding mask. [Reported] [52][54] (hand-written attention reported in applied-science loops; sighting July 2026); attention "at root level" and how a Transformer works were asked verbally in AS II loops [4][6]
    - Why divide by sqrt(d_k)?
    - Where exactly is the mask applied, and why use a large negative value or -inf instead of 0?
    - What are the time and memory costs in sequence length T?
    - What changes if Wq = Wk = Wv?
28. **Multi-head self-attention.** Implement `MultiHeadSelfAttention(d_model, num_heads)` with Q/K/V projections, head split, parallel attention across heads, merge and output projection. Input `x` (B, T, d_model), optional mask; output (B, T, d_model). [Likely] [37] (how heads are computed in parallel and merged was asked verbally on a personalization team)

- What happens if `d_model` is not divisible by `num_heads`?
- How would grouped-query or multi-query attention change your code?

29. **Beam search decoding.** Given `next_token_logprobs(prefix: list[int]) -> np.ndarray` of shape (V,), a beam width k, `max_len` and `eos_id`, implement `beam_search(...) -> list[tuple[list[int], float]]` returning the top-k finished sequences with cumulative log-probabilities. Stop beams that emit EOS. [Reported] [8] (Santa Clara LLM-team AS screen, 2025; the candidate did not finish working code)

    - How and why do you apply length normalization?
    - When do you stop: all beams finished, or `max_len`?
    - What is the complexity per step, and how do you avoid sorting the full k x V candidate set?
    - How does it compare with greedy decoding and with top-k/top-p sampling for an assistant?

30. **Debug a broken Transformer.** You are handed a ~100-line PyTorch decoder-only Transformer that trains without errors but whose loss plateaus far above a reference. Find and fix the bugs. (Practice: plant three bugs from the list below in your own implementation from Q27/Q28 and find them by test.) [Reported] [56] (Amazon AGI, aggregator; underlying reports not named)

    - Which bugs would you look for first (mask direction, missing scaling, wrong softmax axis, residual or norm placement, positional encoding added twice)?
    - Which sanity checks would you run (overfit a single batch, compare shapes against a reference, check that loss starts near log V)?

31. **Contrastive loss.** Implement a numerically stable, symmetric InfoNCE loss in PyTorch for a batch of paired image and text embeddings (image-to-text and text-to-image), with a temperature. [Reported] [52][53][54] (Research Scientist and MLE loops; sighting Feb 2026)

    - Where does the temperature go, and why use log-sum-exp in both directions?
    - How does batch size change the loss, and why are in-batch negatives a biased sample?

32. **RL fundamentals in code.** Implement `q_learning(env, n_episodes, alpha, gamma, epsilon, seed) -> Q (n_states, n_actions)` for a small grid world whose `env.reset() -> s` and `env.step(a) -> (s', r, done)`, with epsilon-greedy exploration; or `reinforce_loss(log_probs: (T,), rewards: (T,), gamma, baseline: (T,)) -> scalar` with discounted returns. [Reported] [56] (Amazon AGI: "reinforcement learning fundamentals + coding"); the specific algorithm is [Likely]

    - Why does a baseline reduce variance without adding bias?

33. **Sampling for generation.** Implement `sample_next(logits: (V,), temperature, top_k, top_p, rng) -> int`. Keep at least one token for top-p and handle `top_k > V`. [Likely] (sibling of Q29)

    - What do temperature 0 and very large temperature reduce to?

34. **BPE tokenizer.** Implement `train_bpe(word_freqs: dict[str, int], num_merges) -> list[tuple[str, str]]` and `encode(word, merges) -> list[str]`. [Likely] [33] (GenAI pre-training and post-training fundamentals advised for AS loops)

    - How would you make pair counting incremental instead of recounting after every merge?

35. **Word-level tokenizer class.** Implement `Tokenizer(min_freq: int, specials=("<pad>", "<unk>", "<bos>", "<eos>"))` with `fit(texts: list[str])`, `encode(text, add_special=True) -> list[int]` and `decode(ids, skip_special=True) -> str`: lowercase and NFKC-normalize, split on whitespace and punctuation, keep words with count >= `min_freq`, map the rest to `<unk>`, and make vocabulary ids deterministic under ties. [Likely]

    - How do you handle casing and Unicode normalization?

36. **KV-cache decode step.** Implement `decode_step(h_t: (B, d_model), W_q, W_k, W_v, W_o, k_cache: (B, H, T_past, d_head), v_cache: (B, H, T_past, d_head)) -> (out (B, d_model), k_cache', v_cache')` for one multi-head attention layer, and show that running it token by token reproduces full causal attention on the whole sequence. [Likely]

    - How much memory does the cache need for a given model size, batch and context length?

37. **Positional encodings.** Implement `sinusoidal_pe(T: int, d: int) -> (T, d)`, then `apply_rope(x: (B, H, T, d_head), positions: (T,), base: float = 10000.0) -> same shape` applied to Q and K (d_head even). Verify that `q_m . k_n` after RoPE depends only on `m - n`. [Likely] [53][54] (the RoPE formula was asked in Amazon science depth rounds)

    - Why does RoPE encode relative position?

38. **Normalization layers.** Implement `layernorm(x: (..., d), gamma, beta, eps=1e-5)` and `rmsnorm(x: (..., d), gamma, eps=1e-6)` normalizing over the last axis, plus the LayerNorm backward pass `-> (dx, dgamma, dbeta)`. [Likely]

    - Pre-norm or post-norm: which trains more stably, and why?

39. **LoRA layer.** Implement `LoRALinear(base: nn.Linear, r: int, alpha: float, dropout: float)` in PyTorch with frozen base weights, `A` (r, in) Gaussian-initialized and `B` (out, r) zero-initialized, forward `base(x) + (alpha / r) * B(A(dropout(x)))`, and `merge()` / `unmerge()` that fold the adapter into the base weight for inference. [Likely]

    - Which parameters are trained, and how many are there for rank r?

40. **SFT loss and collate function.** Implement `collate(batch: list[tuple[prompt_ids: list[int], response_ids: list[int]]], pad_id: int) -> dict(input_ids (B, T), attention_mask (B, T), labels (B, T))` with labels set to -100 on prompt and padding tokens, then `sft_loss(logits: (B, T, V), labels) -> scalar` with the one-position shift, averaged over response tokens in the batch. [Likely]

- Does left or right padding matter for decoder-only generation?

41. **DPO loss.** Implement `dpo_loss(pi_chosen: (B,), pi_rejected: (B,), ref_chosen: (B,), ref_rejected: (B,), beta: float) -> (loss, chosen_reward, rejected_reward)` where each input is the summed response log-probability, using `logsigmoid` for stability. [Likely] (DPO vs RLHF asked verbally in AS II loops [6])
    - What does beta control?
42. **PPO and GRPO.** Implement `ppo_clip_loss(logp_new: (N,), logp_old: (N,), advantages: (N,), clip_eps: float = 0.2) -> scalar`, then `grpo_advantages(rewards: (P, G), eps: float = 1e-8) -> (P, G)` that normalizes each prompt's G rewards by the group mean and std. [Likely] [53][54]
    - What happens when every response in a group gets the same reward?
43. **Blocked attention with online softmax.** Implement `blocked_attention(Q: (T, d), K: (T, d), V: (T, d), block: int, causal: bool) -> (T, d)` that iterates over key blocks with a running max and running sum, so the full T x T score matrix is never stored, and assert `np.allclose` against the naive result. [Likely] [54] (flash attention asked in depth rounds)
    - What extra memory does it use, and how would you write the backward pass without storing the attention matrix?

D. Metrics and evaluation code

44. **ROC AUC from scratch.** Implement `roc_auc(y_true: list[int], y_score: list[float]) -> float` without sklearn, in O(n log n) by sorting (rank / Mann-Whitney formulation or a threshold sweep with trapezoids). Tied scores count as 0.5. [Reported] [3][4][5][6][69] ("Write a program to calculate AUC", AS II, several 2025-2026 reports; "What are ROC and AUC?" asked separately [4])
    - What does your function return if all labels are one class?
    - Why is the O(n^2) pair count too slow, and how do ties break a naive sort-based version?
    - What do ROC and AUC mean, and when is PR-AUC the better choice?
    - How would you compute AUC over a stream or a dataset that does not fit on one machine (fixed-bin score histograms per class)?
    - How would you get a confidence interval for the AUC difference between two models on the same examples?
45. **Chamfer distance.** Implement `chamfer(A: (n, 3), B: (m, 3)) -> float`: the mean squared distance from each point in A to its nearest point in B, plus the same from B to A. Start with vectorized NumPy, then make it efficient. [Reported] [8] (extract "compute the chamfer distance efficiently" from Amazon AS reports; exact review not certain)
    - The O(nm) distance matrix does not fit in memory. How do you chunk it?
    - How does a KD-tree help, and how does it degrade as dimensionality grows?

46. **Percentage RMSE with a bootstrap CI.** Implement `prmse(y_true, y_pred)` and `bootstrap_ci(y_true, y_pred, metric, B=1000, alpha=0.05, seed) -> (point, low, high)` in NumPy only. [Reported] [50] (Amazon Data Scientist coding list, not AS)
    - What do you do when `y_true` contains zeros?
    - Can you vectorize the B resamples without a Python loop, and what does that cost in memory?
47. **Classification metrics.** Implement `confusion_matrix(y_true, y_pred, num_classes)` vectorized, then precision, recall and F1 with macro, micro and weighted averaging; then `best_f1_threshold(y_true, y_score)`. [Likely] [8][24] (precision, recall and AUC asked verbally in AS phone screens)
    - What do you return when a class has no predicted positives?
    - Can you find the best threshold in a single pass after sorting?
48. **PR curve and average precision.** Implement `pr_curve(y_true: (n,), y_score: (n,)) -> (precision, recall, thresholds)` with one point per distinct score (ties grouped) in O(n log n), and `average_precision(y_true, y_score) -> float` as the step-wise sum `sum((R_k - R_{k-1}) * P_k)`. Handle: no positives. [Likely]
    - Why can ROC AUC look optimistic under heavy class imbalance (fraud, abuse)?
49. **Ranking metrics.** Implement precision@k, recall@k, MAP, MRR and NDCG@k given, per query, a ranked list of item ids and a dict of graded relevance; return the mean over queries. Handle k larger than the list and queries with no relevant items (IDCG = 0). [Likely] [26][67] ("metrics for information retrieval" asked verbally)
    - Why does NDCG fit graded relevance in Amazon search better than MAP?
50. **Perplexity.** Given per-token log-probabilities (B, T) and a padding mask, return corpus-level perplexity. Then do it from a (T, V) probability array and target ids. [Likely] [8][24] (perplexity asked verbally in AS screens)
    - Why average over tokens rather than sequences?
51. **Log loss and calibration.** Implement `log_loss(y_true: (N,), probs: (N, C), eps=1e-15) -> float`, `ece(y_true: (N,), probs: (N, C), n_bins: int = 15) -> float` with equal-width confidence bins, and `fit_temperature(val_logits: (N, C), val_labels: (N,)) -> float` minimizing NLL by gradient descent on log T. [Likely]
    - Why does temperature scaling leave accuracy and AUC unchanged?
52. **IoU and NMS.** Implement `iou_matrix(a: (N, 4), b: (M, 4)) -> (N, M)` for boxes in `(x1, y1, x2, y2)` format with broadcasting, then `nms(boxes: (N, 4), scores: (N,), iou_threshold: float) -> list[int]` returning kept indices in descending score order. [Likely]
    - Which edge cases come from zero-area or non-overlapping boxes?
53. **Sequence metrics.** Implement `wer(ref: list[str], hyp: list[str]) -> (wer: float, subs: int, ins: int, dels: int)` with an edit-distance DP and backtrace, then

`corpus_bleu(refs: list[list[str]], hyps: list[list[str]], max_n: int = 4) -> float` with clipped n-gram precision and brevity penalty. Handle: an empty reference. [Likely] (Alexa ASR and translation teams)

54. **Forecasting metrics.** Implement `mape(y, yhat)`, `smape(y, yhat)` and `pinball_loss(y: (n,), yhat_q: (n,), q: float) -> float`, then evaluate P50 and P90 forecasts from a (n, 2) array, all vectorized. [Likely] (supply-chain forecasting teams)
    - What goes wrong with MAPE when actuals are zero?

E. Probability, statistics and sampling code

55. **Simulate the law of large numbers.** Implement `running_means(sampler: Callable[[int], np.ndarray], n: int, seed) -> np.ndarray (n,)` that draws n i.i.d. samples (a fair die, or Bernoulli(p)) and returns the running sample mean without a Python loop, then a check that the final mean is within a stated tolerance of the true expectation. [Reported] [9][18] (L5 AS tech phone screen [9]; "a simulation" as an applied ML-coding variant [18])
    - How does the error shrink with n, and can you show it empirically?
    - How many samples do you need for a given precision, and how do you report the uncertainty?
    - How do you seed it so the result is reproducible?
    - What happens with a heavy-tailed distribution such as Cauchy?
56. **Weighted random pick.** Implement `WeightedSampler(weights: list[float])` with `sample() -> int`, each index returned with probability proportional to its weight. [Reported] [43][52][53] (Senior AS screen, Seattle 2026 [43]; Research Scientist and MLE loops, sighting Apr 2026)
    - What is the cost per sample with prefix sums plus binary search, and can you reach O(1) (alias method)?
    - What should happen with all-zero weights, negative weights or a single element?
    - How would you test that the empirical frequencies are right?
57. **Mean and variance with MLE.** Implement `mean(x: np.ndarray, axis: int | None)` and `var(x, axis, ddof: int = 0)` without `np.mean`/`np.var` and without Python loops, ignoring NaNs if `skipna=True`, then relate them to maximum-likelihood estimation for a Gaussian. [Reported] [52][53][54] (sighting Sep 2026)
    - Which variance estimator is biased, and why do the sample variance and the Gaussian MLE differ?
    - How do you compute variance in a numerically stable way?
58. **Bootstrap CI for a statistic.** Implement `bootstrap_ci(x: np.ndarray, stat_fn, B: int = 1000, alpha: float = 0.05, seed) -> (point, low, high)` with the percentile method for the mean, then remove the explicit Python loop. (Q46 applies the same routine to a paired metric.) [Reported] [52][53] (Research Scientist and MLE screens, sighting Aug 2026)

- Which statistics does the bootstrap estimate poorly?
- Percentile interval or another variant: when does the choice matter?

59. **Weighted sampling without replacement.** Implement `weighted_sample_without_replacement(items, weights, k, rng) -> list` where each draw is proportional to the remaining weights, in O(n log k), and write a test that checks empirical frequencies. [Reported] [50] (Amazon Data Scientist coding list, not AS)

    - How would you do it on a stream of unknown length?

60. **Grid walk with probabilistic moves.** Implement `p_on_grid(m: int, n: int, start: tuple[int, int], k: int, probs: dict[str, float]) -> float`: an agent moves up/down/left/right with the given probabilities (summing to 1) and stops forever once it leaves the grid; return the probability it is still on the grid after k steps. Variant: probability of reaching a target cell within k steps. [Reported] [34] (2026 AS reports, "grid search variant with probability-weighted moves"; exact review not certain)

    - What are the time and space complexity, and can you reduce space to two layers?
    - How would you check your answer by Monte Carlo?

61. **Reservoir sampling.** Implement `reservoir_sample(stream: Iterable[T], k: int, rng) -> list[T]` returning k items sampled uniformly from a stream of unknown length in O(k) memory (all items if the stream has fewer than k). [Likely] [64]

    - Can you argue why every item ends up with probability k/n?
    - How would you extend it to weighted sampling?

62. **A/B test analysis.** Given control and treatment conversions and visitor counts, compute the lift, a two-proportion z-test p-value and a 95% CI; implement a bootstrap CI for the difference in mean revenue; then `sample_size(baseline, mde, alpha, power)` per arm. [Likely] [41][63] (statistics questions reported in depth rounds)

    - How does the required sample size change for a 1% relative lift vs a 10% one?

63. **Streaming statistics.** Implement `class RunningStats` with `update(x: float)`, `update_batch(xs: np.ndarray)`, `mean`, `variance(ddof=1)` using Welford's algorithm, and `merge(a, b) -> RunningStats` combining two shards' (count, mean, M2). [Likely]

    - Why does the naive `E[x^2] - E[x]^2` fail for values near 1e9 with small spread?

64. **Monte Carlo vs closed form.** Implement `p_collision_mc(k: int, n: int, trials: int, rng) -> (estimate, std_error)` for the probability that at least two of k shoppers pick the same item out of n uniformly, vectorized over trials, and `p_collision_exact(k, n)` computed in log space; compare them. [Likely]

65. **Bandits.** Implement `simulate(policy: "thompson" | "ucb1", true_ctr: np.ndarray (K,), horizon: int, seed) -> np.ndarray (horizon,)` returning cumulative regret for Bernoulli arms (ad creatives), with Beta(1, 1) priors for Thompson sampling. [Likely]

- How do you handle non-stationary click-through rates?

66. **CUPED.** Implement `cuped(y: (n,), x_pre: (n,), group: (n,) in {0, 1}) -> (theta, y_adj, var_reduction)` with theta estimated on pooled data, then the treatment effect and its standard error before and after adjustment. [Likely]

    - What do you do for new users with no pre-period data?

67. **Drift metrics.** Implement `psi(train: np.ndarray, serve: np.ndarray, n_bins: int = 10, eps: float = 1e-6) -> float` with bin edges from training quantiles, and `kl_divergence(p, q, eps)` on the same bins. Handle: serving values outside the training range, a constant feature. [Likely]

    - Why are both sensitive to the binning?

F. Data wrangling (pandas / SQL / pipelines)

68. **ML dataset coding round.** Given a CSV of labeled tabular data (numeric and categorical columns, missing values, a binary target with about 10% positives), use pandas, NumPy and sklearn to clean it (missing values, categorical encoding), engineer features, split train/test or cross-validate, train a baseline, and report precision, recall and F1, explaining each choice as you go. [Reported] [13][42] (junior AS loop with "ML dataset coding (pandas, numpy, sklearn)" [13]; London AS 2024: load, clean, feature-engineer and fit [42])

    - Which of your steps can leak information from the test set?
    - Why that imputation and that encoding for a high-cardinality categorical?
    - How would you pick a decision threshold for the business metric?

69. **Python data wrangling.** Given `orders: list[dict(order_id, customer_id, amount, ts)]` and `customers: list[dict(customer_id, region)]`, return revenue per region and the top-3 customers by spend per region, once in plain Python (no pandas) and once in pandas. [Reported] [72]

    - How would you do the same join if neither table fits in memory?

70. **Rolling average with missing dates.** Implement `rolling_avg(rows: list[tuple[date, float]], window: int = 7) -> list[tuple[date, float]]` over a calendar window (not a row window), returning one value per calendar day from the first to the last date, given unsorted input with gaps and possible duplicate dates. [Reported] [16] (L5 AS virtual onsite)

    - Should missing days count as zero or be skipped? Defend your choice.

71. **Org-tree aggregation.** Given `rows: list[tuple[employee_id, manager_id | None, salary]]`, implement `avg_direct_report_salary(rows) -> dict[manager_id, float]`. [Reported] [53][54] (Research Scientist and MLE loops)

    - What do you return for a manager with no reports, and how would you average over all (not only direct) reports in one pass?

    - What if the input is a nested `Employee(id, salary, reports: list[Employee])` tree instead, and how do you avoid recursion-depth limits?

72. **Ad clicks in the last k minutes.** Implement `ClickCounter(window_seconds: int)` with `record(ad_id: str, ts: int)` (timestamps non-decreasing) and `count(ad_id, now: int) -> int` returning clicks in `(now - window, now]`, amortized O(1) per call. [Reported] [53][54] (MLE and Research Scientist screens)

    - How does memory grow with the number of ads, and how do you expire idle ads?
    - What if events arrive out of order?

73. **Day-to-day feature pipeline.** Given a DataFrame of events (`customer_id`, `item_id`, `timestamp`, `event_type`, `price`), build per-customer features as of a label date (purchases in the last 30 days, average price, days since last event, rolling 7-day activity), compute the label "purchased within the next N days", and make a time-based train/test split with no leakage. Return a feature DataFrame keyed by `customer_id`. [Likely] [7][23] (the "day to day ML implementations" round was reported; the prompt is inferred)

    - Where exactly can future data leak in, and how does your code prevent it?
    - How would you make it run on a year of events across all customers (Spark, incremental)?

74. **Crowdsourced label quality.** Given a DataFrame (`item_id`, `annotator_id`, `label`, `timestamp`) with categorical labels and a variable number of annotators per item, compute inter-annotator agreement, each annotator's accuracy against the majority vote, and label entropy per item, then filter a high-quality subset. [Likely] [8] (asked as a design question in a Bellevue 2025 onsite)

75. **SQL.** Given `orders(order_id, customer_id, order_date, amount)` and `returns(order_id, return_date)`, return per month the active customers, the return rate and a 7-day rolling average revenue; then each customer's second purchase date with window functions. [Likely] [65]

76. **Splits.** Implement `group_stratified_split(df, group_col: str, label_col: str, fractions=(0.8, 0.1, 0.1), seed) -> (train_idx, val_idx, test_idx)` so no group appears in two splits and each split's label rate is within a tolerance of the overall rate. [Likely]

77. **Preprocessing classes.** Implement `StandardScaler` with `fit(X)`, `transform(X)`, `inverse_transform(X)` (zero-variance columns scale by 1) and `OneHotEncoder(handle_unknown="ignore" | "error", min_frequency: int)` with `fit(cols: list[list[str]])` and `transform -> np.ndarray` (unseen categories become all-zero rows). [Likely]

78. **Out-of-core batching.** Implement `batch_iter(shard_paths: list[str], batch_size: int, buffer_size: int, seed) -> Iterator[np.ndarray]` that shuffles shard order each epoch, streams rows through a bounded shuffle buffer and yields fixed-size batches (last batch may be short). [Likely]

- How do you approximate a full shuffle with bounded memory?

G. DSA problems seen in AS loops

This section lists problems tied to Amazon AS loops. For the wider Amazon DSA list, see `amazon-l5-applied-scientist/chapters/04-coding-dsa.md`.

79. **String compression.** Run-length encode a string (`"aabccccccaaa"` becomes `"a2b1c5a3"`), returning the original if the result is not shorter; variant: compress a char array in place and return the new length. O(n) time. [Reported] [6][22][25] (AS II Seattle 2026; Hyderabad AS 2025-2026; AS II June 2026)
    - How do you handle counts of 10 or more in the in-place variant?
    - Can you do it in O(1) extra space?
80. **Number of islands.** Count 4-connected islands in a grid of `'1'` and `'0'`. [Reported] [27][28] (London AS phone screen; 45-minute Python coding round, Jan 2025)
    - What changes for a very large grid (iterative BFS, union-find), or one streamed row by row?
81. **Two Sum.** Return the indices of two numbers that sum to a target, in O(n). [Reported] [24][26][40] (AS II NLP phone screen [26]; AS online coding [40])
    - What if the array is sorted, or you must return all unique pairs?
82. **Median of a data stream.** Implement `addNum(int)` and `findMedian()` with O(log n) insert. [Reported] [29] (Luxembourg AS; "Leetcode Medium or Hard")
    - What if all numbers are in [0, 100]?
    - How would you support a sliding-window median?
83. **2D binary search.** Search a target in a matrix where each row is sorted and each row starts after the previous row ends, in O(log(mn)); follow-up: rows and columns sorted independently. [Reported] [8][24] (AS phone screens, with ML breadth on precision, recall, AUC and perplexity)
    - Can you compare the staircase search with binary search per row?
84. **Rotate a matrix.** Rotate an n x n matrix 90 degrees clockwise in place with O(1) extra space. [Reported] [35][36] (Cambridge AS screen; 2024 AS report)
    - How would you rotate a non-square image tensor with NumPy?
85. **Merge intervals.** Merge overlapping [start, end] intervals. [Reported] [30]
    - Should touching intervals merge? How would you insert one interval into an already merged list?
86. **Batching to minimize padding cost.** Split an ordered list of sequences into contiguous batches, where each batch costs a fixed overhead plus padding up to its

longest item, so total cost is minimal. Give the DP state and recurrence. [Reported] [52][53] (Applied Scientist phone screen, sighting Apr 2026)

    - What is the complexity, and can bounding the batch size speed it up?
    - How would a greedy bucketing heuristic compare for an inference server?

87. **Edit distance.** Compute the Levenshtein distance between two strings. [Reported] [44] (Cambridge AS, 2020: "a coding problem similar to edit distance")

    - Can you reduce memory to O(min(m, n))?

88. **Search insert position.** Return the index of the target in a sorted array, or where it would be inserted, in O(log n) without `bisect`. [Reported] [47] (AS live coding, PracHub)

89. **A DP medium.** A Blind-150-style DP problem; practice prompt: `coin_change(coins: list[int], amount: int) -> int` returning the minimum number of coins (-1 if impossible), then `count_ways(coins, amount) -> int` counting combinations (order does not matter). [Reported] [23][33] (the specific problem was not given)

    - Can you write the recurrence first, then the bottom-up version, then O(B) memory?

90. **Count inversions.** Implement `count_inversions(a: list[int]) -> int`, the number of pairs i < j with a[i] > a[j], in O(n log n) (merge sort or a Fenwick tree over ranks). [Reported] [26]

91. **A graph problem in the screen.** Practice prompt: given `n` nodes and `edges: list[tuple[int, int]]`, implement `count_components(n, edges) -> int`, `has_cycle_directed(n, edges) -> bool` and `shortest_path_unweighted(n, edges, src, dst) -> list[int]`. [Reported] [49] (Senior AS tech screen, Dec 2025; the exact problem was not given; the candidate did not finish and still passed)

    - Which traversal fits, and how do you show the rest of the solution if you run out of time?

92. **Two-pointer problem, then 3Sum.** Implement `two_sum_sorted(a: list[int], target) -> tuple[int, int] | None` with two pointers, then `three_sum(a: list[int], target: int) -> list[tuple[int, int, int]]` returning all unique value triplets in O(n^2). [Reported] [45] (Sr. AS L6 phone screen, 2022)

93. **Implement a data structure.** `LRUCache(capacity)` with O(1) `get(key) -> int` (-1 if missing) and `put(key, value)`, then a `MinHeap` with `push`, `pop` and `heapify(list)` without `heapq`; state the cost of each operation. (Trie: see Q94.) [Reported] [31] (2021 AS: "4-5 questions on implementation of some data structures"; the specific structures are illustrative)

94. **Trie autocomplete.** Implement a trie with `insert`, `search` and `starts_with`, then `suggest(products: list[str], query: str) -> list[list[str]]` returning the three lexicographically smallest product names for each typed prefix of `query`. [Likely] [66]

(a cross-company Applied Scientist results page mentions a medium trie problem; it could not be tied to one Amazon review)

95. **LCA in a BST.** Return the lowest common ancestor of two nodes in O(h). [Reported] [46] (new-grad AS loop, L4)
96. **Course Schedule III.** Given courses as [duration, lastDay], return the maximum number you can take. [Reported] [46] (new-grad AS loop, L4, with a disguised description)
97. **Fruits Into Baskets III.** Place each fruit type into the leftmost unused basket with enough capacity and count unplaced fruits, faster than O(n^2). [Reported] [46] (new-grad AS loop, L4)
98. **Basic Calculator II.** Implement `calculate(s: str) -> int` for non-negative integers, `+ - * /` (division truncates toward zero) and spaces, without `eval`, in O(n). [Likely] [48] (Amazon-tagged on PracHub, but the role is not shown, so not tied to an AS loop)
99. **My Calendar III.** Implement `book(start, end)` returning the maximum overlap so far, then extend it to cancellations. [Reported] [50] (Amazon Data Scientist coding list, not AS)

H. Level calibration: L5 probes

The bank above is broad, but many prompts stop at a junior bar (write the loop, get it right). An L5 Applied Scientist is expected to go further unprompted: vectorize, prove numerical safety, state complexity, test, and extend. Use these harder variants after the base version of each prompt. All are [Likely]: they are inferred from the L5 evaluation criteria above [17][53][54], not from a specific report.

100. **AUC at scale (extends Q44).** Implement `streaming_auc(batches: Iterable[tuple[y (b,), score (b,)]], n_bins: int) -> float` using per-class score histograms, bound its error versus exact AUC as a function of `n_bins`, then `merge` histograms from many workers. Write a property test against your exact O(n log n) version on random data with heavy ties. [Likely]
101. **DeLong-free AUC comparison (extends Q44, Q58).** Implement `paired_bootstrap_auc_diff(y, s_a, s_b, B, seed) -> (diff, low, high)` vectorized over B with a single argsort per resample, and explain why resampling rows independently is wrong when rows cluster by user. [Likely]
102. **Fused, stable softmax-cross-entropy (extends Q17).** Implement forward and backward for logits (N, C) with C = 1e5 in float16 inputs: accumulate in float32, never materialize the full softmax for the loss, support `ignore_index` and label smoothing, and write a finite-difference gradient check that passes in float64. [Likely]
103. **k-means on 1e9 rows (extends Q1).** Implement `minibatch_kmeans(batch_iter, k, n_steps, seed)` with per-centroid learning rates `1/count`, k-means|| style seeding from a sample, and empty-cluster reseeding from the farthest points; state the memory bound and how you would shard it across workers. [Likely]
104. **Logistic regression that cannot diverge (extends Q2).** Add L2, class weights and a Newton/IRLS solver using `np.linalg.solve` (no `inv`); detect separable data and show

what happens to `||w||` with and without L2. Compare iterations-to-converge with GD. [Likely]

105. **Attention with memory bounds (extends Q27, Q43).** Implement causal attention for T = 32k with a padding mask in which some rows are entirely masked: return zeros, not NaN, for those rows, never allocate T x T, and assert equivalence with the naive version on small T. [Likely]
106. **Batched beam search (extends Q29).** Vectorize beam search over a batch of B prompts with a `(B, k)` beam state, using `argpartition/topk` on `(B, k*V)`, length penalty, early stopping per batch element, and a test that k = 1 reproduces greedy decoding. [Likely]
107. **Weighted sampler, production version (extends Q56).** Implement the alias method in O(n) build and O(1) sample, support `update(i, w)` in O(log n) with a Fenwick tree, and write a chi-square test of the empirical frequencies with a fixed seed. [Likely]
108. **Gradient check harness (extends Q16, Q20, Q21).** Implement `grad_check(f, params: dict[str, np.ndarray], eps=1e-6) -> dict[str, float]` returning relative error per parameter using central differences on a random subset of entries; use it to catch a planted bug in your batchnorm backward. [Likely]
109. **Leak-proof feature pipeline (extends Q73, Q76).** Given the Q73 events table, write unit tests that fail if any feature uses an event with `timestamp >= label_date`, and make the pipeline produce identical features whether computed in one batch or incrementally day by day. [Likely]
110. **Variance at extremes (extends Q57, Q63).** Show with a test that the naive two-pass variance on float32 data around 1e8 loses all precision, then fix it with Welford or a shifted two-pass and with pairwise summation; state the error bounds informally. [Likely]
111. **Profile and speed up (any prompt).** Take your pure-Python k-NN or Chamfer solution (Q8, Q45), state its complexity, then rewrite it to be chunked and vectorized with memory capped at M bytes, and give the chunk size as a function of M, n, m and d. [Likely]

References

1. Applied Scientist Interview Prep, amazon.jobs (Amazon, official), n.d. https://amazon.jobs/content/en/how-we-hire/applied-scientist-interview-prep
2. SDE II Interview Prep (Livecode editor, syntactically correct code), amazon.jobs (Amazon, official), n.d. https://amazon.jobs/content/en/how-we-hire/sde-ii-interview-prep
3. Amazon interview review RVW102987402 (Applied Scientist II, Santa Monica; AUC; two easy LeetCode plus ML coding), Glassdoor, 2026 (read from search extract; snippet-to-review mapping approximate). https://www.glassdoor.ie/Interview/Amazon-Interview-E6036-RVW102987402.htm
4. Amazon interview review RVW66719970 (Applied Scientist II; AUC; logistic regression loss and MLE; attention), Glassdoor, 2024-2025 (search extract). https://www.glassdoor.com.au/Interview/Amazon-Interview-E6036-RVW66719970.htm
5. Amazon interview review RVW95575263 (Applied Scientist II; code AUC from scores and labels), Glassdoor, 2026. https://www.glassdoor.fr/Entretien/Amazon-Entretien-E6

036-RVW95575263.htm
6. Amazon interview review RVW96903377 (Applied Scientist II, Seattle; AUC; string compression; attention; bagging vs boosting; DPO vs RLHF), Glassdoor, 2026 (search extract). https://www.glassdoor.fr/Entretien/Amazon-Entretien-E6036-RVW96903377.htm
7. Amazon interview review RVW95289060 (Applied Scientist, Berlin; k-means in Python; also cited for a 2026 "day to day ML implementations" loop), Glassdoor, 2025-2026 (search extract; mapping approximate). https://www.glassdoor.fr/Entretien/Amazon-Entretien-E6036-RVW95289060.htm
8. Amazon interview review RVW96068026 (Applied Scientist II, LLM team, Santa Clara: beam search; Bellevue: 2D binary search, precision/recall/AUC/perplexity, crowdsourced-data metrics, Chamfer distance), Glassdoor, 2025-2026 (search extract; mapping approximate). https://www.glassdoor.com.hk/Interview/Amazon-Interview-E6036-RVW96068026.htm
9. "Amazon L5 Applied Scientist Machine Learning Engineering Tech Phone Screen Experience" (law-of-large-numbers simulation), 1point3acres, 2024-2025. https://www.1point3acres.com/interview/thread/1098917
10. "Tips for coding interview, Applied Scientist at Amazon" (OLS, gradient descent, batch norm fair game; not bagging/boosting), Blind, 2021-2024. https://www.teamblind.com/post/tips-for-coding-interview-applied-scientist-at-amazon-txexmtsb
11. "Amazon applied science coding interview" (AS interviewers ask PCA and least squares), Blind, 2021-2024. https://www.teamblind.com/post/amazon-applied-science-coding-interview-onryzukx
12. "Amazon applied scientist" (asked to write a neural network and add layers), Blind, 2021-2024. https://www.teamblind.com/post/amazon-applied-scientist-6m0hwyih
13. "Amazon Jr Applied Scientist" (ML dataset coding with pandas, NumPy, sklearn; code an ML model from scratch), Blind, 2022. https://www.teamblind.com/post/amazon-jr-applied-scientist-zan6avr8
14. "Amazon applied scientist interview" (OLS, gradient descent, batch normalization fair to ask), Blind, n.d. https://www.teamblind.com/post/amazon-applied-scientist-interview-hn6cwts0
15. "Amazon applied scientist interview loops" (logistic regression with gradient descent, k-means), Blind, n.d. https://www.teamblind.com/post/amazon-applied-scientist-interview-loops-lucdk0uq
16. "Amazon Applied Scientist (L5) Virtual Onsite" (one coding round; rolling average with missing dates), Blind, circa 2025. https://www.teamblind.com/post/amazon-applied-scientist-l5-virtual-onsite-navwesml
17. "Applied scientist interview Amazon" (LeetCode at L4 SWE bar; correctness first, complexities), Blind, 2021-2025. https://www.teamblind.com/post/applied-scientist-interview-amazon-4udsobcp
18. "Amazon applied scientist loop" (simulation or a CNN convolutional layer as applied ML coding), Blind, n.d. (search extract). https://www.teamblind.com/post/amazon-applied-scientist-loop-odlzgrp3
19. Amazon interview review RVW81277838 (Applied Scientist Intern; implement gradient descent from scratch), Glassdoor, n.d. (search extract; mapping approximate). https://fr.glassdoor.be/Entretien/Amazon-Entretien-E6036-RVW81277838.htm

20. Amazon interview review RVW79806572 (implement backpropagation without libraries), Glassdoor, n.d. (search extract; mapping approximate). https://fr.glassdoor.ch/Entretien/Amazon-Entretien-E6036-RVW79806572.htm
21. Amazon interview review RVW40731696 (implement SGD: derive gradients, update, momentum), Glassdoor, n.d. (search extract; mapping approximate). https://fr.glassdoor.ca/Entretien/Amazon-Entretien-E6036-RVW40731696.htm
22. Amazon interview review RVW98336019 (Applied Scientist, Hyderabad; phone screen coding a traditional ML classifier; string compression; four technical rounds), Glassdoor, 2025-2026 (search extract; mapping approximate). https://www.glassdoor.es/Entrevista/Amazon-Entrevista-E6036-RVW98336019.htm
23. Amazon interview review RVW101665167 (Applied Scientist; Melbourne Nov 2025 one ML plus one LeetCode coding question; DP medium; 2026 "day to day ML implementations" loop), Glassdoor, 2025-2026 (search extract; mapping approximate). https://www.glassdoor.fr/Entretien/Amazon-Entretien-E6036-RVW101665167.htm
24. Amazon interview review RVW47917252 (Applied Scientist; code a traditional ML classifier; 2D binary search; precision, recall, AUC, perplexity), Glassdoor, 2020-2026 (search extract). https://www.glassdoor.co.in/Interview/Amazon-Interview-E6036-RVW47917252.htm
25. Amazon interview review RVW101785177 (Applied Scientist II; bias-variance, bagging vs boosting, string compression), Glassdoor, 2026 (search extract). https://www.glassdoor.com.mx/Entrevista/Amazon-Entrevista-E6036-RVW101785177.htm
26. Amazon interview review RVW32314669 (Applied Scientist II NLP phone screen: Two Sum; counting inversions; IR metrics; imbalanced logistic regression), Glassdoor, n.d. https://www.glassdoor.com/Interview/Amazon-Interview-E6036-RVW32314669.htm
27. Amazon interview review RVW75899476 (Applied Scientist, London; number of islands; linear regression closed form and inversion cost), Glassdoor, n.d. https://www.glassdoor.com.hk/Interview/Amazon-Interview-E6036-RVW75899476.htm
28. Amazon interview review RVW96647342 (45-minute Python coding round; number of islands), Glassdoor, 2025. https://www.glassdoor.co.in/Interview/Amazon-Interview-E6036-RVW96647342.htm
29. Amazon interview review RVW99133644 (Applied Scientist, Luxembourg; median from data stream), Glassdoor, 2023-2026. https://www.glassdoor.sg/Interview/Amazon-Interview-E6036-RVW99133644.htm
30. Amazon interview review RVW58753536 (merge intervals as the medium question), Glassdoor, n.d. (search extract). https://www.glassdoor.fr/Entretien/Amazon-Entretien-E6036-RVW58753536.htm
31. Amazon interview review RVW36269508 (Applied Scientist; implementation of data structures), Glassdoor, 2021. https://www.glassdoor.fr/Entretien/Amazon-Entretien-E6036-RVW36269508.htm
32. Amazon interview review RVW104768247 (Applied Scientist, London area; gradient descent vs Adam; 60-minute rounds), Glassdoor, 2026 (search extract). https://fr.glassdoor.ca/Entretien/Amazon-Entretien-E6036-RVW104768247.htm
33. Amazon interview review RVW94715488 (DP problem from Blind 150; GenAI fundamentals), Glassdoor, 2025 (search extract). https://www.glassdoor.fr/Entretien/Amazon-Entretien-E6036-RVW94715488.htm
34. Amazon interview review RVW91266671 (Applied Scientist, Seattle; grid problem with probability-weighted moves), Glassdoor, 2026 (search extract; mapping

approximate). https://www.glassdoor.com.hk/Interview/Amazon-Interview-E6036-RVW91266671.htm
35. Amazon interview review RVW65272345 (Applied Scientist, Cambridge UK; matrix rotation), Glassdoor, 2024-2025 (search extract). https://www.glassdoor.es/Entrevista/Amazon-Entrevista-E6036-RVW65272345.htm
36. Amazon interview review RVW89381532 (Applied Scientist; rotate matrix 90 degrees), Glassdoor, 2024. https://www.glassdoor.fr/Entretien/Amazon-Entretien-E6036-RVW89381532.htm
37. Amazon interview review RVW87352113 (Applied Scientist, personalization; multi-head attention in parallel; cross-entropy formula), Glassdoor, n.d. (search extract; mapping approximate). https://www.glassdoor.es/Entrevista/Amazon-Entrevista-E6036-RVW87352113.htm
38. Amazon interview review RVW9853746 (logistic regression assumptions and loss; recommendation system), Glassdoor, n.d. https://fr.glassdoor.ca/Entretien/Amazon-Entretien-E6036-RVW9853746.htm
39. Amazon interview review RVW92098279 (k-means distance metric; features with different dynamic ranges), Glassdoor, n.d. https://fr.glassdoor.ca/Entretien/Amazon-Entretien-E6036-RVW92098279.htm
40. Amazon interview review RVW22038746 (Applied Scientist online coding: Two Sum), Glassdoor, 2018-2019. https://www.glassdoor.fr/Entretien/Amazon-Entretien-E6036-RVW22038746.htm
41. Amazon interview review RVW71908933 (statistics questions in depth rounds), Glassdoor, n.d. https://fr.glassdoor.ch/Entretien/Amazon-Entretien-E6036-RVW71908933.htm
42. Amazon interview review RVW99272421 (Applied Scientist, London, July 2024; pandas load, clean, feature-engineer and fit), Glassdoor, 2024. https://static.glassdoor.it/Interview/Amazon-Interview-E6036-RVW99272421.htm
43. Amazon interview review RVW102577344 (Senior Applied Scientist, Seattle; weighted random number generator), Glassdoor, 2026. https://www.glassdoor.co.nz/Interview/Amazon-Interview-E6036-RVW102577344.htm
44. "Amazon | Applied/Research Scientist | Cambridge UK | May 2020 [Reject]" (coding problem similar to edit distance), LeetCode Discuss, 2020. https://leetcode.com/discuss/interview-experience/658771/amazon-appliedresearch-scientist-cambridge-uk-may-2020-reject
45. "Amazon | Sr. Applied Scientist L6 | Phone Screen | June 2022" (three-sum follow-up), LeetCode Discuss, 2022. https://leetcode.com/discuss/interview-experience/2221480/
46. "Amazon new grad Applied Scientist interview experience: five virtual onsite rounds, L4 offer" (LCA in BST, LC 3479, Course Schedule III), PracHub, 2025. https://prachub.com/interview-experiences/amazon-new-grad-applied-scientist-interview-experience-five-virtual-onsite-rounds-mostly-behavioral-end-in-an-l4-offer
47. Amazon Interview Questions, page 15 (AS live coding: Search Insert Position without helper libraries), PracHub, 2025. https://prachub.com/companies/amazon?page=15
48. Amazon Interview Questions, page 28 (Basic Calculator II; role not shown), PracHub, n.d. https://prachub.com/companies/amazon?page=28
49. "Amazon Senior Applied Scientist Interview Experience: Passed the Tech Screen Despite an Unfinished Graph Question", PracHub, December 2025 (search extract). https://prachub.com/interview-experiences/amazon-senior-applied-scientist-interview-experience-passed-the-tech-screen-despite-an-unfinished-graph-question

50. Amazon Data Scientist Coding & Algorithms Interview Questions (weighted sampling without replacement; percentage RMSE with bootstrap CI; My Calendar III), PracHub, 2025. https://prachub.com/companies/amazon/positions/data-scientist/categories/coding-and-algorithms
51. "Amazon Applied Scientist Onsite Interview Experience: Machine Learning Focus" (two-day onsite: coding, ML system design, LPs), 1point3acres, n.d. (title and summary only). https://www.1point3acres.com/interview/thread/1020600
52. "Amazon interview process, OA & interview questions" (dated sightings synced from trueinterview.io candidate reports), kevin-2023-code/Tech-Interview-Questions, GitHub (aggregator), 2026. https://github.com/kevin-2023-code/Tech-Interview-Questions/blob/main/companies/amazon.md
53. "Amazon Research Scientist Interview Process" (aggregator adapted from trueinterview.io; science loop format shared with AS), kevin-2023-code/Tech-Interview-Questions, GitHub, 2026. https://github.com/kevin-2023-code/Tech-Interview-Questions/blob/main/companies/amazon/interview-process-research-scientist.md
54. "Amazon Machine Learning Engineer Interview Process" (aggregator; includes applied-science loops with hand-written sigmoid, softmax and attention), kevin-2023-code/Tech-Interview-Questions, GitHub, 2026. https://github.com/kevin-2023-code/Tech-Interview-Questions/blob/main/companies/amazon/interview-process-machine-learning-engineer.md
55. "Implement Softmax" (Amazon online assessment; no third-party libraries), kevin-2023-code/Tech-Interview-Questions, GitHub (aggregator), 2026. https://github.com/kevin-2023-code/Tech-Interview-Questions/blob/main/questions/algorithm/implement-softmax/README.md
56. "AI Companies Interview Questions", Amazon AGI section (debug a broken Transformer; build/train a small classifier live; RL fundamentals + coding), ombharatiya/FAANG-Coding-Interview-Questions, GitHub (aggregator), 2026. https://github.com/ombharatiya/FAANG-Coding-Interview-Questions/blob/main/AI-Companies-Interview-Questions.md
57. "Implement decision tree from scratch" (Amazon-tagged MLE onsite practice listing), Codemia, 2025. https://codemia.io/practice/implement-decision-tree-from-scratch-10
58. "Implement k-means from scratch" (Amazon-tagged MLE onsite practice listing), Codemia, 2025. https://codemia.io/practice/implement-k-means-from-scratch
59. "Implement naive Bayes from scratch" (practice listing; not Amazon-attributed), Codemia, 2025. https://codemia.io/practice/implement-naive-bayes-from-scratch-13
60. "Implement logistic regression from scratch" (practice listing; attributed to other companies), Codemia, n.d. https://codemia.io/practice/implement-logistic-regression-from-scratch-8
61. "What to expect in ML coding interview?" (#amazon), Blind, n.d. https://www.teamblind.com/post/ml-coding-interview-kq506p3t
62. "Amazon applied scientist role interview prep", Blind, n.d. https://www.teamblind.com/post/amazon-applied-scientist-role-interview-prep-tz8wfbdk
63. "Amazon Applied Scientist Interview Guide, Alexa International Tech", Medium, n.d. https://medium.com/interview-preparation/amazon-applied-scientist-interview-guide-alexa-international-tech-022d3a056feb
64. theprogclub, "Amazon Applied Scientist Interview Experience", Medium, n.d. https://medium.com/@theprogclub/amazon-applied-scientist-interview-experience-e38e3fb2ec48

65. "The 2026 Amazon Applied Scientist Interview Guide", Interview Kickstart (vendor guide), 2026. https://interviewkickstart.com/interview-questions/companies/amazon/applied-scientist
66. Applied Scientist Interview Questions, page 67 (medium Trie problem among Amazon reports), Glassdoor, 2025. https://static.glassdoor.com.mx/Interview/applied-scientist-interview-questions-SRCH_KO0,17_IP67.htm
67. Applied Scientist Interview Questions, page 81 (metrics for information retrieval), Glassdoor, n.d. https://static.glassdoor.es/Interview/applied-scientist-interview-questions-SRCH_KO0,17_IP81.htm
68. "Implement K-Means in NumPy or PyTorch" (vectorized, GPU; tied to another company), 1point3acres, 2025. https://www.1point3acres.com/interview/post/7100456
69. "Amazon Applied Scientist Interview Questions & Guide 2026" (vendor guide; k-means; "write a program to calculate AUC"), Dataford, 2026. https://dataford.io/interview-guides/amazon/applied-scientist
70. "Amazon Applied Scientist Interview Debrief: Five-Round Loop + LP" (vendor-published debrief; gradient descent without libraries; LP time in technical rounds), OA VO Service, n.d. https://oavoservice.com/en/articles/amazon-applied-scientist-vo-loop-lp-ml-coding
71. "Amazon Coding Interview Questions: The Real Bar (2026 Guide)" (code usually not executed), copilotinterview.com, 2026. https://copilotinterview.com/blog/amazon-coding-interview-questions
72. Amazon Applied Scientist II Interview Experience & Questions (Python data wrangling alongside DSA), Glassdoor, 2025-2026. https://www.glassdoor.com/Interview/Amazon-Applied-Scientist-II-Interview-Questions-EI_IE6036.0,6_KO7,27.htm
73. Amazon Applied Scientist Interview Experience & Questions (SDE vs AS interviewer coding style), Glassdoor, 2020-2026. https://www.glassdoor.com/Interview/Amazon-Applied-Scientist-Interview-Questions-EI_IE6036.0,6_KO7,24.htm
74. "Amazon L5 Applied Scientist Fulltime Tech Phone Screen Overview", 1point3acres, n.d. https://www.1point3acres.com/interview/thread/1170176

02. Amazon L5 Applied Scientist: Practical ML System Design

> Scope: the ML system design round (also called "Science Application" or "Sys Design (Team-specific)") in the Amazon L5 Applied Scientist (Applied Scientist II) loop, 2024-2026. This chapter contains questions only. It gives no answers or solutions.
>
> How it relates to the older chapters: this chapter merges and dedupes four new search passes (October 2026) with the prompts in `amazon-l5-applied-scientist/chapters/06-ml-system-design.md`, `12-ml-system-design-reference-diagrams.md` and `13-tagging-and-document-understanding-system-design.md`. Those chapters keep the long tail: SCOT buying, payments fallback, Alexa accents, enterprise RAG, document understanding and the full attribute-extraction deep dive. Each prompt here gets a skeleton diagram listing the boxes a strong answer must cover. The boxes are deliberately left empty: they name components, not choices.
>
> Source caveat: almost every source is an anonymous self-report on Glassdoor, Blind, Reddit or LeetCode, or a prep-vendor page. Many were read only as search-result extracts. Some Glassdoor review IDs were matched to the right report only approximately (noted in the references). Read each [Reported] tag as "a candidate said this was asked", not as verified fact.

Tag legend: **[Reported]** means a cited source says the prompt, or a close paraphrase, was asked in an Amazon science interview. **[Likely]** means a grounded inference from Amazon job postings, prep guides or the hiring team's products, not seen verbatim in a candidate report. Follow-up bullets are untagged unless a source reports them. They are the probes interviewers typically pull on.

How this round works

- **Where it sits in the loop.** Recent AS II loops are a phone screen plus 4-5 onsite interviews of about 55-60 minutes each. Seattle 2026 reports give the loop as "1 Science ML Breadth, 1 Science Depth, 1 Sys Design (Team-specific), 1 coding round and 1 Bar Raiser" [2][3][4]. A Luxembourg loop was a tech talk plus Coding, Science Depth, Science Breadth and Science Application [11]. An August 2024 AS2 loop had 2 phone screens and 5 onsites, one of them a business-problem ML design round [1]. Some recent loops had two practical ML design rounds "related to day to day ml implementations" rather than LeetCode [6][26].
- **What kind of design.** This is ML system design, not software architecture. A Berlin candidate found it "higher-level than classic software design", focused on recognizing and applying standard or recent ML approaches [12]. The Alexa round explicitly asked for "ML system design, not software design": datasets, task framing, model

choice with justification, and evaluation metrics [1]. Interviewers still ask about latency, scaling, serving, drift monitoring and rollout [22][40].

- **Where the prompt comes from.** Usually from the hiring team's own work. A Blind AS2 thread describes the round as taking the team's business problem, framing it as a science problem, and defining data, model, metrics and the learning process [16]. In 2025-2026, GenAI and agentic prompts became common: agentic recommenders, a Copilot-style agent, LLM training and decoding [2][3][9][24][25].
- **Format and tools.** Expect a verbal walkthrough on a shared virtual whiteboard or document, with no code to run. The sources do not name a specific tool. The interviewer interrupts with follow-ups after the first end-to-end pass [10]. Plan for about 40 minutes of design and 10-15 minutes of Leadership Principle questions. Amazon interviewers each own one or two LPs, often Customer Obsession, Dive Deep, Invent and Simplify, and Think Big.
- **What the L5 bar is.** You must own the whole problem end to end: scope a vague business goal, choose data and labels, justify the model against alternatives, pair offline metrics with an online (Weblab) metric, and plan deployment, monitoring and iteration [16][29]. Depth probes test why you chose each component and whether you know the alternatives. One candidate was down-leveled partly for lacking production experience and for narrowing the problem too much [13]. Breadth questions such as bias-variance and bagging versus boosting can be woven into the design round [2][4].
- **Out of scope here.** ML coding (for example ROC AUC from scratch, or a CNN layer forward pass [2][15]) is covered in the ML coding chapter of this focus guide.

Top 15 to practice first

Reported prompts come first, ordered by how many independent reports mention them.

1. Design an agentic recommendation system for Amazon shopping, and evaluate its quality and safety. [Reported] [2][3] (see Q1)
2. Team-specific design: turn the hiring team's business problem into a science problem, end to end. [Reported] [3][4][12][16] (see Q2)
3. Design an end-to-end pipeline that translates multimodal catalog objects into another language. [Reported] [5][6][7][8] (see Q3)
4. Design Alexa from scratch as an ML system. [Reported] [1] (see Q4)
5. Summarize product reviews into key concepts for the product page. [Reported] [3] [11] (see Q5)
6. Design and evaluate a product recommendation model or engine for Amazon. [Reported] [10][20][21] (see Q6)
7. Frame an open-ended ML product problem: scope, data, domain adaptation, distillation, metrics. [Reported] [13] (see Q7)
8. Design a spam (or fake-review) detection system. [Reported] [17] (see Q8)
9. Classify short noisy text: the language of reviews, or tweets into categories. [Reported] [10][18][19][20] (see Q9)
10. Design a GitHub Copilot-style coding agent. [Reported] [9] (see Q10)
11. Design a system for global request detection and labeling. [Reported] [22] (see Q11)
12. Design product search ranking, including LLM-powered search. [Likely; LLM-search variant Reported] [14][24][35] (see Q12)

13. Design a Rufus-style LLM shopping assistant. [Likely; DPO and decoding probes Reported] [24][25] (see Q13)
14. Design CTR/CVR prediction and placement for Sponsored Products ads. [Likely] [42] [43][46] (see Q14)
15. A production model is degrading: diagnose, retrain and safely redeploy it. [Likely] [26][40] (see Q22)

---

1. Design an agentic recommendation system

Prompt. "Design an agentic recommendation system (for example, for Amazon shopping). How would you structure the system, what models or components would you use, and how would you evaluate its quality and safety?" [Reported] [2][3] (Seattle, May-June 2026; the later rounds of that loop focused on agentic product thinking.)

Maps to: Amazon Stores personalization, Rufus, Alexa+ shopping.

```
SERVING
[Shopper msg + context] -> [Intent / planner LLM] -> [Tool router]
                                                      |-> [Catalog search tool]
                                                      |-> [Recs retrieval tool]
                                                      |-> [Cart / order tools]
                           [Ranker / re-ranker] <-----+
                           [Grounding + safety guard] -> [Response + product cards]
TRAINING / EVAL LOOP
[Session + tool-call logs] -> [Human + LLM-judge labels] -> [SFT / pref tuning]
[Offline eval suite: task, safety, red-team] -> [Weblab A/B] -> [Monitoring] -> back
```

Follow-ups:

- What does "agentic" add over a standard retrieval-and-ranking recommender? What can the agent do that a ranked carousel cannot?
- Which decisions belong to the LLM planner, and which stay with classical retrieval and ranking models?
- How do you define tools (search, filters, compare, add to cart), and how do you stop the agent calling them in loops?
- How do you ground every recommended product in the live catalog so the agent never invents an ASIN, price or spec?
- How do you build an offline evaluation set for multi-turn shopping tasks, and what is "task success"?
- When would you trust an LLM-as-judge for quality, and how do you calibrate it against human raters?
- Which safety failures matter here (unsafe products, age-restricted items, biased steering, prompt injection from listing text), and how do you red-team them?
- Which online metrics decide the launch, which are guardrails, and how do you randomize a conversational A/B test?
- What is your latency budget per turn, and where would you use a small model or a cache instead of the large LLM?
- How do you personalize using purchase history without breaking privacy expectations?
- How do sponsored products enter the agent's answers without hurting trust?

- How do agent recommendations feed back into training data without amplifying the agent's own biases?

2. Team-specific ML system design: from business problem to science problem

Prompt. "Design the ML system behind the hiring team's product. Take their business problem (for example, improving a metric on their product), turn it into a science problem, then define the data, labels, model, metrics, and the learning or iteration loop." [Reported] [3][4][12][16] (AS II loops in Seattle 2026 and Berlin; the Alexa International Tech guide [23] and a senior AS's design guide [29] describe the same framing. At Seattle the round was followed by modeling-choice probes such as bagging versus boosting [4].)

Maps to: whichever team you interview for: Ads, Search, Alexa, Prime Video, SCOT, Catalog and others.

```
[Business goal + constraints] -> [Science problem: decision, target, unit]
SERVING
[Request] -> [Features (batch + real-time)] -> [Model] -> [Policy / threshold]
                                                              -> [Action]
                                                                   |
[Logged outcomes] <------------------------------------------------+
TRAINING LOOP
[Data sources] -> [Labels / proxy label] -> [Train + offline eval] -> [Registry]
[Shadow -> Weblab A/B] -> [Launch decision] -> [Monitoring + retrain trigger]
```

Follow-ups:

- What exactly is the decision the model makes, and who acts on it?
- What is your proxy label, and how does it diverge from the business outcome you actually care about?
- What is the simplest baseline (heuristic or rules) you would ship first, and what would make you move beyond it?
- Which standard and which recent ML approaches apply to this problem, and why do you prefer one?
- Why this model over the alternatives? Would you use bagging or boosting on the tabular features, and how does that tie to bias and variance? [Reported] [2][4]
- Which offline metric correlates with the online business metric, and how do you check that it does?
- How would you design the Weblab experiment: randomization unit, primary metric, guardrails, duration and power?
- What is the iteration loop after launch? How do new labels arrive, and how often do you retrain?
- What would v1 look like in one quarter, and what does v2 add?
- What is the single riskiest assumption, and how would you de-risk it in two weeks?
- What would make you kill the project after launch?

3. Design a multimodal translation pipeline for catalog objects

Prompt. "Model a pipeline end to end for translating one set of multimodal objects (for example, product listings with text, images and structured attributes) to another language. Cover data, model choice, training, quality evaluation and serving." [Reported] [5][6][7][8] (Applied Scientist II, Hyderabad, 2026; asked as the system design part of a data-science round.)

Maps to: Amazon catalog localization for new marketplaces, International Stores, the Machine Translation science team.

```
SERVING / BATCH
[Listing: title, bullets, attrs, images] -> [Segment + protect: brand, units, attrs]
    -> [Image encoder] --+
    -> [Text encoder] ---+--> [MT model / multimodal LLM] -> [Post-edit rules]
    -> [Quality estimation (QE) score] -> [Publish | Human translator queue]
TRAINING LOOP
[Parallel corpora + in-domain pairs] -> [Fine-tune] -> [Offline eval: auto + human]
[Translator edits + customer signals] -> [Retrain] -> [Per-locale monitoring]
```

Follow-ups:

- What is the unit of translation: each field separately, or the whole multimodal object together?
- When does the image actually help disambiguate the text, and how would you measure whether it does?
- Where would you get in-domain parallel data for a new marketplace that has very little of it?
- How do you keep brand names, model numbers, sizes and units from being translated or mistranslated?
- Would you use a dedicated MT model, a multimodal LLM, or both? How do cost and latency at catalog scale drive that choice?
- How do you evaluate quality without enough human references (BLEU or chrF versus COMET versus human evaluation)? [Likely follow-up to [7]]
- How would you train a quality-estimation model to decide which items can publish automatically and which go to human translators?
- How do translator post-edits feed back into training?
- Which online metric shows the translation helped customers (conversion, returns, search recall in the target language)?
- How would you handle attributes that need conversion rather than translation (sizes, measurements, currency)?
- What changes for a right-to-left or low-resource language?

4. Design Alexa from scratch as an ML system

Prompt. "Design Alexa from scratch. This is ML system design, not software design. Which datasets would you need, how would you frame the ML tasks (wake word, ASR, NLU intent and slot, response selection), which models would you choose and why, and how would you evaluate them offline and online?" [Reported] [1] (Applied Scientist 2, August 2024, business-problem round.)

Maps to: Alexa AI: wake word, ASR, NLU, dialog, Alexa+.

```
SERVING (on-device -> cloud)
[Audio] -> [Wake word (device)] -> [ASR] -> [NLU: domain, intent, slots]
        -> [Skill / API router] -> [Dialog + response gen] -> [TTS] -> [User]
TRAINING LOOP
[Annotated utterances + transcripts] -> [Per-stage models] -> [Per-stage offline eval]
[Implicit feedback: rephrase, barge-in, abandon] -> [Defect labels] -> [Retrain]
[Weblab A/B on end-to-end task success] -> [Monitoring per locale / device]
```

Follow-ups:

- Which datasets do you need for each stage, and how do you collect and annotate them while respecting privacy?
- Would you build a cascaded pipeline or an end-to-end speech-to-intent (or speech LLM) model? What are the trade-offs?
- What runs on the device and what runs in the cloud, and why?
- Which offline metric fits each component (false accepts and false rejects per hour, WER, intent accuracy, slot F1)?
- How do you tie component metrics to online customer metrics such as task completion or user friction? [Likely follow-up to [1]]
- How would you detect intent misclassification from implicit feedback such as rephrases, barge-ins or abandons? [Likely follow-up to [1]]
- How do you add a new intent or skill that has no training data?
- How would you handle accents, code-switching and new locales? (See also Q62 in chapter 06.)
- What end-to-end latency budget would you set, and how would you split it across the stages?
- Which online metric shows that a new NLU model is better, and how do you run that Weblab test?
- How do you stop errors compounding from ASR into NLU?
- How would you justify each model choice against a simpler baseline?

5. Summarize product reviews into key concepts

Prompt. "Science Application: design a system that summarizes product reviews by key concepts or aspects (for example, 'battery life', 'fit') for display on the product page. How do you extract aspects, aggregate sentiment, generate summaries, and evaluate faithfulness?" [Reported] [3][11] (Luxembourg AS onsite, posted August 2025; also a May 2026 Seattle report.)

Maps to: "Customers say" and review highlights on the detail page; Community Shopping.

```
BATCH / NEAR-REAL-TIME
[New reviews] -> [Abuse / fake-review filter] -> [Aspect extraction]
  -> [Aspect sentiment] -> [Aggregate per ASIN: counts, polarity, quotes]
  -> [Summary generator (LLM)] -> [Faithfulness check] -> [Detail-page widget]
TRAINING / EVAL LOOP
[Annotated aspects + summaries] -> [Fine-tune / prompt] -> [Offline faithfulness eval]
[Widget engagement + reports] -> [Weblab A/B] -> [Refresh + drift monitoring]
```

Follow-ups:

- Would you use a fixed aspect taxonomy per product type, open extraction, or both?

- How do you merge synonymous aspects ("battery", "charge lasts")?
- How do you aggregate sentiment so a summary reflects proportions and not a few loud reviews?
- Extractive or abstractive summaries? Where does LLM hallucination creep in?
- How do you measure faithfulness at scale, and when would you trust an automatic checker over human raters?
- How do you keep fake or incentivized reviews from steering the summary? [Likely follow-up to [11]]
- What do you show for long-tail products with only a handful of reviews?
- How often do you refresh summaries, and how do you avoid sudden flips?
- How do you represent minority opinions and safety complaints so they are not averaged away?
- Which online metric shows the widget helps customers (conversion, returns, time to decision)?
- How do you handle reviews in many languages for one ASIN?

6. Design an end-to-end product recommendation engine

Prompt. "How would you design and evaluate a model that improves product recommendations on Amazon?" [Reported] [10]. Variants: "design various models from Recommendation Systems", comparing two recommender models for an Amazon product [Reported] [20]; "a real-time product recommendation engine" [Likely; role and date unconfirmed] [21]; "customers who viewed or bought X" on product pages, covering retrieval, ranking, features, real-time personalization, cold start and A/B testing [Likely] [36][48]; "a large-scale shopping recommender: when does two-tower retrieval beat a traditional ranking stack?" [Likely] [30][37][38].

Maps to: Amazon Personalization and Recommendations (detail page, homepage, "customers also bought").

```
SERVING
[User + context + anchor ASIN] -> [Candidate gen: co-purchase | two-tower | trending]
  -> [Merge + dedupe] -> [Ranker] -> [Re-rank: diversity, business rules] -> [Widget]
                                                                              |
[Impressions, clicks, purchases logged with positions] <-----------+
TRAINING LOOP
[Behavior logs + catalog] -> [Labels + negative sampling] -> [Retrieval + ranker train]
[Offline eval] -> [Embedding index refresh] -> [Weblab A/B] -> [Monitoring]
[Real-time feature stream] -> [Online feature store]
```

Follow-ups:

- Which recommendation surfaces would you design for, and does each need its own objective?
- How do you generate candidates: item-to-item co-occurrence, two-tower retrieval, sequence models, or a mix?
- When would a two-tower retrieval model beat a traditional ranking stack, and how would you combine them?
- How do you train the two-tower model (in-batch negatives, hard negatives, sampling bias correction), and serve it with ANN search?

- What does the ranker optimize: click, add-to-cart, purchase, or a multi-task combination?
- Which features need real-time freshness (the session so far), and which can be batch?
- How do you handle cold start for new users and new ASINs?
- How do you correct for position bias and feedback loops in the training data?
- Which offline metrics would you use (recall@k, NDCG), and why might they disagree with online conversion?
- How would you design the Weblab test, and how do you account for cannibalization between widgets?
- What is your latency budget for candidate generation versus ranking?
- How do you balance short-term conversion against diversity and long-term customer value?

7. Frame an open-ended ML product problem

Prompt. "Given an open-ended ML product problem, frame it: reduce the scope, describe how you would gather data, how you would handle domain adaptation, whether you would distill a large model for serving, and which metrics you would use. Justify each choice against the alternatives." [Reported] [13] (Applied Scientist loop, 2021; the candidate felt they narrowed the scope too much and was down-leveled partly for lacking production experience.)

Maps to: any science team; this is how many team-specific prompts open.

```
[Vague goal] -> [Scoped v1 problem] -> [Data plan: existing | weak | human labels]
TRAINING
[Source-domain data] -> [Large teacher model] -> [Domain adaptation on target data]
    -> [Distill to student] -> [Offline eval vs baselines + ablations]
SERVING
[Request] -> [Student model] -> [Fallback / teacher for hard cases] -> [Output]
[Online metric + guardrails] -> [Iteration on scope]
```

Follow-ups:

- How did you decide what to cut from the scope, and what happens if you widen it again? [Reported] [13]
- Why this model or loss over the alternatives? [Reported] [13]
- What would you do without production-scale labeled data? [Reported] [13]
- Which domain-adaptation technique fits (fine-tuning, continued pretraining, pseudo-labeling, adversarial adaptation), and why?
- When is distillation worth it, compared with quantization or simply serving a smaller model?
- How do you know the student kept the teacher's quality on the slices that matter?
- Which offline metric would you report, and which online metric would decide the launch?
- Which ablations would you run to prove each component earns its place?
- What would you need to see to move from v1 scope to the full problem?

8. Design a spam or fake-review detection system

Prompt. "Design a spam detection system (for example, spam product reviews or messages). Cover labels, features, model choice, class imbalance, thresholds and how you would measure success." [Reported] [17] (Amazon AS virtual onsite, August 2025; this was an L4 loop.) The L5 variant: "Design a system that detects fake or incentivized reviews and review-abuse rings: reviewer-seller-product graph features, text signals, label scarcity, adversarial adaptation, and the human investigator queue." [Likely] [14][40][53]

Maps to: Community Shopping and Reviews integrity, Selling Partner abuse prevention.

```
SERVING
[New review / message] -> [Fast rules + text model] -> [Real-time score]
    -> [Threshold policy] -> [Publish | Hold | Investigator queue]
BATCH
[Reviewer-seller-product graph] -> [Graph / ring detection] -> [Account-level score]
TRAINING LOOP
[Investigator decisions + takedowns] -> [Labels (weak + gold)] -> [Retrain]
[Precision-at-capacity eval] -> [Adversarial drift monitoring] -> back
```

Follow-ups:

- Where do labels come from, and how biased are labels that come only from items investigators chose to look at?
- How would you use weak supervision or active learning when gold labels are scarce?
- Which signals matter besides text (timing bursts, reviewer history, graph links to sellers, verified purchase)?
- When would a graph model beat per-review classification?
- How do you handle extreme class imbalance in training and in evaluation?
- How do you pick the operating threshold given investigator capacity and the cost of false positives on honest reviewers?
- Is precision at fixed capacity, recall, or prevented harm the right success metric?
- How do you detect when spammers adapt, and how fast can you retrain?
- How do you evaluate a new model when the old model shaped which items got labeled?
- How do you run an A/B test on an abuse system without exposing customers to more spam?
- How do LLM-generated fake reviews change your design?

9. Classify short, noisy text (review language, tweets)

Prompt. "Design a model that classifies the language of customer reviews." Expect follow-ups after your solution. [Reported] [10] Related: "design an algorithm or ML system to classify tweets into categories (topic, sentiment, complaint or abuse)." [Reported] [18][19] [20]

Maps to: Reviews and Community, Customer Service, Social and brand listening, multilingual marketplaces.

```
SERVING
[Raw text] -> [Normalize: emoji, URLs, script] -> [Fast model]
    -> [Heavier model for hard cases?] -> [Label + confidence]
    -> [Downstream routing / analytics]
TRAINING LOOP
[Weak labels: marketplace, hashtags, rules] + [Human gold set] -> [Train]
[Per-class / per-length eval] -> [Deploy] -> [Drift + new-class monitoring]
```

Follow-ups:

- Where do the labels come from, and how noisy are weak labels such as marketplace or hashtags?
- Which features or representations would you use: character n-grams, subword embeddings, or a pretrained multilingual encoder? Why?
- How do you handle very short texts, code-mixed text (for example Hinglish) and transliteration?
- Is this multi-class or multi-label, and how does that change the loss and the metrics?
- How do you handle class imbalance and rare categories?
- How would you evaluate it, by class, by text length and by marketplace?
- When is a simple linear model good enough, and when would you pay for a transformer?
- How do you add a new category or language after launch?
- How do you set the confidence threshold for "unknown"?
- What are the throughput and latency requirements, and would you run it in batch or in real time?

10. Design a GitHub Copilot-style coding agent

Prompt. "Design a GitHub Copilot agent: model choice, retrieval of repository context, tool use, evaluation, and latency and cost constraints." [Reported] [9] (Amazon AS report, 2025-2026. Chapter 06 lists it from a Senior AS loop.)

Maps to: Amazon Q Developer, AWS AI coding tools, internal developer productivity.

```
SERVING
[Editor context + request] -> [Context retrieval: repo index, open files, symbols]
  -> [Prompt builder] -> [Code LLM] -> [Tools: run tests, lint, search]
  -> [Edit / completion] -> [Developer accepts / edits / rejects]
TRAINING / EVAL LOOP
[Repo + task benchmarks] -> [Offline eval: pass@k, test pass] -> [Fine-tune / route]
[Acceptance + retention telemetry] -> [Weblab A/B] -> [Monitoring + cost tracking]
```

Follow-ups:

- Inline completion versus a multi-step agent: which product are you building, and how does that change the model and the latency budget?
- How do you index and retrieve repository context, and how do you rank what fits in the context window?
- Which tools would the agent use, and how do you sandbox code execution?
- Would you use one large model or route between a small fast model and a large one?
- How do you evaluate offline: pass@k, test pass rate on repository tasks, human review?

- How do you measure whether it helps developers online (acceptance rate, retained code, downstream bugs)?
- How do you prevent leaking secrets or licensed code?
- What is your cost per request, and how would you halve it?
- How would you fine-tune on internal code without overfitting to stale patterns?
- How do you detect when the agent is confidently wrong?

11. Design a system for global request detection and labeling

Prompt. "Design a system for global request detection and labeling. State your assumptions, propose an architecture, and discuss scaling, latency and reliability." [Reported] [22] (Listed on an Amazon ML system design question page; the role is not specified.)

Maps to: traffic classification, abuse and bot detection, Ads traffic quality, AWS WAF-style services.

```
SERVING (multi-region)
[Requests] -> [Edge feature extraction] -> [Fast detector (rules + light model)]
    -> [Label: benign | suspect | known class] -> [Action / routing]
    -> [Async deep model on sampled traffic] -> [Label store]
TRAINING LOOP
[Sampled requests] -> [Human / heuristic labels] -> [Train + per-region eval]
[Model distribution to regions] -> [Drift + latency monitoring] -> back
```

Follow-ups:

- What exactly counts as a "request" and what labels do you need? Which assumptions would you state first?
- Which detection must happen inline within the latency budget, and which can run asynchronously?
- Where do training labels come from at global scale, and how do you keep label quality consistent across regions?
- One global model or per-region models? How do you handle regional distribution shift?
- How do you keep the system available when the model service fails (fail open or fail closed)?
- How would you sample traffic for labeling so rare classes are covered?
- How do you roll out a new model across regions safely?
- Which metrics define success: precision, recall, latency, cost per million requests?
- How do you detect new request patterns that match no known label?

12. Design Amazon product search ranking

Prompt. "Design a ranker for Amazon product search and explain how it fits into the product: query understanding, retrieval (lexical plus semantic), learning-to-rank features, labels from clicks and purchases, position bias, and offline (NDCG) versus online (conversion) metrics." [Likely] [14][35][39][47] (Blind advises studying how rankers fit into

products [14]; Search Ranking postings describe early-stage efficient models, personalization and exploration [47].) Variant: "How would you design a search system with LLMs?" [Reported] [24] (AS II, LLM team, March 2025).

Maps to: Amazon Search (A9), Search Relevance, multimodal search.

```
SERVING
[Query + user] -> [Query understanding: spell, product type, attributes]
  -> [Retrieval: lexical | semantic two-tower ANN] -> [Pre-rank (light)]
  -> [Rank (LTR)] -> [Re-rank: diversity, sponsored blend, LLM rerank?] -> [SERP]
                                                                          |
[Impressions + clicks + purchases with positions] <-------------------+
TRAINING LOOP
[Logs + human relevance labels] -> [Debias] -> [Train retrieval + rankers]
[Offline NDCG / recall] -> [Interleaving / Weblab A/B] -> [Drift monitoring]
```

Follow-ups:

- What is relevance on Amazon: textual match, purchase likelihood, or both? How do you combine them?
- Which labels would you use (clicks, add-to-cart, purchases, human judgments), and how do you weight them?
- How do you handle position bias in click training data?
- How do lexical and semantic retrieval complement each other, and how do you merge their candidates?
- Why have a pre-ranking stage, and what latency budget does each stage get?
- How do you personalize without hurting the relevance of head queries?
- How do you explore new products (bandits or RL) without hurting conversion? [Likely] [47]
- How would you add an image query (multimodal search) to this stack? [Likely] [33]
- Where would an LLM help (query rewriting, re-ranking, generated answers), and what does it cost in latency?
- How do you handle tail queries and queries with zero results?
- How do you monitor feature drift across batch and real-time features at high QPS? [Likely] [32]

13. Design a Rufus-style LLM shopping assistant

Prompt. "Design an LLM shopping assistant that answers product questions using catalog data, reviews and Q&A. Cover retrieval-augmented generation (RAG), grounding and hallucination control, latency and cost, safety guardrails, and offline (LLM-as-judge, human evaluation) and online evaluation." [Likely] [2][3][24][25] (Recent AS II design rounds moved toward LLM and agentic systems; no report quotes a Rufus prompt verbatim.)

Maps to: Rufus, Amazon Stores generative AI.

```
SERVING
[Question + page context + user] -> [Query rewrite / intent] -> [Retrieve: catalog,
  reviews, Q&A, web?] -> [Rerank chunks] -> [LLM generation] -> [Grounding +
  safety check] -> [Answer + cited products]
TRAINING / EVAL LOOP
[Pretrained LLM] -> [SFT on shopping dialogs] -> [Preference tuning (RLHF / DPO)]
[Offline eval: faithfulness, helpfulness, safety] -> [Weblab A/B] -> [Monitoring]
[Thumbs, follow-ups, purchases] -> [Preference + error data] -> back
```

Follow-ups:

- What do you retrieve, and how do you chunk and index catalog attributes versus free-text reviews?
- How do you stop the model inventing specs, prices or compatibility claims?
- When would you use DPO instead of RLHF to align the assistant, and what preference data would you collect? [Reported] [25]
- How do decoding choices (greedy, beam search, sampling) affect latency, diversity and factuality? [Reported] [24]
- How would you fine-tune on domain data (SFT, then RLHF or DPO) without losing general ability?
- How do you build an offline evaluation set, and when is LLM-as-judge trustworthy?
- Which online metric shows the assistant helps (engagement, purchase conversion, returns), and what are the guardrails?
- How do you meet a latency budget at peak traffic (distillation, caching, routing, speculative decoding)?
- How do you detect and block unsafe, biased or policy-violating answers? Which red-teaming process would you use?
- How do sponsored products appear in answers without misleading customers?
- How do you keep answers fresh when prices and stock change by the minute?

14. Design CTR/CVR prediction and ad placement for Sponsored Products

Prompt. "Design CTR and CVR prediction for Sponsored Products ads in Amazon search, and decide which ads to show, where to place them, how many, and to which customers, under low-latency, high-volume constraints. How do you combine query-ad relevance, predicted CTR and the advertiser's bid?" [Likely] [42][43][44][46] (AS II and L5 Sponsored Products postings describe deep-learning CTR/CVR models, NLP relevance, bandits and causal inference.)

Maps to: Sponsored Products, Marketplace Intelligence, Sponsored Sourcing.

```
SERVING (strict p99)
[Query + shopper] -> [Ad sourcing: lexical + semantic ANN] -> [Relevance filter]
  -> [pCTR / pCVR model] -> [Calibration] -> [Auction: bid x pCTR + quality]
  -> [Placement + ad-load policy] -> [SERP slots]
                                        |
[Impressions, clicks, delayed conversions] <-+
TRAINING LOOP
[Logs] -> [Delayed-label handling + debias] -> [Train pCTR / pCVR]
[Offline AUC, log-loss, calibration] -> [Weblab A/B + advertiser metrics] -> [Monitor]
```

Follow-ups:

- How do you index millions of ads and retrieve relevant candidates for a query (lexical plus semantic two-tower)? [Likely] [45]
- Which features matter (shopper, query, ad, context), and which must be real-time?
- Which model architecture would you choose (GBDT, DeepFM, DCN, a multi-task CTR/CVR model), and why?

- Why does calibration matter for auction pricing, and how do you calibrate and monitor it?
- How do you correct for position bias in the training labels?
- How do you handle conversions that arrive days after the click?
- How do you cold-start new ads and new advertisers?
- How would a contextual bandit choose placements or creatives instead of a static model? How do you define the reward and evaluate a new policy off-policy? [Likely] [42]
- How many ads per page? How do you trade ad revenue against shopper experience?
- How do you A/B test when advertiser budgets couple treatment and control (budget cannibalization)?
- How would you build GenAI-based shopper understanding and audience targeting, and measure its lift? [Likely] [43]
- What does a 1 ms increase in model latency cost you, and where would you cut?

15. Detect invalid or bot traffic on Amazon Ads

Prompt. "Design a system that detects invalid or non-human (bot) traffic on Amazon Ads at billions of events per day. Cover features from user, context and creative engagement, the lack of labels, adapting to new attack patterns, and self-supervised or clustering methods." [Likely] [41] (Ad Fraud Detection, Traffic Quality posting.)

Maps to: Amazon Ads Traffic Quality, Ad Fraud Detection.

```
STREAMING
[Ad events: impressions, clicks] -> [Sessionize + real-time features]
  -> [Rules + online model] -> [Invalid / valid] -> [Billing filter]
BATCH
[Event history] -> [Self-supervised embeddings] -> [Clustering / anomaly detection]
  -> [Analyst review] -> [Labels] -> [Supervised model retrain]
[Advertiser refunds, audits] -> [Delayed ground truth] -> [Eval + drift monitoring]
```

Follow-ups:

- With almost no labels, where does ground truth come from (honeypots, known bot lists, analyst review)?
- Which signals separate bots from humans (timing, device, IP diversity, engagement depth, creative interaction)?
- How would self-supervised representations or clustering surface new attack patterns?
- What must be decided in real time before billing, and what can be corrected afterwards in batch?
- How do you set thresholds when false positives cost publishers and false negatives cost advertisers?
- How do you evaluate a detector when attackers adapt to it?
- How do you scale feature computation to billions of events a day?
- How would you explain a decision to an advertiser who disputes a refund?
- How do you detect sophisticated bots that mimic human behavior?

16. Design a fraud detection system for payments or orders

Prompt. "Build a real-time fraud detection pipeline for Amazon orders and payments. How do you trade off false positives (blocking good customers) against missed fraud? Cover delayed labels such as chargebacks, adversarial drift and real-time scoring latency." [Likely] [36][37]

Maps to: Transaction Risk Management, Amazon Payments, Buyer Risk Prevention.

```
SERVING (checkout path)
[Order + account + payment] -> [Real-time features: velocity, device, graph]
  -> [Risk model] -> [Decision policy: approve | challenge | review | block]
BATCH
[Account-device-address graph] -> [Graph features] -> [Feature store]
TRAINING LOOP
[Chargebacks (weeks later) + investigator labels] -> [Label maturity window]
  -> [Retrain] -> [Cost-weighted eval] -> [Shadow -> Weblab] -> [Drift monitoring]
```

Follow-ups:

- How do you define the label, and how do you train when chargebacks arrive 30-90 days later?
- How do you handle extreme class imbalance in training and evaluation?
- Which features need sub-second freshness (velocity counters), and how do you keep them consistent between training and serving?
- How do you choose operating thresholds from the dollar cost of fraud versus customer friction?
- Blocked orders never produce a label. How do you correct for that selection bias?
- How do you detect adversarial drift, and how fast can you retrain?
- What role do graph models play in catching fraud rings?
- How would you A/B test a fraud model without letting extra fraud through?
- How do you meet the latency budget of the checkout path?

17. Extract structured product attributes for catalog quality

Prompt. "Design a system that extracts and normalizes product attributes (brand, size, color, material) from noisy seller titles, descriptions and images to improve catalog quality. How would you get training labels at scale and measure precision and recall?" [Likely] [6] [14][34] Related reported probe: "Describe the Segment Anything Model (SAM) and how it works. How would you use it in a product-image pipeline, for example for background removal or attribute extraction?" [Reported] [8]

Maps to: Amazon Catalog, Selling Partner listing quality. Chapter 13 covers this prompt in depth; this entry is the compact version.

```
BATCH + ON LISTING CREATE
[Title, bullets, description, images] -> [Product-type classifier]
  -> [Text extractor (NER / seq2seq / LLM)] + [Image model (VLM, SAM crops)]
  -> [Fuse + normalize to schema] -> [Confidence policy] -> [Auto-fill | Human review]
TRAINING LOOP
[Existing attributes (noisy)] + [LLM labels] + [Gold audits] -> [Train]
[Per-attribute P/R eval] -> [Seller accept / reject feedback] -> [Retrain]
```

Follow-ups:

- How do you get labels at scale (existing seller attributes, weak supervision, LLM labeling, human audits)?
- NER, sequence-to-sequence, or an LLM? How does cost at catalog scale change the answer?
- How do you handle thousands of product types with different attribute schemas?
- How do you normalize values ("XL", "extra large", "42 EU")?
- When does the image add information the text lacks, and how do you fuse the two?
- How would you evaluate per attribute and per product type, and which metric do you report upward?
- How do you choose confidence thresholds for auto-filling versus human review?
- How do you measure the downstream impact on search and conversion?
- How do you handle sellers who game attributes to rank higher?

18. Detect duplicate listings in the catalog

Prompt. "Design a duplicate-listing detection system for the Amazon catalog that matches the same product across sellers, using text and image embeddings, blocking or ANN retrieval, and a pairwise matcher." [Likely] [38]

Maps to: Catalog quality, product matching, entity resolution.

```
BATCH + INCREMENTAL
[New / updated listing] -> [Text + image encoders] -> [Blocking: ANN | key fields]
  -> [Candidate pairs] -> [Pairwise matcher] -> [Clustering / merge decision]
  -> [Auto-merge | Human review | No match]
TRAINING LOOP
[Known merges + human-labeled pairs + hard negatives] -> [Train encoders + matcher]
[Pair P/R + cluster purity eval] -> [Merge-reversal feedback] -> [Retrain]
```

Follow-ups:

- What is a "duplicate": the same product, the same variant, or the same product in a different pack size?
- How do you generate candidate pairs without comparing every pair of listings?
- Which hard negatives matter most (same brand, different model; near-identical images)?
- How do you go from pairwise scores to consistent clusters (transitivity)?
- Is a wrong merge or a missed merge more costly, and how does that set the threshold?
- How do you evaluate recall when you cannot label all pairs?
- How do you handle sellers who copy another seller's images?
- How do you run this incrementally for millions of new listings a day?

19. Design a demand forecasting system for millions of ASINs

Prompt. "Design a demand forecasting system for millions of ASINs across fulfillment centers. Cover hierarchical and probabilistic (quantile) forecasts, intermittent demand, cold-start items, promotions, and the metrics used for inventory decisions." [Likely] [37][39] [40]

Maps to: Supply Chain Optimization Technologies (SCOT), Forecasting.

```
BATCH (daily / weekly)
[Sales, price, promos, events, catalog] -> [Feature pipeline]
  -> [Forecast model] -> [Forecast per ASIN x FC x horizon]
  -> [Hierarchical reconciliation] -> [Buying / placement optimizer]
TRAINING LOOP
[History with stockout censoring] -> [Train] -> [Backtest]
[Error metrics by segment] -> [Forecast-to-cost eval] -> [Monitoring + retrain]
```

Follow-ups:

- One global model or per-ASIN models? What are the trade-offs?
- Why probabilistic forecasts, and which quantiles does the inventory decision actually consume?
- How do you handle intermittent demand for long-tail items?
- How do you forecast a brand-new ASIN with no history?
- How do you account for stockouts that censor observed demand?
- How do you model promotions, holidays and price changes?
- How do you keep forecasts consistent across the hierarchy (ASIN, category, FC, region)?
- Which metric would you choose (wQL, MAPE, bias), and how does forecast error translate into ordering cost?
- How do you backtest fairly over time?
- How would you detect and respond to a sudden demand shift?

20. Predict delivery ETA (promise date)

Prompt. "Design a delivery ETA (promise date) prediction system for Amazon orders. Which features would you use (carrier, node, warehouse load, traffic, weather), point versus quantile prediction, how would you produce calibrated uncertainty, and how would you trade off the cost of late versus early promises?" [Likely] [30][53] Variant: "Develop an algorithm or ML system to optimize delivery routes for last-mile delivery." [Likely]

Maps to: Last Mile, Delivery Experience, Fulfillment by Amazon promise.

```
SERVING (at detail page and checkout)
[Item + destination + options] -> [Real-time features: FC load, carrier capacity]
  -> [ETA model] -> [Promise policy] -> [Promise shown]
TRAINING LOOP
[Shipment scans + actual delivery] -> [Labels] -> [Train] -> [Calibration eval]
[Late / early rate by lane] -> [Weblab A/B on conversion + on-time] -> [Monitoring]
```

Follow-ups:

- Which quantile becomes the promise, and how do you choose it from the costs of late versus early?
- How do you validate that predicted intervals are calibrated?
- Which features are available at the detail page versus after checkout?
- How do you handle peak season, weather events and carrier disruptions?
- The promise itself changes customer behavior (conversion). How does that feed back into training?
- How do you evaluate a new model online (on-time rate, conversion, contacts)?
- How do you handle new lanes or new delivery stations with no history?
- What latency budget does the ETA model have on the detail page?

21. Estimate price elasticity for pricing

Prompt. "Design a pricing or price-elasticity model for Amazon retail. How would you estimate the causal effect of price on demand from observational data, handle confounding, validate it, and run experiments safely?" [Likely] [38]

Maps to: Pricing, Retail Business Services, Deals.

```
[Price + demand history] -> [Confounder features: promos, competitors, season]
  -> [Causal estimator] -> [Elasticity per ASIN / segment]
  -> [Price optimizer + business constraints] -> [Price change]
VALIDATION LOOP
[Randomized price tests (where allowed)] -> [Compare with causal estimates]
[Outcome monitoring: units, margin, customer trust] -> [Re-estimate]
```

Follow-ups:

- Why is a naive regression of demand on price biased, and which confounders worry you most?
- Which identification strategy would you use (double ML, instrumental variables, natural experiments)?
- How do you get elasticity for long-tail items with little price variation?
- How do you validate an elasticity estimate when you cannot see the counterfactual?
- How do you account for cross-elasticity and cannibalization between similar items?
- What constraints keep a pricing system customer-trustworthy?
- How would you design a safe price experiment?

22. Diagnose and redeploy a degrading production model

Prompt. "You have a model in production whose performance is degrading. Walk through how you would diagnose it (data drift, label drift, pipeline bugs), retrain, and safely redeploy. Cover the feature store, batch versus real-time serving, drift monitoring, and the rollout or rollback strategy." [Likely] [26][40][54] (Recent reports describe practical design rounds tied to day-to-day ML work [6][26]; PracHub says Amazon rounds mix in feature stores, serving trade-offs, drift monitoring and rollout strategy [22].)

Maps to: any production science team.

```
SERVING
[Requests] -> [Online feature store] -> [Model vN] -> [Predictions] -> [Outcomes]
MONITORING
[Input drift] [Prediction drift] [Label / KPI drift] [Skew: train vs serve] [Latency]
  -> [Alert + slice analysis] -> [Root cause: data | pipeline | concept]
RETRAIN + ROLLOUT
[Fresh data] -> [Retrain vN+1] -> [Offline gate] -> [Shadow] -> [Canary] -> [Weblab]
  -> [Promote | Rollback to vN]
```

Follow-ups:

- What is your first hour of investigation: what do you check first, and why?
- How do you tell data drift, concept drift and a pipeline bug apart?
- How do you detect training/serving skew, and how does a feature store help?
- When would you choose batch inference over real-time inference, and how do you keep features consistent between them? [Likely] [54]
- Which metrics do you monitor when labels arrive weeks late?
- What triggers retraining: a schedule, a drift alarm, or a KPI drop?
- What offline gates must a retrained model pass before it sees traffic?
- How do you design the shadow and canary stages, and what triggers an automatic rollback?
- How do you avoid a feedback loop where the degraded model shaped its own training data?
- How would you explain the incident and the fix to a non-science stakeholder?

23. Design Prime Video homepage recommendations with cold start

Prompt. "Design Prime Video's homepage recommendation system: candidate generation, ranking for watch time versus engagement, row and page layout, and freshness for new releases. Include a strategy for both model cold start and new-user cold start, with serving, evaluation and safe rollout." [Likely] [31][52]

Maps to: Prime Video personalization.

```
SERVING
[Profile + device + time] -> [Candidates: CF | content | trending | new releases]
  -> [Title ranker] -> [Row generator + page layout optimizer] -> [Homepage]
COLD START
[New user: onboarding, context priors] | [New title: content embeddings, metadata]
TRAINING LOOP
[Plays, watch time, browse] -> [Labels] -> [Train] -> [Offline eval]
[Weblab A/B on long-term engagement] -> [Monitoring]
```

Follow-ups:

- What is the objective: clicks, starts, watch time, or long-term retention? How do you avoid clickbait?
- How do you rank titles within rows and rows within the page?
- How do you recommend for a brand-new profile with no history?
- How do you surface a new release that has no interaction data?
- What does "model cold start" mean for a new model launch, and how do you roll it out safely?

- How do you handle shared household profiles?
- How do you measure long-term effects in a short A/B test?
- How do licensing windows and content expiry affect the system?

24. Design GenAI ad creative generation for sellers

Prompt. "Design an LLM system that automatically generates ad creatives (headlines, images) for sellers. How do you ensure brand safety, factual consistency with the catalog, and measure performance?" [Likely] [33] Related: "Design the prediction, ranking and bidding system for Amazon's programmatic (DSP) advertising." [Likely] [49]

Maps to: Amazon Ads creative generation, Amazon DSP.

```
GENERATION
[ASIN catalog data + brand guidelines] -> [Text LLM] + [Image generator]
  -> [Fact check vs catalog] -> [Brand-safety + policy classifiers]
  -> [Advertiser review] -> [Creative pool]
SELECTION LOOP
[Creative pool] -> [Bandit / CTR model selects creative] -> [Serve]
[CTR, CVR, advertiser edits] -> [Preference data] -> [Tune generators] -> back
```

Follow-ups:

- How do you guarantee generated claims match the catalog (no invented features)?
- How do you check brand safety and ad-policy compliance before anything is served?
- How do you preserve product fidelity in generated lifestyle images?
- How do you measure creative quality offline before spending advertiser money?
- How would a bandit pick among generated creatives, and how do you evaluate it?
- How do advertiser edits become training signal?
- What is the cost per creative, and how does it scale to millions of ASINs?

25. Cross-cutting probes asked across any prompt

These are not design prompts on their own. They are the probes interviewers attach to whichever prompt you get.

- Your model improves offline AUC or NDCG but the A/B test shows no lift. What could explain this, and how would you design the experiment (randomization unit, guardrail metrics, sample size, novelty effects)? [Likely] [36][40][51]
- How would you handle severe class imbalance and label noise, and how would you choose an operating threshold from business costs? [Likely] [28]
- Explain the bias-variance trade-off and compare bagging with boosting. Which would you use for a tabular fraud or CTR model, and why? [Reported] [2][4]
- Pick a project on your resume. Why did you choose that model, loss and evaluation over the alternatives? What were the baselines and ablations, what failed, and what would you change to deploy it at Amazon scale? [Reported] [1] (a Senior AS prep thread describes the same probe [27])
- How would cold start (new users, new items, a new model with no logged history) change the design you just drew? [Likely] [50]

- What is the proxy label, what is the offline metric versus the business metric, and how will you launch? [Likely] [29]
- Which parts of your design would you build, and which would you reuse from existing Amazon or AWS services?
- How would you cut the cost of this system by 50% with minimal quality loss?
- If you had ten times the data, or one-tenth of it, how would the design change?

Level calibration: harder L5 probes

The bank above leans on component-level follow-ups. These probes test the L5 bar: owning an ambiguous problem end to end, choosing metrics and experiments, and trading model complexity against latency and cost. Attach any of them to any prompt. They are untagged: they are how interviewers typically push past a junior answer, not quoted reports.

1. Before any model, what one-page problem statement would you write, and which open question would you take back to the product manager first?
2. Your proxy label and the business metric disagree on the launch decision. Which do you trust, and what evidence would change your mind?
3. You have p99 headroom for only a model 10x smaller than your best offline model. Where do you spend that budget, and how do you quantify the quality you give up?
4. Name a design choice you would reverse if serving cost doubled, and one you would keep regardless. Why?
5. Which metric would you refuse to optimize directly here, even though leadership asks for it, and what would you propose instead?
6. Your Weblab shows a statistically significant but tiny lift with a guardrail moving the wrong way. Do you launch? Who decides, and on what evidence?
7. The randomization unit you need is not the one the platform supports (for example, interference across sellers, households or advertisers). How do you still get a trustworthy estimate?
8. Which slice (new customers, a small marketplace, a long-tail category) could your launch quietly harm, and how would you detect it before the aggregate metric does?
9. What would you deliberately not build in v1, and how would you defend that cut to a skeptical senior engineer?
10. Six months after launch, the gain has eroded. How do you separate feedback-loop effects, novelty decay and real drift?
11. How do you size the team and timeline for v1, and which dependency on another team is the biggest schedule risk?
12. What offline experiment, run in a week, would most reduce your uncertainty about whether the full system is worth building?

References

Glassdoor links with a review ID (RVW...) point to anonymous self-reports, often read only as search-result extracts. Where the search returned the same review on several country mirrors, the mirrors are listed together. "ID approximate" means the search could not confirm which of several returned review IDs held the quoted text.

1. r/leetcode post, Aug 27 2024 (Applied Scientist 2 loop; "design Alexa from scratch"; resume deep-dive depth round). Reddit via a redlib mirror, 2024. https://redlib.hbubli.cc/r/leetcode/s/PljCbbdntj
2. Amazon Interview review RVW102987402 (Seattle AS, May-June 2026; agentic recommendation design; loop structure; bagging vs boosting; AUC coding). Glassdoor, 2026. https://www.glassdoor.com.au/Interview/Amazon-Interview-E6036-RVW102987402.htm ; mirror https://www.glassdoor.ca/Interview/Amazon-Interview-E6036-RVW102987402.htm
3. Amazon Entretien review RVW104708794 (Seattle AS II, 2026; agentic recommender with quality and safety evaluation; review summarization; team-specific design). Glassdoor (French mirrors), 2026, ID approximate. https://fr.glassdoor.ch/Entretien/Amazon-Entretien-E6036-RVW104708794.htm ; mirror https://fr.glassdoor.ca/Entretien/Amazon-Entretien-E6036-RVW104708794.htm
4. "J'ai passé un entretien chez Amazon (Seattle, WA) en juin 2026", review RVW104288349 (1 Science ML Breadth, 1 Depth, 1 Sys Design (Team-specific), 1 coding, 1 Bar Raiser; bagging vs boosting). Glassdoor, 2026. https://fr.glassdoor.ch/Entretien/Amazon-Entretien-E6036-RVW104288349.htm
5. Amazon Entretien review RVW103457600 (multimodal translation pipeline design). Glassdoor, 2026. https://fr.glassdoor.ch/Entretien/Amazon-Entretien-E6036-RVW103457600.htm
6. Amazon Interview review RVW102945860 (Hyderabad AS, 2026; multimodal translation; day-to-day ML questions). Glassdoor India, 2026, ID approximate. https://www.glassdoor.co.in/Interview/Amazon-Interview-E6036-RVW102945860.htm
7. Amazon Entrevista review RVW96903377 (Hyderabad, April; multimodal translation pipeline). Glassdoor Mexico, 2025-2026; the review ID suggests 2025, so the year is unverified. https://www.glassdoor.com.mx/Entrevista/Amazon-Entrevista-E6036-RVW96903377.htm
8. Amazon Entretien review RVW98336019 (AS II, 2026; multimodal translation; "Describe SAM, how it works"). Glassdoor (French mirrors), 2026, ID approximate. https://www.glassdoor.fr/Entretien/Amazon-Entretien-E6036-RVW98336019.htm ; mirror https://fr.glassdoor.ch/Entretien/Amazon-Entretien-E6036-RVW98336019.htm
9. Amazon Entretien review RVW102577344 ("Design a Github Copilot agent"). Glassdoor, 2025-2026. https://fr.glassdoor.ch/Entretien/Amazon-Entretien-E6036-RVW102577344.htm ; mirror https://www.glassdoor.co.nz/Interview/Amazon-Interview-E6036-RVW102577344.htm
10. Amazon Colloquio review RVW56678112 (classify the language of reviews; recommendation model design). Glassdoor Italy, year not stated, attribution approximate. https://www.glassdoor.it/Colloquio/Amazon-Colloquio-E6036-RVW56678112.htm
11. Amazon Interview review RVW99133644 (Luxembourg AS, posted Aug 2025; Science Application: summarize product reviews by key concepts; LLM training questions). Glassdoor Singapore, 2025. https://www.glassdoor.sg/Interview/Amazon-Interview-E6036-RVW99133644.htm
12. Amazon Interview review RVW79375451 (Berlin; team-specific design focused on standard and recent ML approaches). Glassdoor Singapore, year not stated. https://www.glassdoor.sg/Interview/Amazon-Interview-E6036-RVW79375451.htm
13. "Amazon, MSFT, Google, Apple, FB, Zoom | Applied Scientist | Multiple | Sep 21 [OFFER, rest Reject]". LeetCode Discuss, 2021. https://leetcode.com/discuss/interview

-experience/1585287/amazon-msft-google-apple-fb-zoom-applied-scientist-multiple-sep-21-offer-rest-reject ; alternate https://leetcode.com/discuss/post/1585287/
14. "Amazon MLE to AS transition". Blind, year not stated. https://www.teamblind.com/post/amazon-mle-to-as-transition-jzgcd2gq
15. "Amazon applied scientist loop". Blind, year not stated. https://www.teamblind.com/post/amazon-applied-scientist-loop-odlzgrp3
16. "Amazon AS2 interview loop". Blind, 2025. https://www.teamblind.com/post/amazon-as2-interview-loop-ry5amdu4
17. "Amazon New Grad Applied Scientist interview experience: five virtual onsite rounds, mostly behavioral, end in an L4 offer". PracHub, 2025. https://prachub.com/interview-experiences/amazon-new-grad-applied-scientist-interview-experience-five-virtual-onsite-rounds-mostly-behavioral-end-in-an-l4-offer
18. Amazon Interview review RVW30340277 (AS; "design an algorithm to classify tweets"). Glassdoor India, 2021, ID approximate. https://www.glassdoor.co.in/Interview/Amazon-Interview-E6036-RVW30340277.htm
19. Amazon Entretien review RVW60311509 (designing an ML algorithm to classify tweets). Glassdoor France, year not stated. https://www.glassdoor.fr/Entretien/Amazon-Entretien-E6036-RVW60311509.htm
20. Applied Scientist Interview Questions (listing page; tweets classification; "design various models from Recommendation Systems"; ML application round about an Amazon product). Glassdoor Spain, year not stated. https://static.glassdoor.es/Interview/applied-scientist-interview-questions-SRCH_KO0,17_IP81.htm
21. Amazon Interview review RVW81627346 (real-time product recommendation engine; role not confirmed as AS). Glassdoor Singapore; the extract said May 2026 but the review ID suggests an earlier year, so the date is unverified. https://www.glassdoor.sg/Interview/Amazon-Interview-E6036-RVW81627346.htm
22. Amazon ML System Design Interview Questions (global request detection and labeling; feature stores, drift, rollout). PracHub, 2025. https://prachub.com/companies/amazon/categories/ml-system-design
23. "Amazon Applied Scientist Interview Guide (Alexa International Tech)". Medium (Interview Preparation), 2025. https://medium.com/interview-preparation/amazon-applied-scientist-interview-guide-alexa-international-tech-022d3a056feb
24. Amazon Interview review RVW96068026 (Applied Scientist II, LLM team, Santa Clara, March 2025; beam search decoding for LLMs; "design a search system with LLMs"). Glassdoor Hong Kong, 2025. https://www.glassdoor.com.hk/Interview/Amazon-Interview-E6036-RVW96068026.htm
25. Amazon Entretien review RVW96863276 ("Differences between DPO and RLHF"). Glassdoor (French mirrors), 2025. https://fr.glassdoor.ca/Entretien/Amazon-Entretien-E6036-RVW96863276.htm ; mirror https://www.glassdoor.fr/Entretien/Amazon-Entretien-E6036-RVW96863276.htm
26. Amazon Interview review RVW47917252 (AS; two practical ML design rounds tied to day-to-day work). Glassdoor India; the extract said April 2026 but the review ID suggests an earlier year, so the date is unverified. https://www.glassdoor.co.in/Interview/Amazon-Interview-E6036-RVW47917252.htm
27. "Amazon Senior Applied Scientist prep". Blind, year not stated. https://www.teamblind.com/post/amazon-senior-applied-scientist-prep-xyoamorv
28. "Amazon Applied Scientist 2 interview". Blind, 2025. https://www.teamblind.com/post/amazon-applied-scientist-2-interview-rdv5k0ef

29. "Nailing the Machine Learning Design Interview" (by a senior Amazon AS). Towards Data Science, year not stated. https://towardsdatascience.com/nailing-the-machine-learning-design-interview-6b91bc1d036c/
30. "What is an Applied Scientist at Amazon Services?" Dataford (prep vendor), 2025. https://dataford.io/interview-guides/amazon-services/applied-scientist
31. Prime Video Applied Scientist interview guide. Dataford (prep vendor), 2025. https://dataford.io/interview-guides/prime-video/applied-scientist
32. Amazon Lab126 Applied Scientist interview guide. Dataford (prep vendor), year not stated. https://dataford.io/interview-guides/amazon-lab126/applied-scientist
33. "What is an Applied Scientist at Amazon Advertising?" Dataford (prep vendor), 2025. https://dataford.io/interview-guides/amazon-advertising/applied-scientist
34. "What is an Applied Scientist at Amazon?" Dataford (prep vendor), 2025. https://dataford.io/interview-guides/amazon/applied-scientist
35. "How would you approach designing a search ranking algorithm for Amazon?" Dataford (prep vendor), year not stated. https://dataford.io/questions/how-would-you-approach-designing-a-search-ranking-algorithm-for-amazon
36. Amazon Applied Scientist interview prep. CleverPrep (prep vendor), 2025. https://www.cleverprep.com/companies/amazon/applied-scientist
37. Amazon Machine Learning Engineer guide. Codemia (prep vendor), 2025. https://codemia.io/guides/amazon-machine-learning-engineer
38. Amazon Applied Scientist interview questions. Interview Kickstart (prep vendor), 2026. https://interviewkickstart.com/interview-questions/companies/amazon/applied-scientist
39. Amazon Applied Scientist interview guide. Interview Kickstart (prep vendor), 2025. https://interviewkickstart.com/odio/amazon-applied-scientist-interview-guide
40. Amazon Applied Scientist Interview Questions. PracHub, 2025. https://prachub.com/companies/amazon/positions/applied-scientist
41. Applied Scientist, Ad Fraud Detection, Traffic Quality - ML (Job ID 2642812). amazon.jobs, 2025. https://amazon.jobs/en/jobs/2642812/applied-scientist-ad-fraud-detection-traffic-quality-ml
42. Applied Scientist, Sponsored Products Marketplace Intelligence (Job ID 2734206). amazon.jobs, year not stated. https://amazon.jobs/en/jobs/2734206/applied-scientist-sponsored-products-marketplace-intelligence
43. Applied Scientist II, Sponsored Products (Job ID 2919860). amazon.jobs, 2025. https://amazon.jobs/en/jobs/2919860/applied-scientist-ii-sponsored-products
44. Applied Scientist, Sponsored Products (Job ID 2828425). amazon.jobs (Japan), year not stated. https://amazon.jobs/jp/jobs/2828425/applied-scientist-sponsored-products
45. Applied Scientist, Sponsored Products (Sponsored Sourcing). amazon.dejobs.org, year not stated. https://amazon.dejobs.org/seattle-wa/applied-scientist-sponsored-products-amazon/D36BC87C80E24BD1A253ADB0E0940534/job
46. Applied Scientist (L5), Sponsored Products Marketplace Intelligence, New York. amazon.dejobs.org, 2025. https://amazon.dejobs.org/new-york-ny/applied-scientist-l5-sponsored-products-marketplace-intelligence/DAEEB404497146528424E91C0D070B6E/job
47. Amazon Applied Scientist, Search Ranking. myGwork (job mirror), 2025. https://mygwork.com/jobs/amazon-applied-scientist-search-ranking-6534939
48. Applied Scientist, MAPLE - Recommender System. myGwork (job mirror), year not stated. https://mygwork.com/jobs/amazon-applied-scientist-maple-recommender-syste

m-6249539
49. Senior Applied Scientist, Amazon Ads, Demand Tech. AnitaB.org job board, 2025. https://jobs.anitab.org/companies/amazon-3-60ad394d-c673-4474-9694-344b0cae748f/jobs/84048208-senior-applied-scientist-amazon-ads-demand-tech-amazon-advertising-demand-tech
50. "What the ML System Design Interview Actually Tests in 2026". techinterview.org, 2026. https://www.techinterview.org/post/3233476380/ml-system-design-interview-what-it-tests-2026/
51. "ML System Design: Build a Search Ranking System". techinterview.org, 2026. https://www.techinterview.org/post/3233460105/ml-system-design-build-a-search-ranking-system/
52. Recommender systems (tag page). Amazon Science, year not stated. https://www.amazon.science/tag/recommender-systems?p=16
53. "Amazon System Design Interview". IGotAnOffer, year not stated. https://igotanoffer.com/en/advice/amazon-system-design-interview
54. "Amazon Software Engineer Interview Experience: Boomerang Loop With a Surprise ML Platform System Design" (an SDE loop, not AS). PracHub, 2025. https://prachub.com/interview-experiences/amazon-software-engineer-interview-experience-boomerang-loop-with-a-surprise-ml-platform-system-design

03. Airbnb Senior MLE: ML Coding and Practical Coding

> Scope: the coding rounds of the Airbnb Senior (G9) and Senior+ Machine Learning Engineer loop: the technical phone screen, the onsite coding round, the HackerRank-style ML/data screen and the code review round. This chapter contains questions only. It gives no answers or solutions.
>
> How it relates to the older chapters: this chapter merges and dedupes four October 2026 search passes with `airbnb-senior-mle/chapters/02-coding-practical.md` (practical coding) and `05-ml-fundamentals.md` (ML depth). Ch. 02 keeps the long tail of SWE-track problems (banking, refunds, ski DAG, Diplomacy, multi-level online assessments). Ch. 05 keeps verbal ML-theory probes. This chapter keeps the problems most likely to reach an MLE candidate, written as runnable practice prompts.
>
> Source caveat: almost every source is an anonymous candidate self-report (Blind, LeetCode Discuss, Glassdoor, 1point3acres, PracHub) or a prep-vendor page. Many were read only as search-result extracts, and some 1point3acres and PracHub bodies need a login. Several reports come from SWE, data science or data engineering candidates rather than MLEs; the role is noted where it matters. Read **[Reported]** as "a cited source says this, or a close paraphrase, was asked at Airbnb". **[Likely]** means a grounded inference from the MLE loop structure, prep guides or Airbnb's published ML work; nobody reported it verbatim.

How this round works

- **Where coding sits in the MLE loop.** MLE candidates describe the loop as "LeetCode + standard ML design + an ML experience round" [48][49]. The public MLE-track summary gives one technical phone screen and typically one onsite coding round of 45-60 minutes, and says the track is filtered mainly by design depth [80]. A 2022 senior ML/DS loop instead had two "DS coding" rounds of Airbnb-tagged LeetCode hard and medium problems in Python, plus two ML rounds [47]. A July 2025 ML platform loop had a coding screen and then an onsite with a code review round [53]. Ask your recruiter which mix you will get.
- **Two flavors of coding.** (1) *Practical algorithmic coding*: an Airbnb-framed product problem (split stays, menus, group lodging, pagination, text layout, query parsing) solved with runnable code [69]. A 2026 Senior+ MLE in the search-ranking org got a "format book records as a table" screen and a "document redaction" onsite [43]. (2) *ML/data coding*: a 45-minute HackerRank screen with pandas data manipulation plus ML concepts [54][62], or implement-from-scratch tasks (kNN, k-means, IoU, a small neural net, convolution, z-scores) reported for a data scientist algorithms round [45]. A Staff MLE candidate asked whether coding has an ML component and got no clear

answer [50], so prepare both. One prep site describes a 60-minute screen mixing ML fundamentals, a coding problem and introductory system design [78].

- **Difficulty.** Phone screens are reported as easy to medium. An MLE onsite coding round was LeetCode-hard and the candidate failed it [46]. Interviewers push for an optimal solution: one senior SWE passed with a suboptimal version that ran and passed every test, then was pushed to optimize [1]; another was rejected on split stay over live debugging and deduplication [4].
- **Language and tools.** Python is the norm for MLEs. Expect a shared editor or HackerRank with code you actually run, often with no starter interface, so you define the API and write your own tests [80][81][86]. Candidates stress "bug free, all cases must run" [9].
- **What the senior bar looks like.** Clarify ambiguous specs before coding (tie-breaks, half-open date ranges, return path versus cost). Reach working, tested code with 10-15 minutes left for follow-ups. State complexity, then extend the solution (k listings instead of two, return all optima, stream the input) without a rewrite. For ML coding, interviewers also probe numerical stability, vectorization, leakage-safe splits and how the code would run in production [43][62].
- **Code review.** Some loops add a code review round: about three pull requests in about 60 minutes. Do not spend too long on the first one [52][7][81].

Top 15 to practice first

Reported prompts first, ordered by how many independent reports mention them.

1. Split stay: every pair of listings that covers a date range back to back; then k listings and the cheapest split. [Reported] (Q1)
2. Minimum-cost menu order with combos (DFS, then bitmask DP). [Reported] (Q2)
3. Menu combination sum with float prices: every combination that hits the budget exactly. [Reported] (Q3)
4. Group lodging / property booking optimizer: fewest properties, then least capacity. [Reported] (Q4)
5. Paginate listings so no host repeats on a page. [Reported] (Q5)
6. Text justification, text formatter and the book-records table (the 2026 Senior+ MLE screen). [Reported] (Q6, Q7, Q8)
7. Document redaction with overlap, case and word-boundary rules (2026 Senior+ MLE onsite). [Reported] (Q9)
8. Palindrome Pairs in better than O(n^2 * k). [Reported] (Q10)
9. Cheapest flights within K stops, returning the route. [Reported] (Q11)
10. Sliding puzzle and grid BFS with keys and doors. [Reported] (Q12, Q13)
11. 2D iterator with `remove`. [Reported] (Q14)
12. HackerRank ML screen: parse a string column into an integer average; write a CV loop that returns the variance of a metric. [Reported] (Q52, Q41)
13. Threshold tuning for a fraud/risk classifier plus a reliability curve (calibration topic reported; coding framing inferred). [Likely] (Q42)
14. kNN and k-means from scratch in NumPy. [Reported] (Q35, Q36)
15. Code review round: three PRs with planted bugs. [Reported] (Q33)

Then work through the Level calibration list (Q76-Q85) to check you can hit the Senior bar under time pressure.

A. Airbnb-tagged practical coding problems (most reported first)

1. **Split stay.** Input: a requested range `[start, end)` and a map `listing_id -> available days` (a list of day numbers or date ranges). Output: every unordered pair of distinct listings (A, B) and split day k such that A covers `[start, k)` and B covers `[k, end)`, with no duplicate pairs. Constraints: up to N listings and D days; availability may be unsorted or have gaps. Edge cases: a single listing covering the whole range, one-day stays, `start == end`, listings with no availability. The code must run bug-free live against your own tests. [Reported] [1][4][5][7][8][9][82]
    - Your O(N^2 * D) solution passes. The interviewer says something better exists. Where is the redundant work? [Reported] [8]
    - Must A cover a contiguous prefix and B a contiguous suffix, or is plain set coverage enough? Which reading do you confirm before coding? [Reported] [7]
    - How would a bitmask encoding of each listing's availability change the check? [Reported] [7]
    - Generalize to at most k listings and return the best split by price or by fewest moves. What is the DP state? [Reported] [1][4]
    - Expose this as an HTTP endpoint that takes a date range and listings with availability. What do you validate, paginate and cache? [Reported] [5][6]
    - Are the day numbers nights (check-in day) or calendar days, and how does that change your boundary tests?
2. **Minimum-cost menu order.** Input: menu entries, each a single item or a combo of several items with a price, and a list of wanted items (small, for example at most 3 unique, possibly with repeats). Output: the minimum total cost to get at least the wanted items, and the entries to order. Edge cases: a combo cheaper than its parts, combos containing unwanted items, wanted items not on the menu (return -1 or an error). Write tests first. [Reported] [1][11][80][92]
    - You wrote DFS. Convert it to DP with a bitmask over wanted items. What does the state hold? [Reported] [11]
    - Which menu entries can you prune before searching? [Reported] [92]
    - Return every optimal order, not just one. How does that change memory? [Reported] [92]
    - Variant: rental "services" (wifi, parking, TV) instead of dishes. What changes in the model? [Reported] [91]
3. **Menu combination sum with float prices.** Input: menu items with decimal prices (for example 2.40, 3.15, 10.02) or a `name -> price` map, and a budget (for example 15.05). Output: every combination of items whose total equals the budget exactly, in a deterministic order, with no duplicate combinations. Items may repeat. Edge cases:

float precision, a zero or negative price, an empty menu, a budget no combination reaches. [Reported] [12][13][14][15][16][17][85]

- How do you avoid floating-point equality bugs? [Reported] [12][16]
- Each item may now be used at most once and the input contains duplicates. How do you avoid duplicate results? [Reported] [17]
- Return only the count of combinations. Does order matter? [Reported] [17]
- What pruning do you add, and what is the worst-case complexity? [Reported] [15]

4. **Group lodging / property booking optimizer.** Input: properties in a neighborhood with capacities, a target neighborhood and a group size. Output: the property IDs whose total capacity is at least the group size, minimizing the number of properties and then total capacity (some versions reverse the tie-break). Return `[]` or -1 if impossible. Edge cases: one property fits exactly, properties in other neighborhoods, group size 0, equal-cost ties. [Reported] [2][3][10][83]

   - Which tie-break do you confirm first: fewer properties or less wasted capacity? [Reported] [10]
   - Is this subset-sum DP or backtracking? What is the worst case for 50 properties?
   - How would you return every optimal set?

5. **Paginate listings.** Input: CSV strings `host_id,listing_id,score,city` sorted by score descending, and a page size (12 in the original). Output: pages printed as blocks separated by a blank line. Within a page a host appears at most once while unused listings from other hosts remain; if a page cannot be filled with distinct hosts, backfill with the remaining listings in their original order. Keep score order otherwise and handle a final partial page. [Reported] [18][19][20][85]

   - What happens when every remaining listing belongs to one host?
   - Can you do it in one pass without rescanning the list for each page?
   - How do you test that relative order inside a page is preserved?
   - The page size changes per request and results stream in. How would you adapt this for a search API?

6. **Text justification.** Input: words and `maxWidth`. Output: lines of exactly `maxWidth` characters, fully justified, with extra spaces going to the leftmost gaps, and the last line left-justified. Edge cases: a single-word line, a word exactly `maxWidth` long. [Reported] [13][40][72][84][85]

   - What changes if a word can be longer than `maxWidth`?
   - Lay the output out in k side-by-side columns. What do you print when one column runs out?

7. **Text formatter (45-minute phone screen).** Input: a raw paragraph and a line width. Output: wrapped lines with a chosen alignment (left, right, center). Words longer than the width must be handled (hyphenate, split or overflow). The code must run with tests. [Reported] [41][80]

   - How do you handle repeated whitespace, tabs and newlines in the input?

- What is the complexity for 1e5 words, and where is the bottleneck?

8. **Book records table (Senior+ MLE technical screen, 2026).** Input: a list of book records (title, author, year and similar fields), some with missing fields. Output: a formatted text table with a header row and separators, each column sized to its longest value, and cells padded and aligned. Practice spec: records are dicts with keys `title`, `author`, `year`; columns appear in that order; a missing field prints as an empty cell; separators use `+`, `-` and `|`. Signature: `format_table(records: list[dict]) -> str`. Edge cases: an empty list, very long titles, missing fields, non-ASCII characters. Write production-quality code with tests. [Reported] [43]

    - Truncate or wrap titles beyond a maximum column width. Which is easier to test?
    - How would you right-align numeric columns and left-align text without hardcoding columns?
    - Records arrive as a stream too large to hold. Can you still size columns?

9. **Document redaction (Senior+ MLE onsite, 2026).** Input: a document string and a list of sensitive terms or patterns (names, emails, phone numbers). Output: the document with each match replaced by a mask. Matching is case-insensitive, respects word boundaries and handles overlapping matches. Practice spec (confirm with the interviewer): terms are plain strings, each match is replaced by `[REDACTED]`, the original casing and all non-matched text are preserved, and when matches overlap the longest match starting at the earliest position wins, so a run covered by overlapping matches is redacted once. Signature: `redact(doc: str, terms: list[str]) -> str`. [Reported] [43][80]

    - Two terms overlap ("John" and "John Smith"). Which wins, and how do you test it?
    - Thousands of patterns: how would a trie or Aho-Corasick change the complexity?
    - How would you replace the rule-based matcher with an NER model, and how would you measure missed redactions? [Likely] [43]
    - Which tests prove the redaction is complete, including matches that span punctuation?

10. **Palindrome Pairs.** Input: unique words. Output: all index pairs (i, j), i != j, where `words[i] + words[j]` is a palindrome. Target better than O(n^2 * k) with a hash map of reversed words or a trie. [Reported] [20][21][84][85]

    - How do the empty string and single-character words affect correctness?
    - Solve it with a trie of reversed words and give the time and space for n words of length k. [Reported] [21]

11. **Cheapest flights within K stops, with the route.** Input: flights `(from, to, price)`, src, dst and K. Output: the cheapest price with at most K stops, or -1, and the list of stops on that route. [Reported] [26][27][28][73][84]

    - Which algorithms still work once the route must be returned?
    - How do you handle ties and zero-cost cycles?

- Replace "at most K stops" with "layover between min and max hours". What changes? [Likely] [65]

12. **Sliding puzzle.** Input: a 2x3 board with tiles 1-5 and 0 as the blank (or the 3x3 variant). Output: the minimum moves to reach the solved state, or -1. BFS over board states is expected; DFS was judged suboptimal. [Reported] [24][25][84]

    - How do you detect an unsolvable board before searching?
    - Would A* help, and with which heuristic?
    - Return the move sequence, not only its length.

13. **Grid maze shortest path, including keys and doors.** Input: a 2D grid of strings where `#` is a wall, `.` is open, `@` is the start, `$` is the target, lowercase `a-f` are keys and uppercase `A-F` are doors that need the matching key; moves are 4-directional. Output: the minimum steps, or -1. Write complete, runnable BFS. [Reported] [30][31][32][84]

    - With up to 6 keys, how do you encode the key state, and how large is the state space?
    - Variant: a ball rolls until it hits a wall. What changes in neighbor generation?

14. **2D iterator with remove.** Input: a list of lists (inner lists may be empty). Implement `has_next()`, `next()` and `remove()`, where `remove()` deletes the element last returned by `next()` from the underlying structure. Calling `remove()` twice in a row, or before any `next()`, must raise. [Reported] [22][23][85]

    - How do you make the iterator lazy over a very large nested input?
    - What happens if the caller mutates the lists outside the iterator?

15. **IP to CIDR.** Input: a start IPv4 address and a count n. Output: the shortest list of CIDR blocks (for example `255.0.0.7/32`) that covers exactly `[ip, ip + n - 1]`. [Reported] [20][73][85]

    - How do you test block boundaries, such as a range starting at an odd address or crossing an octet?

16. **Minimum window substring.** Input: strings s and t. Output: the smallest substring of s containing every character of t with multiplicity, or "". An optimal solution was expected with few hints. [Reported] [24][84]

    - Follow-up: minimum window *subsequence*. Why does the sliding window no longer work?

17. **Ten wizards.** Wizard i knows a list of other wizards; travelling from i to j costs (j - i)^2. Return the minimum-cost path from wizard 0 to wizard 9 and the path itself. [Reported] [29][85]

    - Why is BFS wrong here, and what do you use instead?

18. **Smallest common region.** Input: region hierarchies as lists `[parent, child1, child2, ...]` and two regions. Output: their smallest common ancestor region. [Reported] [29][84]

- The hierarchy is huge and queries are many. What do you precompute?

19. **Alien dictionary.** Input: words sorted in an unknown alphabet. Output: a valid letter order, or "" if invalid (a cycle, or a longer word before its own prefix). [Reported] [39] [40][84]
    - How do you report which letters form the cycle?
20. **City preference merge.** Each user gives an ordered list of preferred cities. Merge them into one global ordering consistent with every list, or report that none exists. [Reported] [42][85]
    - Several valid orders exist. How do you make the output deterministic?
21. **Calculate board score.** Input: a grid of tiles (R and C up to 2000). Output: the total score of the board. Sources give the rule only loosely; practice this Kingdomino-style version: each cell has a terrain letter and a crown count `0-3`; a region is a maximal 4-connected group of cells with the same terrain; the board score is the sum over regions of `(region size) * (crowns in region)`. Signature: `board_score(terrain: list[str], crowns: list[list[int]]) -> int`. Reported in a full-time Airbnb ML tech phone screen. [Reported] [35][36]
    - What is the time and memory for a 2000 x 2000 grid, and can you do it in one pass?
22. **ML phone screen graph problem (45 minutes).** A graph problem solved with DFS and connected components. The exact statement is not public; practice this version: given `n` nodes labelled `0..n-1` and an undirected edge list, return the number of connected components and the sorted list of nodes in the largest one (ties broken by smallest minimum node). [Reported] [55]
    - Would recursive DFS overflow the stack on the largest input? When do you switch to iterative DFS or union-find?
    - Edges arrive as a stream. What changes?
23. **Strings that differ by one character.** Input: n strings of equal length m. Output: true if any two differ in exactly one position. Beat O(n^2 * m). [Reported] [33][84]
    - How do you avoid false positives from hash collisions?
24. **Queue from fixed-length arrays.** Implement a queue using only fixed-size arrays (for example size 5) as building blocks, with `push`, `pop` and `size` in O(1) amortized. [Reported] [34][85]
    - How do you reclaim arrays after many pops?
25. **Rooms and keys / boxes with keys.** Rooms (or boxes) contain keys to other rooms and possibly items. Decide whether every room can be visited, in how few steps, or (LC 1298) the maximum candies collectable. LC 1298 spec: `status[i]` is 1 if box i is open, `candies[i]`, `keys[i]` (boxes whose keys are inside box i), `containedBoxes[i]` (boxes inside box i) and `initialBoxes`; you may open any box you hold that is open or for which you hold a key; return the total candies collected. In one MLE loop, LC 1298

arrived with no interface, so the candidate defined the API and tests. [Reported] [37] [86]

    - Which classes and methods would you model so the solution matches the interviewer's intent?

26. **Pour water on terrain.** Input: terrain heights, V water units and a drop index K. Each droplet moves left if it can eventually fall lower, otherwise right, otherwise stays at K. Output: final heights, and the terrain printed as ASCII with water marked. Part 1 may be printing the terrain alone. [Reported] [64][70][82][90]

    - Which way should water go when both sides are equally low?
    - How do you test exact printed output, including trailing whitespace?

27. **Trapping rain water.** Input: a list of non-negative bar heights of width 1. Output: total units of water trapped. Then the 2D version: an `m x n` height map, water can drain off any border cell. [Reported] [38][84]

    - Why does the 1D two-pointer idea not carry over to 2D, and what does?

28. **Connect Four / ConnectN engine.** A configurable board (6x7 default), two players, `drop(col)` returning the result, with win detection horizontally, vertically and diagonally, plus draws and invalid moves. ConnectN with configurable N was the coding round in one MLE loop. [Reported] [63][87][88]

    - How do you avoid rescanning a column for gravity on every move? [Reported] [88]
    - Determine win, draw or ongoing from a static board alone.
    - Which interfaces make a new rule (irregular board, three players) cheap to add?

29. **Key-value store.** Implement an in-memory store with `get`, `set` and `delete`, then state the tests you would write. Onsite, extend it with transactions (`begin`, `commit`, `rollback`, nested) or a versioned `get(key, timestamp)`, and show how the tests change. [Reported] [4][70][80]

    - Variant: `set_sum(k, [keys])` where updates cascade to dependents. Eager propagation or lazy recompute? What happens on a cycle? [Reported] [89]
    - How would you make it safe under concurrent readers and writers?

30. **URL query parser.** Input: the query portion of a GET URL (or a whole URL). Output: a map from keys to values. Repeated keys become lists, percent-encoding and `+` are decoded after splitting, and empty values and fragments are handled, without `urllib`. [Reported] [66][81][82]

    - A valueless key: true, empty string or null? Which spec detail do you confirm first?
    - Support nested keys such as `a[b]=1`. How does the return type change?

31. **CSV parser.** Implement `parse_csv_line(s)` with double-quoted fields containing commas and escaped quotes (`""`). Follow-up: write the inverse, escaping a list of fields into one line. [Reported] [71][85]

- How do you handle a quoted field spanning multiple lines?

32. **Retryer.** Implement `retry(fn, max_attempts, base_interval, max_interval=16.0)` that calls `fn()`, sleeps `min(base_interval * 2**attempt, max_interval)` seconds after each failure, returns the first successful result and rethrows the last error once attempts run out. [Reported] [81][88]

    - How would you plug in jitter, retry-on-specific-exceptions or a time budget?
    - How do you unit-test timing without real sleeps?

33. **Code review round.** Practice by writing or finding three 40-80-line Python diffs and reviewing each in 15-20 minutes with prioritized written comments. Review about three short pull requests (Python service, data pipeline or helper code such as booking or pagination logic) in about 60 minutes. Planted defects include wrong results without a crash, swallowed exceptions, unchecked nulls, races on shared state, missing error handling around external calls, hidden quadratic loops and poor naming. [Reported] [7][52][53][81]

    - The third PR is long and time is short. How do you triage which comments to write first?
    - Rewrite the worst function correctly and add the tests that would have caught the bug.

34. **Warm-up and listed DP problems.** (a) Minimum cost climbing stairs: given `cost[i]` per step, start at step 0 or 1, climb 1 or 2 steps, return the minimum cost to pass the top. (b) Work sessions: given task durations (n <= 14) and `session_time` >= the longest task, return the minimum number of sessions, where tasks are not split across sessions (bitmask DP). (c) Booking requests: given a list of request lengths in nights, accept no two adjacent requests, maximize total nights, and return both the total and the accepted indices (House Robber framing). [Reported] [1][74][85]

    - For work sessions, what is the state space for 14 tasks, and how do you prune it?

B. ML-specific coding

35. **k-nearest neighbors from scratch.** Implement `fit(X, y)` and `predict(X_test, k)` in Python/NumPy with Euclidean distance and majority vote. Handle ties in the vote, k larger than n, and mixed-scale features. Reported for an Airbnb data scientist algorithms round, where the recruiter said coding was ML-implementation focused, not LeetCode. [Reported] [45]

    - Vectorize the distance computation without an n x m Python loop. What is the memory cost?
    - How would you serve kNN over millions of listing embeddings?

36. **k-means from scratch.** Input: X (n x d), k, max_iter, tol. Output: labels and centroids. Cover initialization, the assignment and update steps, convergence, and empty clusters. [Reported] [45]

- Implement k-means++ initialization. Why does it help?
- How do you pick k, and how do you test the function deterministically?

37. **Intersection over Union.** Compute IoU for two axis-aligned boxes `(x1, y1, x2, y2)`, including no overlap, touching edges and degenerate boxes. [Reported] [45]

    - Vectorize it for N x M boxes, then use it to write non-maximum suppression.

38. **2D convolution.** Implement valid-padding 2D convolution with stride s on a NumPy array without library conv functions. [Reported] [45]

    - Add "same" padding and multiple channels. What output shape do you expect?

39. **Small neural network.** Implement a one-hidden-layer feed-forward network for binary classification in NumPy: input `X` (n x d), hidden layer of size h with ReLU, sigmoid output, binary cross-entropy loss; write the forward pass, backprop and mini-batch SGD, and show the loss falling on a toy dataset such as two moons. [Reported] [45]

    - How do you check your gradients numerically?
    - Your loss is NaN after a few steps. What do you check first?

40. **Z-score normalization and outliers.** For a feature column, compute mean and std, z-score it, handle zero variance and NaNs, and return rows with |z| > 3. [Reported] [45]

    - Why must the mean and std come from the training split only?
    - When would you use median and MAD instead?

41. **Cross-validated metric variance (ML screen).** Write `cv_metric(X, y, preprocessor, model, metric, k=5, seed=0) -> (mean, variance)` that, for each of k stratified folds, fits the preprocessing step and the classification model on the training part only, scores the held-out part with `metric(y_true, y_pred_or_score)`, and returns the mean and variance of the k scores. [Reported] [54]

    - Where must preprocessing be fit so that no information leaks across folds?
    - The data is grouped by user and ordered in time. How do you change the splitter?

42. **Threshold tuning and calibration.** Input: `y_true` and `y_score` from a fraud or booking-risk classifier. Output: the threshold that maximizes F1, and the threshold that maximizes recall subject to precision >= P; then a reliability (calibration) curve with expected calibration error. Reported from a 2026 senior MLE onsite rejected "on calibration". [Reported] [44]

    - How would you recalibrate (Platt scaling, isotonic regression), and on which data?
    - Negatives were down-sampled 10:1 in training. How do you correct the probabilities?
    - Explain ties in scores and the case where no threshold meets precision P.

43. **Practical ML notebook (take-home or onsite).** On an Airbnb-like dataset, write Python to clean the data, build and evaluate a classification or regression model, justify the modeling choices, and summarize results in a short tech report. Reported for data science and full-time ML onsite loops. [Reported] [59][60][76]

    - Which baseline do you show first, and why?
    - How do you show that the evaluation split matches how the model will be used?

44. **Nightly price prediction pipeline.** Given a listings DataFrame (location, room type, bedrooms, amenities, seasonality, historical nightly price, occupancy), build a pandas/scikit-learn pipeline with a time-based train/validation split, feature engineering, a baseline and a GBDT, and MAE/MAPE by market. Pricing prediction is reported in Airbnb MLE onsites; the exact coding framing is inferred. [Likely] (topic reported [57])

    - Why can MAPE mislead for cheap listings, and what would you report instead?
    - How would you deploy and monitor this model?

45. **Search ranking without a text query.** For a guest who enters only location and dates, build per (user, listing) features from a log DataFrame (price relative to market, past clicks and bookings, distance, review score). Train a pointwise or pairwise ranker and evaluate offline with NDCG@k on held-out searches. The topic was a 2026 Senior+ MLE onsite round, and a recruiter told another senior MLE candidate the practitioner round targets Airbnb's own engineering problems; the coding framing is inferred. [Likely] (topic reported [43][51])

    - How do you split searches into train and test so the same user's later searches do not leak?

46. **Ranking metrics.** Input: per search, a ranked list of listing IDs and graded relevance (booked = 2, clicked = 1, else 0). Implement NDCG@k, MRR and precision@k, averaged across searches, and compare two rankers. Then implement a "discounted cumulative booking" metric that counts only bookings. Handle searches with no positives. [Likely] [43][48][93]

    - Should searches with no positives be dropped or scored as zero? How does that bias the comparison?
    - How would you get a confidence interval on the NDCG difference between two rankers?

47. **Logistic regression and AUC from scratch.** Implement logistic regression with mini-batch gradient descent and L2 regularization in NumPy, then compute ROC AUC from scratch, including ties. Check both against a library on a toy dataset. [Likely] [50]

    - How do you keep the loss numerically stable for large logits?
    - Compute AUC in O(n log n) rather than over all pairs.

48. **Numerically stable losses.** Implement `bce_from_logits(y, z)` and multiclass `softmax_cross_entropy(Y, Z)` with the log-sum-exp trick, plus their gradients with respect to the logits, and verify the gradients with finite differences. Inputs with `|z| >`

`1000` must not produce NaN or inf. Explain why log loss beats accuracy for a booking-probability model. [Likely] [79]

49. **Factorization machine scoring.** Given sparse features (indices and values), bias w0, weights w and latent matrix V (n x k), compute the FM prediction in O(nk) using the reformulation, then write one SGD update for logistic loss. FM versus embedding + DNN came up in a 2026 senior MLE onsite. [Likely] [44]
    - How does the cost scale with the number of non-zero features rather than n?
50. **Pairwise learning-to-rank loss.** Given per-search scores and labels (booked or not), implement a vectorized pairwise logistic (RankNet) loss over all positive-negative pairs, then add LambdaRank-style weighting by |delta NDCG|. [Likely] [97]
    - How do you avoid the O(n^2) memory blow-up for long result lists?
51. **Two-tower retrieval and top-k.** Implement a two-tower model in PyTorch (query: location, dates, guests; listing: attributes) trained with in-batch softmax negatives. Then write a function returning the top-k listings for a query from precomputed embeddings with brute-force cosine similarity and an optional geo filter, and report recall@k. [Likely] [96]
    - Why do in-batch negatives over-penalize popular listings, and how do you correct it?
    - Pre-filter or post-filter by geography: what happens to recall?

C. Data, pandas and SQL coding

52. **String column to integer average (HackerRank ML screen).** Given a pandas DataFrame with a string column of comma-separated numbers (for example `"3,5,10"`), add a column holding the integer average of each row. Handle empty strings, whitespace, malformed tokens and NaN. Reported alongside the CV-variance question (Q41) in a live two-question HackerRank screen. [Reported] [54]
    - Do it vectorized rather than with a Python loop over rows. Which is faster here, and why?
    - What rounding does "integer average" mean, and how do you confirm it?
53. **Raw event processing (ML/DS phone screen).** Given raw event rows `(user_id, ts, event_type)` with `event_type` in {search, view, wishlist, book}, unsorted and with exact duplicates, drop duplicates, sessionize per user so that a gap of more than 30 minutes starts a new session, and output one row per session (user_id, session_id, start, end, n_events, n_views, booked) plus per-user totals (sessions, median session length, booking rate). [Reported] [56][82]
    - How do you validate that your transformation did not silently drop or double-count users?
    - The file does not fit in memory. How do you restructure the code?

54. **Consecutive-day activity in SQL.** Given `impressions(user_id, listing_id, ts)`, return the users who have at least one impression on each of seven or more consecutive calendar days (UTC). [Reported] [82]
    - How do you handle several impressions per day and time-zone boundaries?
55. **Listing metrics in SQL.** Given `listings(listing_id, city, price)` and `calendar(listing_id, date, status)` with status in {booked, available, blocked}, compute the average vacant (available, not booked) days per listing over a given month, and the median price per city. [Reported] [82]
    - How do you define "vacant" for a night the host blocked?
56. **Cumulative booking metrics.** Given `bookings(guest_id, booking_id, ts)` and `events(guest_id, event_id, ts)`, for each event compute the guest's number of bookings strictly before the event, and the first and last booking times before it (NULL if none), so the columns are point-in-time correct model features. [Reported] [82]
57. **Pandas cleaning + GBDT screen.** Given bookings and listings DataFrames, clean them (nulls, dtypes), aggregate (booking rate per city and week), join with reviews, engineer features (groupby aggregates, date features, one-hot encoding), train a gradient-boosting model and report suitable metrics. Explain the split and the metric. [Likely] [62]
    - Which of your aggregates would leak the label if computed over the whole table?
58. **Point-in-time listing features.** From raw search-log and booking tables, compute per-listing impressions, CTR, booking rate and average price over the trailing 30 days as of each search, with no leakage, using pandas or SQL window functions. [Likely] [62]
    - How do you test that no feature used information from after the label timestamp?
59. **Search-to-booking attribution.** Given `searches(search_id, user_id, ts, market)`, `impressions(search_id, listing_id, position)` and `bookings(user_id, listing_id, ts)`, attribute each booking to the user's most recent search within 7 days that impressed that listing. Output the per-market search-to-booking conversion rate and the average position of booked listings, in vectorized pandas. [Likely] [62]
    - Use `merge_asof` or a window join? What happens with ties in timestamps?
60. **Booking-records practical coding.** Given `listings(listing_id, host_id)` and `bookings(booking_id, listing_id, check_in, check_out, updated_at)` where a booking may appear several times (keep the latest `updated_at`), output bookings per host, and occupancy per listing per month as booked nights divided by nights in the month, with `[check_in, check_out)` half-open. [Likely] [61]
    - A booking spans a month boundary. How do you split its nights?

61. **Calendar availability.** Given existing bookings `[check_in, check_out)` for a listing, decide whether a requested stay can be accepted, processing requests in arrival order. Follow-up: return the minimum number of listings needed so that no guest is double-booked. [Likely] [70][75]

    - Are dates half-open nights, and how does that change your boundary tests?

D. Experimentation and statistics in code

62. **A/B test analysis.** Given per-user rows (variant, sessions, bookings, revenue), compute absolute and relative lift in booking conversion, a two-proportion z-test p-value and a 95% confidence interval; a bootstrap CI for revenue per user; and a chi-square sample-ratio-mismatch check against the expected split. State whether you would ship. [Likely] [1][67][68]

    - A user appears in both variants. What do you do with them?
    - Revenue is heavy-tailed. Do you cap, log-transform or switch tests?

63. **Conversion metrics from raw tables.** Given `assignments(user_id, variant, assigned_at)`, `visits(user_id, ts)` and `bookings(user_id, ts)`, compute per variant the share of assigned users who visited and the share who booked after assignment, counting only activity at or after `assigned_at`. [Reported] [82]

    - Is the denominator visits, unique visitors or searches, and how must that match the randomization unit?

64. **Panel regression.** From `users(user_id, signup_date)` and `bookings(user_id, ts, amount)`, build a user-by-week panel in SQL (zero rows included), then in Python regress weekly bookings on a treatment indicator with user and week fixed effects. [Reported] [82]

    - Which fixed effects do you include, and how do you cluster the standard errors?

65. **Team-draft interleaving.** Given ranked lists A (control) and B (treatment), produce one merged list of length k with no duplicates, recording which ranker contributed each item. Given clicks or bookings on the merged list, compute the per-query winner, the overall preference and a sign-test p-value. [Likely] [94]

    - What can interleaving not tell you that an A/B test can?

66. **Counterfactual offline evaluation.** Given logged impressions with logging-policy propensities, observed bookings and a new ranker's scores, implement IPS and self-normalized IPS estimates of the new ranker's expected bookings at top-k, and compare to the logged policy with bootstrap CIs. [Likely] [95]

    - What happens to variance when the new ranker places items the old one rarely showed?

67. **Ratio metrics.** Given per-user rows `(variant, searches, bookings)`, implement the delta-method variance of bookings per search (sum of bookings over sum of searches)

per variant with user-level randomization, compute a 95% CI on the difference, and compare it with a user-level bootstrap. [Likely]

    - Why is resampling individual searches wrong here?

68. **Sample size and power.** Write `sample_size(p0, rel_lift, alpha=0.05, power=0.8, two_sided=True) -> int` returning the per-arm sample size to detect a relative lift `rel_lift` on a baseline conversion rate `p0` with a two-proportion z-test. [Likely]

    - How does the answer change for a one-sided test or a ratio metric?

69. **CUPED.** Given per-user arrays `y` (in-experiment metric), `x` (same metric in the pre-period) and `variant`, compute `theta = cov(x, y) / var(x)` pooled across variants, the adjusted metric `y - theta * (x - mean(x))`, the treatment effect and CI before and after adjustment, and the variance reduction. [Likely]

    - What happens for new users with no pre-period data?

70. **Deterministic bucketing.** Write `assign(user_id: str, salt: str, weights: dict[str, float]) -> str` that hashes `salt + user_id` into `[0, 1)` and returns the variant whose cumulative weight interval contains it; the same inputs always give the same variant. [Likely]

    - How do you show assignment is uniform and independent across two experiments?
    - How do you ramp from 5% to 50% without reassigning already-exposed users?

71. **Multiple testing.** Write `benjamini_hochberg(pvals: list[float], q=0.05) -> list[bool]` that returns, in input order, which metric p-values from one experiment stay significant at FDR q. [Likely]

    - When would you prefer Bonferroni or a single pre-declared primary metric?

E. Debugging and production-readiness probes

72. **Debug a model.** You get a training script for a booking-probability classifier whose validation AUC is suspiciously high, or which trains but underperforms online. Find and fix the bugs (target leakage from post-booking features, a random split instead of a time split, unscaled features, misaligned labels or a wrong loss), rerun and report corrected metrics. A process guide lists "debugging a model" among the ML loop's rounds. [Likely] [62][43]

    - Which single check would you run first, and why?
    - How do you tell a data-pipeline bug from a genuine modeling problem?

73. **Listing or photo quality classifier.** Given listing metadata and labels, write a training and evaluation script for an imbalanced binary quality classifier and report precision and recall at a chosen threshold. [Likely] [63][67]

    - How do you choose the threshold if removing a good listing is far costlier than missing a bad one?

74. **Fraud, risky-booking or damage-claim model in code.** Train and evaluate a trust classifier (fraud, risky booking, or property-damage claim triage) with rare, delayed labels and a human-review threshold. Damage-claim and pricing ML problems are reported in a full-time ML onsite; the coding framing is inferred. [Likely] [58][77]
    - How do you build a validation set when labels mature 60 days after the booking?
75. **Cross-cutting probes in every coding round.** [Reported] [1][4][80][81]
    - Before you code: what are the input format, output format and tie-breaking rules?
    - Write your own tests now. Which edge cases did you choose, and why?
    - What are the time and space complexity at the largest stated input?
    - A new requirement arrives (return the path, k listings, stream the input). How much code changes?
    - This runs behind an API endpoint tomorrow. What do you validate, log or cache?

F. Level calibration (Senior bar)

Harder probes to run on yourself after the core bank. Each assumes 45-60 minutes, runnable code and your own tests.

76. **Split stay at scale.** 10^5 listings, a 365-day horizon and 10^4 range queries per second. Precompute so that each query `[start, end)` returns every valid two-listing split in output-sensitive time, and state memory use. Then support availability updates without a full rebuild. [Likely]
77. **Menu order with quantities.** Extend Q2 so wanted items have counts up to 5 and up to 6 distinct items, so the bitmask becomes a mixed-radix state. Return the cheapest order and prove the state count fits in memory. Add a property-based test that compares against brute force on random small menus. [Likely]
78. **Streaming pagination.** Redo Q5 as a generator that consumes listings from an iterator in score order and yields pages lazily with O(page_size x hosts on page) memory, keeping the same output as the batch version. Write a test that checks equivalence on 1,000 random inputs. [Likely]
79. **Redaction with patterns.** Extend Q9 so terms can be regexes (emails, phone numbers) mixed with plain strings, matches can span line breaks, and the function returns both the redacted text and a list of `(start, end, term)` spans in the original text. Keep it linear in document length for 10^4 plain terms. [Likely]
80. **Vectorized kNN under a memory cap.** Extend Q35 to n = 10^6 training rows, m = 10^4 queries and d = 64 within 2 GB of RAM: chunked distance computation, `argpartition` for top-k and a distance-weighted vote. Report time and memory, and test against a brute-force reference on a small sample. [Likely]
81. **Mini-batch k-means for streams.** Implement mini-batch k-means with k-means++ seeding on a first batch, per-centroid learning rates and empty-cluster reseeding.

Make results reproducible with a seed and test convergence on synthetic blobs with known centers. [Likely]

82. **Exact AUC and NDCG with ties and weights.** Implement ROC AUC in O(n log n) with tied scores (counting ties as half) and per-row sample weights, and NDCG@k where tied scores are ordered by expected gain over all tie permutations. Validate against scikit-learn on 100 random cases. [Likely]
83. **Leakage-safe time-series CV.** Write a splitter for booking data that is grouped by `guest_id`, ordered by `ts`, and has a label that matures 30 days later. Each fold trains on data before a cutoff minus a 30-day embargo and validates on the next window with no guest in both. Add tests that fail if any guest or future row leaks. [Likely]
84. **Calibrated threshold service.** Package Q42 as a class `ThresholdPolicy.fit(y_true, y_score, sample_weight)` with Platt or isotonic recalibration, a precision-constrained threshold, `predict(scores)`, JSON (de)serialization and input validation (NaNs, a single class, empty input). Write unit tests for each failure mode. [Likely]
85. **IPS evaluator with clipping.** Extend Q66 with weight clipping at a configurable cap, doubly robust estimation using a supplied reward model, and a bootstrap CI that resamples whole searches. Show on a simulation where the true value is known that clipping trades bias for variance. [Likely]

References

1. "Airbnb SWE Interview Breakdown: The DP questions they asked + How I passed." Blind, 2026. https://www.teamblind.com/post/airbnb-swe-interview-breakdown-the-dp-questions-they-asked-how-i-passed-stpegz67
2. "Airbnb" (technical screen, senior; Property Booking Optimizer). LeetCode Discuss, 2025. https://leetcode.com/discuss/post/6717917/airbnb-technical-screen-interview-senior-fzjk/comments/2978747/
3. "Airbnb" (senior loop; hotel allocation). LeetCode Discuss, 2026. https://leetcode.com/discuss/post/7517460/
4. "Airbnb Software Engineer Interview Experience — Bombed the Live Bug-Free Hotel Split-Stay Problem." PracHub, 2025. https://prachub.com/interview-experiences/airbnb-software-engineer-interview-experience-bombed-the-live-bug-free-hotel-split-stay-problem
5. "airbnb Tech Phone Screen: API Design for Split Stay Availability." 1point3acres, 2025. https://www.1point3acres.com/interview/thread/1144028
6. "Airbnb Interview Questions" (page 3; split-stay HTTP API, graph and shortest-path puzzles). PracHub. https://prachub.com/companies/airbnb?page=3
7. 1point3acres post #13 (English mirror): two coding rounds, split stay with bitmask bonus, three-PR code review. FlowSight/coding, GitHub. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/13.md
8. 1point3acres post #44 (mirror): split stay; O(N^2) judged not good enough. FlowSight/coding, GitHub. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/44.md
9. 1point3acres post #15 (mirror): hotel split stay; "bug free, all cases must run". FlowSight/coding, GitHub. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/15.md

10. 1point3acres post #38 (mirror): group-size property selection DP. FlowSight/coding, GitHub. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/38.md
11. 1point3acres post #10 (mirror): menu minimum cost; DFS then bitmask DP. FlowSight/coding, GitHub. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/10.md
12. "Airbnb | Phone screen | Combination Sum" (floating-point menu). LeetCode Discuss, 2017. https://leetcode.com/discuss/interview-question/124782/Airbnb-or-Phone-screen-or-Combination-Sum/
13. "airbnb phone interview" (text justification; spend all of X money on menu items). LeetCode Discuss, 2020. https://leetcode.com/discuss/interview-question/914253/airbnb-phone-interview
14. "Airbnb Software Engineer Fulltime Tech Phone Screen Interview: Menu Combination Question." 1point3acres. https://www.1point3acres.com/interview/thread/917068
15. "Airbnb Technical Phone Interview: Menu Combination DFS Question." 1point3acres. https://www.1point3acres.com/interview/thread/1058797
16. "Airbnb Tech Phone Screen: Menu Combination Problem Explained." 1point3acres. https://www.1point3acres.com/interview/thread/707167
17. "Airbnb Technical Phone Screen Interview Experience: Combination Sum Variation." 1point3acres. https://www.1point3acres.com/interview/thread/1084220
18. "Airbnb | Phone screen | Paginate Listings." LeetCode Discuss, 2018. https://leetcode.com/discuss/interview-question/125518/1st-telephone-interview-paginate-listings/ (also at https://leetcode.com/discuss/interview-experience/125518/)
19. Glassdoor review RVW17324807 (host-listing pagination; SWE intern). Glassdoor. https://fr.glassdoor.ca/Entretien/Airbnb-Entretien-E391850-RVW17324807.htm
20. Glassdoor review RVW17284596 (pagination, Palindrome Pairs, IP to CIDR). Glassdoor, 2017. https://www.glassdoor.sg/Interview/Airbnb-Interview-E391850-RVW17284596.htm
21. "Airbnb interview experience" (Nov 2022; Palindrome Pairs via trie). Naukri Code360, 2022. https://www.naukri.com/code360/interview-experiences/airbnb/airbnb-interview-experience-nov-2022-exp-0-2-years
22. "Airbnb | Phone screen | Iterator over list of lists." LeetCode Discuss, 2019. https://leetcode.com/discuss/interview-question/352704/airbnb-phone-screen-iterator-over-list-of-lists
23. "Airbnb Fulltime Software Engineer Tech Phone Screen: 2D Array Iterator Implementation." 1point3acres. https://www.1point3acres.com/interview/thread/714458
24. "AirBnb SSE rejected" (minimum window substring; sliding puzzle). LeetCode Discuss, 2022. https://leetcode.com/discuss/interview-experience/2133358/airbnb-sse-rejected/
25. "Airbnb | SDE Intern | India | July 2022 [Offer]" (sliding puzzle, DFS then BFS). LeetCode Discuss, 2022. https://leetcode.com/discuss/interview-experience/2345847/
26. "AirBnb Software Engineer L4 Bangalore" (cheapest flights within K stops with the stops). LeetCode Discuss, 2022. https://leetcode.com/discuss/post/1775088/airbnb-software-engineer-l4-bangalore-by-yxg5/
27. Airbnb online assessment post (cheapest flights within K stops). LeetCode Discuss. https://leetcode.com/discuss/interview-experience/1632963/

28. Glassdoor review RVW13237989 (cheapest itinerary between two cities). Glassdoor, 2016. https://fr.glassdoor.ch/Entretien/Airbnb-Entretien-E391850-RVW13237989.htm
29. "Airbnb phone - Smallest Common Region & 10 Wizards" (Senior SE). LeetCode Discuss, 2020. https://leetcode.com/discuss/interview-question/983456/airbnb-phone-smallest-common-region-10-wizards
30. "Airbnb phone interview" (grid with walls, keys and doors). LeetCode Discuss, 2019. https://leetcode.com/discuss/interview-question/418142/Airbnb-phone-interview
31. Glassdoor review RVW16192336 (Senior SWE; LeetCode-hard maze with BFS). Glassdoor. https://www.glassdoor.com.au/Interview/Airbnb-Interview-E391850-RVW16192336.htm
32. Glassdoor review RVW91124632 (senior SWE; hard maze problem). Glassdoor, 2025. https://www.glassdoor.sg/Interview/Airbnb-Interview-E391850-RVW91124632.htm
33. "Airbnb | Phone Screen | Find Strings That Differ by 1 Character." LeetCode Discuss, 2019. https://leetcode.com/discuss/interview-question/382955/Airbnb-or-Phone-Screen-or-Find-Strings-That-Differ-by-1-Character
34. "Airbnb phone screen: fixed-length array queue." LeetCode Discuss, 2019. https://leetcode.com/discuss/interview-question/341250/airbnb-phone-screen-fixed-length-array-queue
35. "Airbnb | Phone Screen | Calculate Board Score." LeetCode Discuss, 2019. https://leetcode.com/discuss/interview-question/341292/Airbnb-or-Phone-Screen-or-Calculate-Board-Score
36. "Airbnb Fulltime Machine Learning Tech Phone Screen Experience" (board score). 1point3acres. https://www.1point3acres.com/interview/thread/1168518
37. "Airbnb onsite - rooms and keys II." LeetCode Discuss, 2022. https://leetcode.com/discuss/interview-question/2160577/airbnb-onsite-rooms-and-keys-ii
38. "AirBnB onsite similar to trapping rain water, found it too hard." LeetCode Discuss, 2021. https://leetcode.com/discuss/interview-question/1193518/airbnb-onsite-similar-to-trapping-rain-water-found-it-too-hard
39. "Interviewing at Airbnb." Blind. https://www.teamblind.com/post/interviewing-at-airbnb-g58oa2au
40. Glassdoor review RVW9496934 (Text Justification, Alien Dictionary; San Francisco). Glassdoor, 2015. https://fr.glassdoor.ch/Entretien/Airbnb-Entretien-E391850-RVW9496934.htm
41. "Airbnb Software Engineer Tech Phone Screen Interview" (text formatter). 1point3acres. https://www.1point3acres.com/interview/thread/728849
42. 1point3acres thread 197945 (text justification; city ranking by topological sort). 1point3acres. https://www.1point3acres.com/bbs/thread-197945-1-1.html
43. "Airbnb Senior+ Machine Learning Engineer Interview Experience — Search Ranking and Team Matching." PracHub, 2026. https://prachub.com/interview-experiences/airbnb-seniorplus-machine-learning-engineer-interview-experience-search-ranking-and-team-matching
44. "Airbnb Senior Machine Learning Engineer Interview Experience — Four ML System Design Rounds, Rejected on Calibration." PracHub, 2026. https://prachub.com/interview-experiences/airbnb-senior-machine-learning-engineer-interview-experience-four-ml-system-design-rounds-rejected-on-calibration
45. "Airbnb Data Scientist Algorithms interview" (kNN, k-means, IoU, ANN, convolution, z-score from scratch). Blind, 2023. https://www.teamblind.com/post/airbnb-data-scientist-algorithms-interview-n5rfjn3t

46. "Failed Airbnb on-site coding interview for an MLE position, how much hope do I have?" Blind, 2023. https://www.teamblind.com/post/failed-airbnb-on-site-coding-interview-for-an-mle-position-how-much-hope-do-i-have-avk5p8ly
47. "Airbnb on site interview" (senior ML/DS: two ML rounds, two DS coding rounds). Blind, 2022. https://www.teamblind.com/post/airbnb-on-site-interview-iyxjf46j
48. "ML Experience round at Airbnb." Blind. https://www.teamblind.com/post/ml-experience-round-at-airbnb-sorv1g3x
49. "Airbnb MLE design rounds." Blind. https://www.teamblind.com/post/airbnb-mle-design-rounds-pulb6mpe
50. "Airbnb Staff MLE interview." Blind. https://www.teamblind.com/post/airbnb-staff-mle-interview-0cfhvln7
51. "Airbnb senior MLE interview prep." Blind. https://www.teamblind.com/post/airbnb-senior-mle-interview-prep-n1mdsd3b
52. "Airbnb Code Review Round." Blind. https://www.teamblind.com/post/Airbnb-Code-Review-Round-OnE7Ex7H
53. "Interviewing for Airbnb ML platform role." Blind, 2025. https://www.teamblind.com/post/interviewing-for-airbnb-ml-platform-role-8in7czso
54. Glassdoor review RVW38727859 (live HackerRank: string-column average; preprocessing + classifier + metric variance). Glassdoor. https://www.glassdoor.com.ar/Entrevista/Airbnb-Entrevista-E391850-RVW38727859.htm
55. "airbnb Machine Learning Tech Phone Screen Review" (graph, DFS, connected components). 1point3acres. https://www.1point3acres.com/interview/thread/1169233
56. "Airbnb Machine Learning Data Science Interview Experience - Tech Phone Screen" (Python data processing). 1point3acres. https://www.1point3acres.com/interview/thread/1124986
57. "Airbnb Fulltime Machine Learning Engineering Onsite Interview Overview" (ML design, pricing prediction, coding). 1point3acres, 2025. https://www.1point3acres.com/interview/thread/1114733
58. "Airbnb Machine Learning Fulltime Onsite Interview Experience and Insights" (coding tests; property damage claims and pricing). 1point3acres. https://www.1point3acres.com/interview/thread/1162455
59. "My Fulltime Machine Learning Onsite Interview Experience at Airbnb" (practical ML rounds and coding challenges). 1point3acres. https://www.1point3acres.com/interview/thread/797269
60. "Airbnb Onsite Interview, Data Science" (take-home: Python, ML, tech report). LeetCode Discuss, 2019. https://leetcode.com/discuss/post/357222/airbnb-onsite-interview-data-science/
61. "Airbnb Senior Data Engineer Bangalore Reject." LeetCode Discuss, 2022. https://leetcode.com/discuss/interview-experience/1752517/airbnb-senior-data-engineer-bangalore-reject/
62. "Airbnb Machine Learning Engineer Interview Guide." Interview Query, 2025. https://www.interviewquery.com/interview-guides/airbnb-machine-learning-engineer
63. "Airbnb Machine Learning Engineer Interview Questions." PracHub. https://prachub.com/companies/airbnb/positions/machine-learning-engineer
64. "Airbnb Interview Questions" (simulate pouring water onto terrain). PracHub, 2025. https://prachub.com/companies/airbnb
65. "Airbnb Interview Questions" (page 4; split-stay / exact-layover booking logic). PracHub. https://prachub.com/companies/airbnb?page=4

66. "Airbnb Interview Questions" (page 5; query-string parser). PracHub. https://prachub.com/companies/airbnb?page=5
67. "Airbnb Interview Questions" (page 6; SQL/Python experiment analyses; photo/listing quality). PracHub. https://prachub.com/companies/airbnb?page=6
68. "Airbnb Machine Learning Interview Questions." PracHub, 2026. https://prachub.com/companies/airbnb/categories/machine-learning
69. "Airbnb Software Engineer Coding & Algorithms Interview Questions." PracHub. https://prachub.com/companies/airbnb/positions/software-engineer/categories/coding-and-algorithms
70. Airbnb problem list (water pouring, text justification, KV store, code review, lodging/booking, URL parsing). 1point3acres, 2025. https://www.1point3acres.com/interview/problems/company/Airbnb
71. Glassdoor review RVW5079102 (CSV parser in 45 minutes). Glassdoor, 2014. https://www.glassdoor.sg/Interview/Airbnb-Interview-E391850-RVW5079102.htm
72. "Airbnb interview questions" (57 questions). Taro. https://www.jointaro.com/interviews/companies/airbnb
73. "Airbnb LeetCode" (2019 company-tag list). Scribd, 2019. https://www.scribd.com/document/486147929/Airbnb-LeetCode
74. "Airbnb interview process and top questions." Educative, 2025. https://educative.io/blog/airbnb-interview-process-and-questions
75. "Blind 75 vs. LeetCode patterns: Effective for Airbnb interviews?" Educative, 2025. https://www.educative.io/blog/blind-75-vs-leetcode-patterns-for-airbnb-coding-interviews
76. "Get a Job at Airbnb: Interview Process and Top Questions." Exponent. https://tryexponent.com/blog/airbnb-interview-process
77. "Airbnb Machine Learning Engineer at a Glance." DataInterview. https://www.datainterview.com/blog/airbnb-machine-learning-engineer-interview
78. "Airbnb Machine Learning Engineer." CleverPrep. https://www.cleverprep.com/companies/airbnb/machine-learning-engineer
79. Airbnb Machine Learning Engineer interview questions (page 6). Prepfully. https://prepfully.com/interview-questions/airbnb/machine-learning-engineer?page=6
80. "Airbnb Machine Learning Engineer Interview Process" (TrueInterview guide, mirrored; 3 candidate reports, mainly 2026 Q2). GitHub, 2026. https://github.com/kevin-2023-code/Tech-Interview-Questions/blob/main/companies/airbnb/interview-process-machine-learning-engineer.md
81. "Airbnb Software Engineer Interview Process" (TrueInterview guide, mirrored; 41 candidate reports, 2025 Q4 to 2026 Q2). GitHub, 2026. https://github.com/kevin-2023-code/Tech-Interview-Questions/blob/main/companies/airbnb/interview-process-software-engineer.md
82. "Airbnb" question list with report dates (algorithm, OOP and SQL, Sep 2025 to Sep 2026). GitHub, 2026. https://github.com/kevin-2023-code/Tech-Interview-Questions/blob/main/companies/airbnb.md
83. "Airbnb" company guide (2025-2026 reports). jugaldb/faang-interview-prep, GitHub, 2026. https://github.com/jugaldb/faang-interview-prep/blob/main/docs/companies/airbnb.md
84. LeetCode company-tag mirror: Airbnb. liquidslr/leetcode-company-wise-problems, GitHub, 2025. https://github.com/liquidslr/leetcode-company-wise-problems/tree/main/Airbnb

85. "AirBnB Interview Questions" (classic problems). allaboutjst/airbnb, GitHub. https://github.com/allaboutjst/airbnb
86. 1point3acres post #40 (mirror): MLE-type loop; coding LC 1298; ML design (LTV, host pricing). FlowSight/coding, GitHub. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/40.md
87. 1point3acres post #79 (mirror): two ML design rounds, ConnectN OOD coding, ML experience round. FlowSight/coding, GitHub. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/79.md
88. 1point3acres post #69 (mirror): Connect 4 phone screen with about 10 tests; onsite retryer. FlowSight/coding, GitHub. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/69.md
89. 1point3acres post #65 (mirror): set_value / set_sum / get_value key store. FlowSight/coding, GitHub. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/65.md
90. 1point3acres post #12 (mirror): menu bundles; terrain print and water drop. FlowSight/coding, GitHub. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/12.md
91. 1point3acres post #53 (mirror): rental-services minimum-cost variant. FlowSight/coding, GitHub. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/53.md
92. 1point3acres post #37 (mirror): 16 algorithm questions with clarification tips (menu pruning and all optima, Connect 4, select property). FlowSight/coding, GitHub. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/37.md.md
93. "Academic publications: Airbnb Tech 2025 year in review" (NDCB metric). Airbnb Tech, 2025. https://airbnb.tech/infrastructure/academic-publications-airbnb-tech-2025-year-in-review/
94. "Beyond A/B Test: Speeding up Airbnb Search Ranking Experimentation through Interleaving." Airbnb Tech, 2021. https://airbnb.tech/?p=251
95. "Harnessing the Power of Interleaving and Counterfactual Evaluation for Airbnb Search Ranking." arXiv 2508.00751, 2025. https://arxiv.org/html/2508.00751
96. "Embedding-Based Retrieval for Airbnb Search." Airbnb Tech, 2025. https://airbnb.tech/ai-ml/embedding-based-retrieval-for-airbnb-search/
97. "Beyond Pairwise Learning-To-Rank At Airbnb." Airbnb Tech, 2025. https://airbnb.tech/?p=653

04. Airbnb Senior MLE: Practical ML System Design

> **Scope.** This chapter covers the ML system design rounds in the Airbnb Senior Machine Learning Engineer loop, including the experimentation and A/B testing half of those rounds, for 2024-2026. It lists questions only and gives no answers or solutions.
>
> **How it relates to the older chapters.** Four new search passes (October 2026) were merged and deduped with the prompts in `airbnb-senior-mle/chapters/03-ml-system-design.md`, `10-ml-system-design-architecture-diagrams.md` and `04-ab-testing-experimentation.md`. Those chapters keep the long tail: 140+ search, embedding, pricing and trust sub-questions, full architecture diagrams, and the ERF, CUPED, interleaving and sequential-testing deep dives. This chapter keeps the 24 strongest design prompts. Each one gets a skeleton diagram of the boxes a strong answer must cover. The boxes name components only; they do not say which choice to make.
>
> **Source caveat.** Almost every source is an anonymous self-report (PracHub [19][20][21], 1point3acres, Blind, or TrueInterview-derived GitHub indexes) or a prep-vendor page. Many were read only as search-result snippets or login-gated teasers. Read each [Reported] tag as "a candidate said this was asked", not as verified fact.

Tag legend:

- **[Reported]**: a cited candidate report says the prompt, or a close paraphrase, was asked in an Airbnb MLE loop.
- **[Likely]**: a grounded inference from Airbnb papers, the engineering blog, job postings or prep guides. Nobody was seen being asked it verbatim. A citation next to [Likely] points to the system or source that makes it plausible.

Follow-up bullets are untagged unless a source reports them. They are the probes interviewers typically pull on.

How this round works

- **Where it sits in the loop.** Expect the design rounds to decide the outcome. Reported loops:
  - An April 2026 Senior MLE loop had **four rounds built around ML systems, ranking and marketplace scenarios**, with past-project deep dives mixed in. It ended in rejection "on calibration" [2].
  - A September 2026 Senior+ loop (Relevance and Personalization) covered document redaction, project decisions, search ranking without a text query, and retrieval-assisted support. Team matching followed [1].

    - A TrueInterview-derived summary says the loop is "filtered by design depth, not by code", with 1-2 decisive ML design rounds [9].
    - 1point3acres loops had two ML design rounds each: LTV plus host pricing [11]; listing recommendation plus price prediction [12]; property damage claims plus pricing [3].
    - An ML platform loop had a coding screen, then an onsite with code review, one general system design round, one ML system design round and behavioral [7].
- **Two flavors of design round.**
    - The **ML design** round is "typically a search or recommendation question" [6].
    - The **ML practitioner** round asks you to propose an ML solution to one of Airbnb's real, current engineering problems [6].
    - The **ML experience** (project) round can turn into an unplanned design question [8][12].
    - Prep guides also list a "debug a model with unexpected behavior" round [22].
- **Duration and tools.** Rounds last about 45-60 minutes over video, with a shared doc or whiteboard [9]. You walk through the design verbally; there is no code to run. Interviewers steer partway through, for example from regression to bucketized classification in the LTV round [9].
- **What the senior bar looks like.**
    - Drive the problem from business goal to target definition, labels, features, model, offline evaluation, online experiment, serving, and monitoring and debugging in production [1][9].
    - Defend every choice against a simpler and a more complex alternative. Reported probes ask why not a more complex model, how you would trade off precision, recall, latency and infrastructure cost, how you would tune thresholds, and how you would choose between factorization machines and an embedding plus DNN model [2].
    - Show business sense. The April 2026 write-up says "the team really value business sense and decision making" [2].
    - Show two-sided marketplace thinking, covering guests, hosts, supply and demand [6][24].
- **Experimentation weight for this role.** Assume every design ends with "how would you launch and measure it?". Prepare for interference-aware randomization (guest, listing, market or two-sided), guardrails, lead-day bias in pricing tests, and what to do when offline and online metrics disagree [2][53][54][56].

Top 15 to practice first

All 15 are [Reported] prompts, ordered by number of independent reports, then by importance.

1. Search ranking with no text query (destination, dates, guests only). [Reported] [1][10][13] (see Q1)
2. Pricing prediction and Smart Pricing price recommendations for hosts. [Reported] [3][4][5][11][12] (see Q2)
3. Retrieval-assisted (RAG) customer support for guests, hosts and agents. [Reported] [1][9][17] (see Q3)

4. User embedding and two-stage semantic retrieval and ranking for booking conversion. [Reported] [2][10][14] (see Q4)
5. Offline metrics win but the online A/B test is flat or negative: diagnose it and decide whether to ship. [Reported] [2] (see Q5)
6. Guest or listing lifetime value (LTV). [Reported] [9][10][11][15] (see Q6)
7. Photo and listing quality models that feed search. [Reported] [9][19][20] (see Q7)
8. Property damage claims: triage, payout estimate and fraud. [Reported] [3] (see Q8)
9. "Family friendly" listing classifier with noisy host labels and cold start. [Reported] [2] (see Q9)
10. Threshold choice and model choice (FM vs embedding plus DNN; precision, recall, latency, cost). [Reported] [2] (see Q10)
11. Real-time content personalization (homepage and categories feed). [Reported] [2][22] (see Q11)
12. PII and sensitive-information detection and redaction in documents and messages. [Reported] [1] (see Q12)
13. Listing recommendation and similar-listing embeddings. [Reported] [12] (see Q13)
14. Relisting detection (the same property relisted under a new listing). [Reported] [9] [16] (see Q14)
15. The ML parts of the accommodation booking funnel (booking likelihood from search to stay). [Reported] [2] (see Q15)

---

1. Search ranking without a text query

Prompt. "Design an ML ranker for Airbnb home search when the user gives no text query, only destination, dates and guest count. Separate the features available at ranking time from future outcomes, set product objectives and guardrails, then pick labels and an architecture. Cover data, evaluation and how you would debug it in production." [Reported] [1][10][13] (Senior+ Relevance and Personalization onsite, September 2026. The broader "design Airbnb search ranking / personalized search ranking" prompt is also reported [6][9] [18].)

Maps to: Airbnb stays search ranking, which orders listings by estimated booking probability, evolved from GBDT to deep neural networks [44][45][46]. It also maps to the Relevance and Personalization team [60].

```
SERVING
[Query: dest, dates, guests] -> [Availability + geo filter] -> [Candidate retrieval]
         -> [First-pass ranker] -> [Second-pass ranker] -> [Re-rank: diversity,
            business rules] -> [Results list / map]
[Feature store: listing | guest | query | real-time session] --> both rankers
TRAINING LOOP
[Impression + click + booking logs] -> [Label + position-bias handling]
-> [Point-in-time feature join] -> [Train] -> [Offline NDCG eval]
-> [Interleaving / A/B] -> [Monitoring + prod debugging] -> back to logs
```

Follow-ups:

- With no query text, what replaces query relevance: location intent, dates, price range, guest history, season?

- Which features exist at ranking time, and which are future outcomes that would leak into training (eventual booking, later reviews, host cancellation)?
- What is the primary objective (bookings, booking value, uncancelled stays, long-term guest and host value), and which guardrails protect hosts and marketplace health?
- Bookings are rare and many searches end without one. Would you train on bookings, clicks, long views, wishlists or contacts to the host? How would you combine them?
- How do you treat a booking request the host declined or later cancelled?
- Would you frame the model as pointwise, pairwise or listwise, and how do you build training pairs from a session with one booked listing?
- How do you correct for position bias in logged results, and what position value do you feed at inference time?
- How do you give cold-start listings controlled exposure without hurting conversion?
- When does moving from a GBDT to a neural ranker pay off, and what usually breaks (feature scaling, sparse IDs, price sensitivity)?
- What is the latency budget for each stage, and which features are batch versus real-time?
- How would ranking change for the map view, for Experiences, or for long-term stays? [Likely] [29][47]

2. Pricing prediction and Smart Pricing

Prompt. "Design a pricing prediction / price recommendation (Smart Pricing) model for Airbnb hosts. Cover the features (listing, seasonality, demand, comparable listings), the label when you only observe booked prices, and how you evaluate it." [Reported] [3][4][5] [11][12] (1point3acres onsites 2022-2025: "ML round 1: a price prediction system" [5]; "design a system to help hosts set prices" [11]; "price prediction" [12]. Prep guides also list "dynamic pricing recommendation engine" [22][25][26].)

Maps to: Airbnb dynamic pricing (Smart Pricing), which combines a booking-probability model with a customized price regression [51]. It also maps to the host calendar's comparable-listing embeddings [42].

```
SERVING
[Listing + night + lead time] -> [Demand / booking-prob model] -> [Price strategy
         model] -> [Host constraints: min/max, rules] -> [Suggested price + reason]
[Comps: similar listings] [Seasonality + events] [Market supply/demand] --> models
TRAINING LOOP
[Calendar + price + booking logs] -> [Label: booked / unbooked nights, censoring]
-> [Train] -> [Offline eval: price regret, calibration] -> [Host adoption logs]
-> [Pricing A/B with lead-day handling] -> [Monitoring] -> back to logs
```

Follow-ups:

- What exactly are you predicting: the market-clearing price, booking probability at a given price, or the host's revenue-maximizing price?
- You only see the price for booked nights and the host's asking price for unbooked ones. How do you build a label from that censored, host-chosen data?
- Observed prices are set by hosts, not randomized. How do you separate the price effect from listing quality (endogeneity)?

- What feature families matter (listing attributes, location, seasonality, local events, lead time, comps, market occupancy)?
- How do you pick comparable listings, and could listing embeddings do it better than hand-built rules?
- What offline metric fits when the "right price" is never observed?
- How do you respect host goals (occupancy versus nightly rate) and host-set minimums?
- How do you explain a suggestion so that hosts adopt it?
- Many hosts in one market adopt the suggestion. How do you account for that feedback loop?
- Would you randomize a pricing test by listing, by night or by market? Why does lead-day bias matter? [Likely] [56]
- How would you replace a rule-based pricing function (weekend, event and competitor bounds) with ML, and how would you prove it is better? [Likely] [27]
- How does the system behave for a brand-new listing with no booking history?
- How would damage or cancellation risk enter pricing or security-deposit decisions? (One reported loop paired a pricing round with a damage-claims round.) [Likely] [3]

3. Retrieval-assisted (RAG) customer support

Prompt. "Design a retrieval-assisted customer support system for Airbnb guests and hosts that answers questions or helps agents. Cover knowledge-base retrieval, LLM generation, evaluation, guardrails and escalation to humans." [Reported] [1][9][17] (Senior+ onsite, September 2026 [1]. 2026 senior loops added retrieval and ranking prompts for GenAI support tools [9]. A related prompt is "generate customer-support instructions from company policies" [17].)

Maps to: Airbnb Community Support chat and voice AI assistants. The team's postings list SFT, RL, distillation, RAG/search, LLM evaluation, feedback-based learning and guardrails [62][64][65]. An October 2025 paper covers LLMs in Airbnb support [67], and an earlier case study covers support text generation [68].

```
SERVING
[Guest/host message + reservation ctx] -> [Intent + risk classifier]
  -> [Retriever: help center, policies, past cases] -> [Re-ranker]
  -> [LLM generator (distilled)] -> [Guardrails: policy, PII, safety]
  -> [Answer | agent suggestion | escalate to human]
TRAINING / EVAL LOOP
[Tickets + resolutions + CSAT] -> [Human + LLM-judge labels] -> [SFT / pref tuning]
-> [Offline eval: groundedness, resolution] -> [A/B] -> [Feedback logs] -> back
```

Follow-ups:

- Which intents should the bot answer itself, which should it draft for an agent, and which must go straight to a human (safety, emergencies, high-value refunds)?
- How do you chunk and index help-center and policy content that changes often and differs by country?
- How do you retrieve reservation-specific context (dates, cancellation policy, payment status) safely?

- Would you use RAG, SFT or both? When does distilling into a smaller model pay off on latency and cost?
- How do you build an offline evaluation set, and which metrics matter (groundedness, policy correctness, resolution, tone)?
- When can you trust an LLM-as-judge, and how do you calibrate it against human reviewers?
- Which guardrails stop the model promising refunds or exceptions that policy does not allow?
- How do you detect and hand off escalations without losing context?
- Which online metrics decide the launch (resolution rate, contact rate, CSAT, handle time), and would you randomize by ticket, by user or by agent?
- How do you learn from agent edits and user feedback without amplifying the model's own mistakes?
- How do you protect against prompt injection from user messages or listing text?
- What is the per-turn latency budget for chat versus voice?

4. User embeddings and two-stage semantic retrieval and ranking

Prompt. "Design a user embedding plus two-stage semantic retrieval and ranking system for Airbnb search, optimized for booking conversion. Walk through candidate generation, the ranker, features, labels and online serving." [Reported] [2][10][14] (April 2026 Senior MLE onsite [2]. "Design user-embedding semantic search", reported April 2026 [10][14].)

Maps to: Airbnb embedding-based retrieval (a two-tower model with listing embeddings precomputed daily, targeted negative sampling, IVF chosen over HNSW, and traffic-replay evaluation) [39][40][41]. It also maps to user-type embeddings [43] and BiListing multimodal listing vectors [52].

```
SERVING
[Guest ctx + query] -> [User / query tower] -> [ANN index (listing tower,
    precomputed)] + [Availability / geo / price filters] -> [Top-K candidates]
  -> [Ranker (booking objective)] -> [Results]
[Batch: listing embeddings daily] [Stream: session events -> user embedding]
TRAINING LOOP
[Search sessions + bookings] -> [Positives + negative sampling] -> [Two-tower train]
-> [Offline recall@K, traffic replay] -> [Index rebuild] -> [A/B] -> back
```

Follow-ups:

- How do you build a user embedding from sparse, long-gap travel histories, and how do you refresh it in-session?
- What counts as a positive (booked, wishlisted, long view), and which negatives do you sample (in-batch, same-market, hard negatives)?
- Why might random negatives produce embeddings that mostly learn geography?
- Price and availability change daily. What stays inside the embedding, and what goes into filters at query time?
- When would you pick IVF over HNSW, given index rebuild cost and frequent filtering?
- How do you evaluate retrieval recall separately from end-to-end ranking quality, when inventory changes every day?

- How do you share the latency budget between retrieval and ranking? Each stage scores about an order of magnitude fewer homes than the one before [40].
- How do you represent a new guest or a new listing in the embedding space?
- How do you keep retrieval from narrowing what the ranker can ever learn from (a feedback loop)?
- How would you fold photo and text embeddings into one listing vector, and how would you measure the NDCG and revenue impact? [Likely] [52]
- What do you A/B test first, retrieval or ranker, and how do you attribute the gain?

5. Offline wins, online flat: diagnose and decide

Prompt. "Your new ranking model wins on offline metrics such as NDCG/AUC but is flat or negative in the online A/B test. How do you diagnose the gap, and what do you ship?" [Reported] [2] (April 2026 Senior MLE: "What do you do when the offline metric and the online metric disagree?". A related prompt, "debug a model with unexpected behavior", appears in prep guides [22], and the Senior+ report advises preparing production debugging [1].)

Maps to: Airbnb's deep-learning search papers, which describe many offline wins that failed online [44][45], plus interleaving for faster ranking tests [58].

```
[Offline eval set] --vs-- [Online A/B readout]
       |                          |
[Label / objective check]  [Experiment health: SRM, exposure, power, novelty]
[Leakage + split check]    [Interference between arms]
[Candidate-set mismatch]   [Segment / market breakdown]
       \                          /
      [Training-serving skew: feature parity + latency]
                  |
        [Decision: ship / iterate / kill] -> [Post-launch monitoring]
```

Follow-ups:

- What is your first check, the experiment or the model?
- How do you test for training-serving skew, feature leakage and stale features?
- Was the offline candidate set the same as what the online ranker actually scored?
- Could position bias or a feedback loop (training on control's rankings) explain the gap?
- Does the offline metric measure the online objective (bookings versus clicks, uncancelled stays)?
- Did the new model add latency, and how would you separate the latency effect from the quality effect?
- Could interference between arms or novelty effects hide a real gain?
- Which segments or markets moved in opposite directions?
- Would interleaving have given a faster or clearer read? [Likely] [58]
- The result is +0.3% bookings with a wide interval, plus extra latency and infrastructure cost. Do you ship? [Likely]
- How would you build an offline metric that predicts online wins better next time?
- Conversion drops in one market the day after a new ranker ships. Walk through rollback criteria and root-cause analysis (data, features, serving skew, model). [Likely] [1][22]

- The interviewer pushes back on your launch call. How do you defend it on business grounds? [Reported] [2]

6. Guest or listing lifetime value (LTV)

Prompt. "Design a system to calculate customer (or listing) lifetime value." The interviewer first made the candidate define "lifetime value", then steered the design from regression to bucketized classification. [Reported] [9][10][11][15] (1point3acres loop [11]; listing LTV reported May 2026 [10][15].)

Maps to: Airbnb's future-value measurement, used to standardize trade-offs across teams (see the older chapter `airbnb-senior-mle/chapters/03-ml-system-design.md`).

```
SERVING
[Guest / listing entity] -> [Feature snapshot at time t] -> [LTV model]
  -> [Calibrated value or value bucket] -> [Consumers: marketing, ranking,
     host acquisition, incentives]
TRAINING LOOP
[Booking + revenue + churn history] -> [Target definition: horizon, margin]
-> [Censoring-aware labels] -> [Train] -> [Offline eval: calibration, rank]
-> [Backtest by cohort] -> [Downstream A/B] -> [Drift monitoring] -> back
```

Follow-ups:

- How do you define LTV: revenue or contribution margin, over what horizon, and for which entity?
- How do you handle right-censoring for recent users and listings?
- Why might bucketized classification beat regression on a heavy-tailed target?
- How do you predict LTV for brand-new guests or listings?
- Which features are allowed without leaking future behavior?
- How do you evaluate it: calibration by decile, rank correlation, or business backtests?
- Who consumes the score, and how would each consumer misuse it?
- How would you validate that decisions based on LTV (incentives, host acquisition spend) actually raise long-term value?
- How often do you retrain, and how do you detect drift after a macro shock (for example, a travel slowdown)?

7. Photo and listing quality models

Prompt. "Design photo and listing quality models for Airbnb. Cover the labeling strategy, feature engineering (image and text), model selection, and how quality scores feed into search." [Reported] [9][19][20] (PracHub-listed Airbnb prompt [19][20]; cover-photo and high-quality-listing prompts in a 2026 summary [9].)

Maps to: Airbnb Categories built with ML and human-in-the-loop review [70], and to listing-quality models used in merchandising and ranking [20].

```
SERVING
[Listing photos + text + amenities] -> [Image encoder] + [Text encoder]
  -> [Quality model / cover-photo selector] -> [Quality score + reasons]
  -> [Consumers: search ranking feature, host tips, merchandising]
TRAINING LOOP
[Human ratings] + [Engagement proxies] + [Review signals] -> [Label strategy]
-> [Train + calibrate] -> [Offline eval: agreement with raters]
-> [Ranking A/B + host-side metrics] -> [Rater audits] -> back
```

Follow-ups:

- What does "quality" mean: photo aesthetics, accuracy versus reality, completeness, or predicted guest satisfaction?
- Would you label with human raters, engagement proxies (CTR on a photo) or post-stay reviews? What bias does each bring?
- How do you keep the model from learning that "expensive-looking" equals "high quality"?
- How do you pick the cover photo per listing, and could it be personalized per guest?
- How would the score enter search: as a ranker feature, as a filter, or as a boost? What feedback loop does that create?
- How would you measure the model's effect on hosts (photo uploads, edits) as well as guests?
- How do you handle new listings with few photos, or photos that are not of the home?
- What inter-rater agreement do you need before you trust the labels?
- How do you serve image models cost-efficiently: batch on upload, or on demand?
- How do you audit quality scores for bias against particular regions or property types?

8. Property damage claims

Prompt. "Design an ML system for property damage claims, for example triaging and estimating claims or detecting fraudulent ones, using claim text, photos and host/guest history. Cover labels, features, class imbalance, human review and metrics." [Reported] [3] (1point3acres Airbnb MLE onsite, 2025.)

Maps to: Airbnb's support products. A staff MLE posting lists "automated property damage claim item entity extraction" and "agent tools to investigate claims and possible fraud through real-time image, text, and user similarity search" [63].

```
SERVING
[Claim: text + photos + receipts] -> [Entity extraction: items, amounts]
  -> [Similarity search: image / text / user] -> [Validity + fraud model]
  -> [Payout estimator] -> [Router: auto-approve | agent review | investigate]
[Host / guest / reservation history features] --> models
TRAINING LOOP
[Resolved claims + agent decisions + appeals] -> [Labels (delayed)]
-> [Train + calibrate] -> [Offline eval by cost] -> [Shadow / A/B] -> back
```

Follow-ups:

- What is the label: approved claim, amount paid, or claim confirmed fraudulent? How do you handle months of delay before resolution?
- Labels come from past agent decisions. How do you avoid learning the agents' biases?
- How do you detect reused or stock photos, and claims copied across listings?

- How would you estimate payout amounts from free-text item lists and receipts?
- Where do you set thresholds for auto-approval, agent review and investigation, given each error's cost?
- How do you handle heavy class imbalance for fraud?
- How do you keep the system fair to both the host filing and the guest being charged?
- What does the agent see, and how do you measure that the tool makes agents faster and more accurate?
- How would you test the system online without exposing guests to wrong charges?
- How would you monitor for new fraud patterns after launch?

9. "Family friendly" listing classifier

Prompt. "Design a classifier that tags listings as 'family friendly' (a filter for guests searching with children). How do you get labels (review keywords, post-trip ratings), which features do you use (listing metadata, review text embeddings, image features), which model do you pick, and how do you evaluate it?" Follow-up: "Some hosts mislabel their listings, and new listings have no reviews. How do you handle noisy host labels and cold start?" [Reported] [2] (April 2026 Senior MLE onsite; evaluated on precision, recall, F1 and calibration.)

Maps to: Airbnb Categories and attribute classification with human-in-the-loop review [70].

```
SERVING
[Listing metadata + amenities] [Photos] [Review text] -> [Feature encoders]
  -> [Classifier] -> [Calibrated prob] -> [Threshold] -> [Tag / filter in search]
TRAINING LOOP
[Weak labels: host tags, review keywords, ratings from families]
  + [Human-labeled gold set] -> [Label denoising] -> [Train]
-> [Offline P / R / F1 / calibration] -> [Search A/B] -> [Complaint feedback] -> back
```

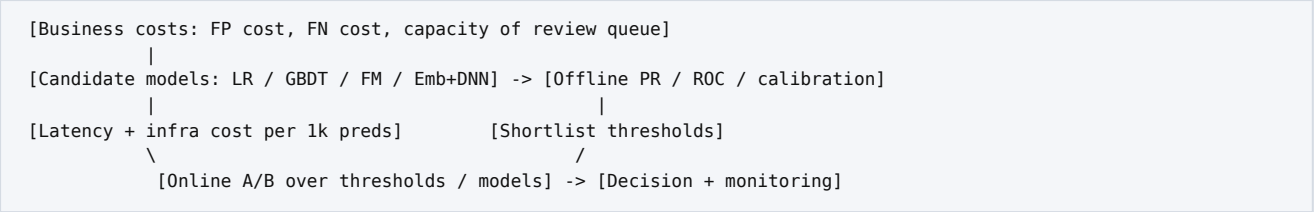
Follow-ups:

- Which label sources would you combine, and how do you estimate each one's noise rate?
- Hosts may tag listings "family friendly" to gain traffic. How do you treat host-provided labels?
- How do you classify a new listing with no reviews?
- Which errors are worse here, false positives (an unsafe listing shown to families) or false negatives (lost supply)?
- How do you pick the threshold? [Reported] [2] Would it differ by market, and what risk does that bring?
- Why does calibration matter if the output is only a binary tag?
- How would you evaluate with a human-labeled gold set, and how big must it be?
- How do you measure the tag's effect on bookings and post-stay satisfaction for families?
- How would you generalize to dozens of other attributes (pet friendly, accessible, workspace) without one model per tag?
- Why choose this model over a more complex multimodal one, given latency and cost? [Reported] [2]

10. Threshold choice and model choice under cost constraints

Prompt. "For a classifier in production at Airbnb, how would you pick the decision threshold, balancing precision, recall, latency and infrastructure cost, and how would you validate that threshold online? For the ranking or CTR model, would you use factorization machines or an embedding plus DNN model?" [Reported] [2] (April 2026 Senior MLE onsite. The candidate argued that the threshold choice needs an online A/B test, and the round also covered why not a more complex model.)

Maps to: Airbnb search ranking's evolution from GBDT to neural networks [44][46], and trust and classification thresholds generally.



Follow-ups:

- How do you turn business costs into a threshold rather than taking the default of 0.5?
- How would an A/B test choose between shortlisted thresholds?
- When do factorization machines win on sparse feature interactions and serving cost?
- What does an embedding plus DNN model buy you for long-term user behavior, and what does it cost?
- How do latency and infrastructure cost enter the model choice for a search-path model?
- Why does calibration matter when scores feed thresholds or a multi-objective blend?
- How do you re-tune the threshold after a retrain shifts the score distribution?
- What monitoring tells you that the threshold has drifted out of its intended operating point?

11. Real-time content personalization (homepage and categories)

Prompt. "Design a real-time content personalization system for Airbnb, for example a personalized homepage or a categories/experiences feed." [Reported] [2][22] (April 2026 Senior MLE report [2]; prep guide reports [22]. The ML design round is "usually a search or recommendation problem tied to Airbnb's real product problems", for example personalized homepage recommendations [6].)

Maps to: Airbnb real-time personalization with in-session embeddings [43]. It also maps to guest-journey sequence models (long-term and short-term history, decoupled from real-time ranking) [50] and the Relevance and Personalization team's low-latency, batch plus real-time pipelines [60].

```
SERVING
[App open: guest id, location, season] -> [Session event stream]
  -> [Candidate sources: categories, destinations, listings, experiences]
  -> [Sequence encoder (long + short term)] -> [Ranker] -> [Page layout / slots]
[Batch features (daily)] + [Real-time features (seconds)] --> ranker
TRAINING LOOP
[Impressions + engagement + bookings] -> [Labels] -> [Train]
-> [Offline eval] -> [A/B + holdout] -> [Freshness / drift monitors] -> back
```

Follow-ups:

- What is the objective for a homepage with no query: clicks, searches started, wishlists or bookings?
- How do you represent long-term booking history versus short-term views, and keep latency low?
- What freshness do in-session signals need, and what streaming infrastructure supports it?
- How do you rank heterogeneous items (destinations, categories, listings) on one page?
- How do you handle logged-out and brand-new guests?
- How do you avoid a filter bubble and still let guests explore?
- Which features are computed in batch and which in real time? Why?
- How do you keep training and serving features consistent?
- How would you A/B test a layout change when it shifts which searches guests run next?

12. Sensitive-information detection and redaction

Prompt. "Design an ML system that detects and redacts sensitive information, such as PII, contact details and payment info, in documents and messages on Airbnb. Cover labeling, model choice, the precision/recall threshold and serving." [Reported] [1] (Senior+ onsite, September 2026, "document redaction".)

Maps to: guest-host messaging ML (scam and off-platform payment detection) [66] and support document handling [62].

```
SERVING
[Message / uploaded doc (OCR if image)] -> [Rules + regex pre-pass]
  -> [NER / span tagger] -> [Context classifier (intent: off-platform pay?)]
  -> [Policy: redact | warn | block | send to review] -> [Delivered content]
TRAINING LOOP
[Human-annotated spans] + [Synthetic PII] + [Reviewer decisions]
-> [Train] -> [Span-level P / R by entity type] -> [Shadow mode]
-> [A/B on messaging metrics] -> [Missed-PII reports] -> back
```

Follow-ups:

- Which entity types matter (phone, email, address, payment, ID documents), and which are allowed at which booking stage?
- How do you get labeled spans without exposing annotators to real PII?
- When are regex rules enough, and when do you need a learned tagger or an LLM?
- How do you catch obfuscated contact details ("five five five, at gmail")?

- Which error is worse, a missed redaction or an over-redaction that breaks a legitimate message? How does that set the threshold?
- How do you handle many languages and scripts?
- What is the latency budget for real-time messaging versus batch document processing?
- How would you extend the system to detect scams and off-platform payment attempts? [Likely] [66]
- How do you measure success online (scam reports, off-platform leakage, message completion)?
- How do you audit the system and keep logs without storing the very data you redact?

13. Listing recommendation and similar listings

Prompt. "Design an Airbnb listing recommendation system" (1point3acres loop, alongside price prediction). [Reported] [12] The common variant is: "Design a similar-listings carousel on the listing page using listing embeddings learned from guest click sessions. How do you build negatives, correct for sampling bias, handle cold-start listings and the booking signal, and evaluate offline and online?" [Likely] [19][42]

Maps to: Airbnb listing embeddings for similar listings and real-time personalization (KDD 2018) [42][43]. The same embeddings feed host-side products such as host-calendar comparable listings [42].

```
SERVING
[Viewed listing + guest ctx] -> [Embedding lookup] -> [ANN neighbours]
  -> [Filters: dates, guests, market] -> [Re-ranker] -> [Similar-listings carousel]
TRAINING LOOP
[Click sessions + booked listing] -> [Training pairs from sessions]
-> [Negative sampling] -> [Train embeddings]
-> [Offline: rank of booked listing] -> [A/B on carousel CTR + bookings] -> back
```

Follow-ups:

- How do you define a session and its context window?
- How do you make the booked listing influence every listing in the session?
- Why add same-market negatives, and what goes wrong without them?
- How do you embed a brand-new listing with no sessions?
- How do you evaluate embeddings offline before shipping them?
- How do guest-side and host-side (comparable listings for pricing) objectives differ? [Likely] [42]
- How often do you retrain embeddings, and how do you keep downstream consumers consistent?
- How do you stop the carousel from cannibalizing search bookings, and how would you measure that?
- When would a sequence model replace skip-gram embeddings?

14. Relisting detection

Prompt. "Design Airbnb relisting detection": find hosts who take down a listing (often after bad reviews or enforcement) and relist the same property as new. The interviewer steered into geohashing. [Reported] [9][16]

Maps to: Airbnb trust and safety and listing integrity [26].

```
SERVING
[New / edited listing] -> [Candidate blocking: geohash, host graph, device]
  -> [Pairwise features: photo sim, text sim, address, amenities, host links]
  -> [Match model] -> [Score] -> [Action: auto-link | review queue | allow]
TRAINING LOOP
[Confirmed relists from investigations] + [Hard negatives: same building]
-> [Train + calibrate] -> [Offline P / R at review capacity]
-> [Shadow] -> [Reviewer feedback + appeals] -> back
```

Follow-ups:

- How do you generate candidate pairs cheaply among millions of listings (geohash cells, locality-sensitive hashing)?
- How do you separate a relisted home from a legitimate neighbor in the same building?
- Which photo and text similarity signals survive deliberate edits (crops, filters, rewording)?
- Where do labels come from, and how biased are investigation-driven labels?
- How do you set thresholds given a fixed human review capacity?
- What should happen to the reviews and history once a relist is confirmed?
- How do you measure success online without a clean ground truth?
- How does this connect to the fake-listing and review-fraud detectors?

15. ML across the booking funnel

Prompt. "Design the ML parts of an accommodation booking system for Airbnb, such as ranking and predicting booking likelihood across the guest-to-booking funnel." [Reported] [2] (April 2026 Senior MLE report. Prep sites list "design an accommodation booking system for Airbnb (availability, reservations, consistency) and explain where ML fits into it" [28][35].)

Maps to: the multi-stage funnel of search, listing view, contact or request, host acceptance, booking and stay, modeled by Journey Ranker [49].

```
[Search] -> [Listing view] -> [Request / Instant Book] -> [Host accept]
    |            |                    |                       |
[Rank model] [P(book | view)]  [P(host accepts)]       [P(cancel before stay)]
    \____________|_____________________|_____________________________/
                 [Shared features: guest, listing, calendar, price]
TRAINING: [Funnel event logs] -> [Per-stage labels, delayed] -> [Multi-task train]
          -> [Offline per-stage + end-to-end eval] -> [A/B] -> back
```

Follow-ups:

- Where along the funnel does ML change a decision, and where would rules suffice?
- How do you combine per-stage probabilities into one ranking score?
- How do calendar accuracy and availability consistency affect your labels?

- How would you predict host acceptance, and use it in ranking without hurting guest experience? [Likely] [19]
- How would you predict guest or host cancellations, and what action would you take on the prediction? [Likely] [20][28]
- How do you handle the delay between booking and stay when labeling cancellations?
- Which latency budgets apply at each stage?
- How do you measure the end-to-end effect when one stage's model changes the next stage's inputs?

---

16. Two-sided marketplace A/B test for a ranking or pricing launch

Prompt. "You launch a new ranking or pricing model. How do you design the A/B test in Airbnb's two-sided marketplace, where treatment and control compete for the same inventory? Would you randomize by guest, by listing or by market, and when is each design unbiased?" [Likely] [33][34][53][54] (Experiment-design prompts appear in the Airbnb question index [10] and an Airbnb data-science prep guide [37], role unspecified. Interference is a core topic in Airbnb-coauthored research [53].)

Maps to: Johari, Li, Liskovich and Weintraub (Stanford and Airbnb): customer randomization is unbiased in demand-constrained markets, listing randomization in supply-constrained ones, and two-sided randomization (TSR) is proposed [53]. Holtz and Aral show that ignoring interference can inflate estimates by up to 100% [54]. Airbnb's own experimentation write-ups cover the platform side [59].

```
[Hypothesis + primary metric + guardrails (guest AND host)]
     -> [Randomization unit: guest | listing | two-sided | market cluster]
     -> [Power calc with clustering] -> [Assignment + exposure logging]
     -> [Run: novelty window, SRM checks]
     -> [Analysis: variance reduction, interference diagnostics, segments]
     -> [Launch decision] -> [Holdout / long-term readout]
```

Follow-ups:

- How does interference bias a guest-randomized test when inventory is limited?
- When would you randomize listings instead, and what bias does that leave?
- How does two-sided randomization work, and what bias-variance trade-off remains?
- When is market-level or cluster randomization worth the loss in power?
- How do you compute power when randomizing clustered units?
- Which guest metrics and which host metrics do you track as primary and guardrail?
- A new search filter raised bookings but lowered host response rates. Would you launch? [Likely] [34]
- How would you detect novelty effects in a ranking test?
- How would you measure cannibalization of a new category, with holdout regions or synthetic controls? [Likely] [34]
- Which pitfalls (peeking, dilution, marketplace-level bias) should the platform guard against automatically? [Likely] [57]
- Walk through the experiment from hypothesis to launch call. [Likely] [38]

17. Pricing experiments on expiring inventory

Prompt. "How would you run a pricing (or fee) experiment on Airbnb, where nights are expiring inventory and bookings for a given night arrive over many lead days? How do you randomize, correct for lead-day bias, and avoid the standard estimator pointing you in the wrong price direction?" [Likely] [33][55][56][57]

Maps to: Airbnb's CIKM 2023 paper on lead-day bias in pricing experiments, where bias correction limits rollout to nights far enough from the lead day [56]. It also maps to price experimentation and interference, where canonical estimators can move prices the wrong way in Airbnb-like markets [55].

```
[Price / fee change] -> [Unit: listing-night | listing | market]
  -> [Treatment window by stay date + lead day]
  -> [Booking curve per night (bookings arrive over lead days)]
  -> [Estimator + bias corrections]
  -> [Short-term vs long-term readout] -> [Decision + guardrails (host earnings)]
```

Follow-ups:

- Why can the short-term result move opposite to the long-term effect?
- What is lead-day bias, and how does restricting treatment by lead day correct it?
- How does interference between treated and control nights or listings bias the estimate?
- Should you randomize listings, nights or markets for a price change?
- Which host-side metrics (earnings, occupancy, churn) guard the launch?
- How long must the test run before the booking curves for treated nights are complete?
- How would you handle host overrides of suggested prices during the test?
- How do you communicate a pricing test to hosts, and does that communication bias it?

18. Fraud and trust: fake bookings, fake listings, account takeover and review fraud

Prompt. "Design a real-time fraud detection system for fake bookings or fake listings on Airbnb. Cover labels from chargebacks and investigations, delayed feedback, class imbalance, latency, and how you trade off the friction a false positive causes against fraud loss." [Likely] [25][26][27][30][31] Variants: account takeover and payment fraud with layered defenses [22][27]; review fraud using new accounts, shared IP or device between reviewer and host, and copied text [26].

Maps to: Airbnb trust and safety. Postings list payments, fraud prevention and anti-discrimination work [69]. A guide says trust prompts make up a large share of Airbnb MLE design rounds [31].

```
SERVING
[Event: signup | login | listing | booking | payment | review]
  -> [Identity + device + graph features] -> [Rules layer] -> [Risk model]
  -> [Action: allow | friction (verify) | hold | block | review]
[Real-time feature store] [Entity graph: user-device-IP-card]
TRAINING LOOP
[Chargebacks + investigations + appeals (delayed)] -> [Labels]
-> [Train with imbalance handling] -> [Offline cost-weighted eval]
-> [Holdout / champion-challenger] -> [Drift + attack alerts] -> back
```

Follow-ups:

- Labels arrive weeks later (chargebacks). How do you train and evaluate on fresh data?
- The model's own blocks hide outcomes. How do you avoid that selection bias (holdouts, exploration)?
- How do you trade off fraud loss against the friction a false positive puts on a real guest?
- What latency budget applies at booking and payment time?
- Which graph signals (shared devices, cards, IPs) help most, and how do you serve them in real time?
- How do you respond in the first 48 hours after a new fraud pattern appears?
- How would you detect review fraud, and what does it mean for ranking features built on reviews?
- How do you audit for proxy discrimination in risk decisions?
- Is a random holdout from the fraud model ethical or practical? What would you propose instead?

19. Multi-task ranking that balances guests and hosts

Prompt. "Design a multi-task search ranker that uses intermediate guest actions (clicks, wishlists, contacting the host) as milestones toward booking, and balances guest relevance with host success and marketplace health. Which objective and which guardrail metrics would you use?" [Likely] [25][49]

Maps to: Journey Ranker, Airbnb's multi-task deep learning model over guest-journey milestones [49].

```
[Shared features] -> [Shared towers] -> [Heads: click | long view | wishlist |
                                         contact | book | host accept | cancel]
                                    -> [Score combiner (weights)] -> [Ranking]
TRAINING: [Journey logs across sessions/days] -> [Per-head labels] -> [Joint train]
          -> [Per-head offline eval] -> [Weight tuning] -> [A/B: guest + host]
```

Follow-ups:

- Why model the whole journey rather than a single search?
- How do you weight the heads, and how do you re-tune after one head is retrained?
- How do host-side outcomes (acceptance, cancellations, earnings) enter guest-facing ranking?
- Which guardrails protect new hosts from losing exposure?
- How do you handle conflicting gradients between heads?

- How do you evaluate a multi-task model offline when the heads disagree?
- How do you attribute an online win to a specific head?
- Searches span many days before one booking. How does that change labels and logging?

20. Diversity and map ranking

Prompt. "Search results are too homogeneous. How would you change the ranker to promote diversity, given that a listing's booking probability depends on the other listings shown? And how would you rank the map view differently from the list view, given that position-bias assumptions built for lists break down on maps?" [Likely] [47][48]

Maps to: "Learning To Rank Diversely at Airbnb" [48] and Airbnb's map-ranking material [47].

```
[Ranked candidates] -> [Context-aware re-ranker]
  -> [Diversity objective: price, type, location] -> [List slots]
  -> [Map pin selection] -> [Map view]
TRAINING: [Impression logs with full result set + map viewport]
          -> [Context-dependent labels] -> [Train] -> [Offline diversity + NDCG]
          -> [A/B: list and map separately]
```

Follow-ups:

- Why is a listing's booking probability not independent of the other results?
- How would you measure diversity, and how would you trade it off against relevance in the loss?
- How does user attention on a map differ from attention on a list?
- How do you discount position when there is no top-to-bottom order?
- Which pins do you show when hundreds of listings fit the viewport?
- How do you log impressions on a map?
- How would you A/B test list and map ranking without one confounding the other?
- How would a re-ranker that sees the whole result set fit within the latency budget?

21. Cold start for new hosts, listings and guests

Prompt. "Design an ML system for new hosts and new guests in Airbnb ranking or recommendations, with cold start as the main focus. Should features be computed in real time or in batch?" [Likely] [23][24]

Maps to: cold-start handling in Airbnb search ranking [45] and listing embeddings [42].

```
[New listing / new guest] -> [Content features: attributes, photos, text, geo]
  -> [Prior from similar entities (embeddings, market averages)]
  -> [Exploration policy] -> [Ranker]
[Batch features (daily)] | [Real-time features (session)] -> [Feature store]
TRAINING: [Early engagement logs] -> [Warm-up model] -> [Eval on new-entity slice]
          -> [A/B with new-host guardrails] -> back
```

Follow-ups:

- How do you estimate engagement for a listing with no bookings or reviews?

- How much exposure do new listings get, and who pays for that exploration?
- When does a contextual bandit beat a supervised model plus an A/B test here?
- How do you personalize for a brand-new guest from session signals alone?
- Which features justify real-time computation, and which are fine in batch?
- How do you evaluate cold-start performance offline on a held-out slice of new entities?
- Which host-side metrics (time to first booking, new-host churn) would you guard?
- How do you stop exploration traffic from contaminating the training labels of the main ranker?

22. Natural-language query understanding and AI search

Prompt. "Design query understanding for Airbnb search: autocomplete, query tagging/NER, query expansion, intent modeling and LLM-powered natural-language search" (for example, "a quiet cabin near Lake Tahoe for 6 with a hot tub"). [Likely] [61]

Maps to: the Relevance and Personalization Query Intelligence team [61] and embedding-based retrieval [39].

```
[Free-text query] -> [Autocomplete] -> [Parser: NER / LLM]
  -> [Hard filters | soft preferences | semantic query] -> [Retrieval + ranking]
  -> [Results + explanation]
TRAINING: [Query logs + reformulations + bookings] -> [Labeled parses]
          -> [Train / distill] -> [Offline parse accuracy] -> [A/B] -> back
```

Follow-ups:

- How do you decide which parsed attributes become hard filters and which become soft preferences?
- How do you evaluate parsing errors that silently drop a hard constraint?
- Where would you use an LLM, and how would you bound its latency and cost on the search path?
- How do you handle ambiguous locations ("Portland")?
- How do you measure whether natural-language search increases bookings, rather than just novelty?
- How do you cache or distill the parser for common queries?
- How do you build labeled parses at scale, and when would you use LLM-generated labels?
- How do autocomplete suggestions shape the queries guests type, and how does that bias your training logs?

23. Monitoring, training-serving skew and the feature store

Prompt. "How do you minimize training-serving skew in a search ranking stack that has both batch and real-time features? Design real-time monitoring for production models: feature and prediction drift, label delay, alerting and rollback." [Likely] [36][60]

Maps to: the Relevance and Personalization posting, which lists skew minimization, A/B tests and feature and model selection [60].

```
[Feature definitions (one source)] -> [Batch backfill] + [Streaming compute]
  -> [Online store] -> [Serving] -> [Logged features at serve time]
  -> [Training set = logged or point-in-time joins]
[Monitors: feature drift | score drift | skew diff | label-delayed metrics]
  -> [Alerts] -> [Rollback / fallback model]
```

Follow-ups:

- Where does skew usually come from (different code paths, time travel, defaults for missing values)?
- When do you log features at serving time instead of recomputing them for training?
- How do you build point-in-time-correct training data?
- How do you monitor a model whose labels arrive days or weeks late?
- What triggers an automatic rollback, and what is the fallback?
- How often do you retrain, and do you A/B test every refresh?
- A predicted booking rate drops suddenly in one market. What do you check first? [Likely] [22]
- How do you tag experiment traffic so that one arm's logs do not contaminate the next model's training data?

24. Distributed recommendation system at Airbnb scale

Prompt. "Design a distributed recommendation system at Airbnb scale. Discuss data quality, the feature store, model serving, A/B testing, and the trade-offs between model complexity, interpretability and latency." [Likely] [32] (An ML platform loop paired an ML design round with a general system design round [7].)

Maps to: Airbnb's batch and real-time ranking infrastructure [60], and the end-to-end search stack from pipelines to experimentation [60].

```
[Event ingestion] -> [Data quality checks] -> [Feature store: offline + online]
  -> [Training cluster] -> [Model registry] -> [Serving fleet (sharded, cached)]
  -> [Request path: candidates -> ranker -> response]
[Experiment assignment] -> [Metrics pipeline] -> [Dashboards + alerts]
```

Follow-ups:

- Where do the latency and throughput bottlenecks sit, and how would you shard and cache?
- How do you catch upstream data-quality breaks before they reach training?
- How do you version models and features together?
- How do you roll out a model safely (shadow, canary, A/B)?
- How would you let other teams (Experiences, Services, long-term stays) reuse the stack?
- How do you serve the same model to several surfaces (search, email, homepage) with different latency budgets?
- What would you precompute in batch versus score on request, and what does each cost?

Level calibration: senior probes

The bank above already includes many senior probes. Use these harder, cross-cutting ones to check that answers show end-to-end ownership and marketplace and experimentation judgment. [Likely] [2][53][60]

1. You own search ranking end to end. Which three metrics would you put on your team's quarterly scorecard, and which metric would you refuse to optimize directly?
2. A ranking change raises guest bookings 1% but shifts bookings from new hosts to top hosts. How do you decide, and who has to agree?
3. Your A/B test is guest-randomized and positive, but a market-level switchback shows no effect. Which result do you trust, and what do you ship?
4. Three teams (ranking, pricing, homepage) all claim the same booking lift. How would you design experiments and holdouts so that the gains add up?
5. A model needs two more quarters of infrastructure work to beat the GBDT baseline. How do you build the case to fund it, or to stop it?
6. Your pricing model's suggestions are adopted by 60% of hosts in a market. How do you tell whether it now sets the market price, and what would you change?
7. Labels for your model come from decisions made by an earlier version of the same model. How do you design logging and exploration so the next model can still learn?
8. A launch is positive on the 2-week readout. What evidence would you require before trusting the long-term effect, and how long would you keep a holdout?
9. Leadership wants an LLM in the search path next quarter. How would you scope the first launch, set the latency and cost budget, and define the kill criteria?
10. A model you shipped a year ago is now hurting a segment nobody monitored. How do you find it, and what process change prevents the next one?

References

Sources were read as search-result snippets or as previously fetched pages, as noted in the source chapters. 1point3acres threads are login-gated. Confirm every link before relying on it.

1. "Airbnb Senior+ Machine Learning Engineer Interview Experience — Search Ranking and Team Matching", PracHub, 2026. https://prachub.com/interview-experiences/airbnb-seniorplus-machine-learning-engineer-interview-experience-search-ranking-and-team-matching
2. "Airbnb Senior Machine Learning Engineer Interview Experience — Four ML System Design Rounds, Rejected on Calibration", PracHub, 2026. https://prachub.com/interview-experiences/airbnb-senior-machine-learning-engineer-interview-experience-four-ml-system-design-rounds-rejected-on-calibration
3. "Airbnb Machine Learning Fulltime Onsite Interview Experience and Insights", 1point3acres, 2025. https://www.1point3acres.com/interview/thread/1162455
4. "Airbnb Fulltime Machine Learning Engineering Onsite Interview Overview", 1point3acres, 2025. https://www.1point3acres.com/interview/thread/1114733
5. Airbnb MLE onsite post ("ML round 1: price prediction system"; the exact thread match is uncertain), 1point3acres, 2022. https://www.1point3acres.com/bbs/thread-93

4565-1-1.html
6. "Airbnb senior MLE interview prep", Blind, 2024. https://www.teamblind.com/post/airbnb-senior-mle-interview-prep-n1mdsd3b
7. "Interviewing for Airbnb ML platform role", Blind, 2025. https://www.teamblind.com/post/interviewing-for-airbnb-ml-platform-role-8in7czso
8. "ML experience round at Airbnb", Blind, year not given. https://www.teamblind.com/post/ml-experience-round-at-airbnb-sorv1g3x
9. "Airbnb Machine Learning Engineer interview process" (TrueInterview-derived summary of 3 candidate reports), kevin-2023-code/Tech-Interview-Questions, GitHub, 2026. https://github.com/kevin-2023-code/Tech-Interview-Questions/blob/main/companies/airbnb/interview-process-machine-learning-engineer.md
10. "Airbnb" question index with report dates, kevin-2023-code/Tech-Interview-Questions, GitHub, 2026. https://github.com/kevin-2023-code/Tech-Interview-Questions/blob/main/companies/airbnb.md
11. 1point3acres post #40 (English mirror: LTV and host-pricing design rounds), FlowSight/coding, GitHub, year not given. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/40.md
12. 1point3acres post #79 (English mirror: listing recommendation and price prediction rounds), FlowSight/coding, GitHub, year not given. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/79.md
13. "Rank Home Search Results Without a Text Query", TrueInterview, 2026. https://trueinterview.io/questions/rank-home-search-results-without-a-text-query
14. "Design User Embedding Semantic Search", TrueInterview, 2026. https://trueinterview.io/questions/design-user-embedding-semantic-search
15. "Listing Lifetime Value — Estimation", TrueInterview, 2026. https://trueinterview.io/questions/listing-lifetime-value-ml-design
16. "Design Airbnb Relisting Detection", TrueInterview, 2026. https://trueinterview.io/questions/design-airbnb-relisting-detection
17. "Generate Customer-Support Instructions from Company Policies", TrueInterview, 2026. https://trueinterview.io/questions/generate-customer-support-instructions-from-company-policies
18. "Design A Personalized Search Ranking System", TrueInterview, 2026. https://trueinterview.io/questions/d6c8316c-c7a9-4074-bec9-f19e4252f21d
19. "Airbnb ML System Design Interview Questions", PracHub, 2025-2026. https://prachub.com/companies/airbnb/categories/ml-system-design
20. "Airbnb Machine Learning Engineer Interview Questions", PracHub, 2025-2026. https://prachub.com/companies/airbnb/positions/machine-learning-engineer
21. "Airbnb Interview Questions", PracHub, year not given. https://prachub.com/companies/airbnb
22. "Airbnb Machine Learning Engineer Interview Guide", Interview Query, 2025. https://interviewquery.com/interview-guides/airbnb-machine-learning-engineer
23. "Airbnb AI Engineer Interview Guide: Real Questions & Expectations", Interview Query, 2026. https://www.interviewquery.com/prep-guides/airbnb-ai-engineer
24. "Airbnb Machine Learning Engineer interview guide", Dataford, 2025. https://dataford.io/interview-guides/airbnb/machine-learning-engineer
25. "Airbnb Machine Learning Engineer", CleverPrep, 2025. https://www.cleverprep.com/companies/airbnb/machine-learning-engineer

26. "Airbnb Interview Guide 2026: Search Systems, Trust and Safety, and Full-Stack Engineering", techinterview.org, 2026. https://techinterview.org/airbnb-interview/
27. "Airbnb Interview Guide 2026: Search Systems, Trust and Safety, and Full-Stack Engineering" (post version), techinterview.org, 2026. https://www.techinterview.org/post/3233460279/airbnb-interview-guide-2026-search-systems-trust-and-safety-and-full-stack-engineering/
28. "Airbnb interview process and top questions", Educative, 2025. https://educative.io/blog/airbnb-interview-process-and-questions
29. "Get a Job at Airbnb: Interview Process and Top Questions", Exponent, 2025. https://tryexponent.com/blog/airbnb-interview-process
30. "Airbnb Machine Learning Engineer Applications: Resume Keywords and Interview Prep", Jobrise, 2026. https://jobrise.io/en/blog/airbnb-machine-learning-engineer-resume-interview-2026/
31. "Airbnb Machine Learning Engineer" guide, DataInterview, year not given. https://www.datainterview.com/blog/airbnb-machine-learning-engineer-interview
32. "Design a Recommendation Service", Codemia, 2025. https://codemia.io/practice/design-a-recommendation-service-12
33. "Design an A/B test for a pricing model", Codemia, year not given. https://codemia.io/practice/design-an-a-b-test-for-a-pricing-model-48
34. "Airbnb Data Scientist guide", Codemia, 2025. https://codemia.io/guides/airbnb-data-scientist
35. "Airbnb Machine Learning Engineer interview questions" (page 5), Prepfully, 2025. https://prepfully.com/interview-questions/airbnb/machine-learning-engineer?page=5
36. "Airbnb Machine Learning Engineer interview questions" (page 8), Prepfully, year not given. https://prepfully.com/interview-questions/airbnb/machine-learning-engineer?page=8
37. "Airbnb Data Scientist Interview Prep Guide", PracHub, year not given. https://prachub.com/interview-prep/9_Zq5otS4cjz
38. "Walk through an experiment from design to decision", PracHub, year not given. https://prachub.com/interview-questions/walk-through-an-experiment-from-design-to-decision
39. "Embedding-Based Retrieval for Airbnb Search", Airbnb Tech Blog (airbnb.tech), 2025. https://airbnb.tech/ai-ml/embedding-based-retrieval-for-airbnb-search/
40. "Applying Embedding-Based Retrieval to Airbnb Search", arXiv, 2026. https://arxiv.org/html/2601.06873v1
41. "Airbnb embedding retrieval system" (case study), Machine Learning at Scale, 2024. https://machinelearningatscale.com/blog/airbnb-embedding-retrieval-system/
42. "Learning and Applying Airbnb Listing Embeddings", airbnb.tech (PDF), 2024. https://airbnb.tech/wp-content/uploads/sites/19/2024/12/Learning-and-Applying-Airbnb-Listing-Embeddings-.pdf
43. Grbovic and Cheng, "Real-time Personalization using Embeddings for Search Ranking at Airbnb", KDD, 2018. https://dl.acm.org/doi/10.1145/3219819.3219885
44. Haldar et al., "Applying Deep Learning to Airbnb Search", arXiv / KDD, 2019. https://arxiv.org/abs/1810.09591
45. Haldar et al., "Improving Deep Learning for Airbnb Search", arXiv / KDD, 2020. https://arxiv.org/abs/2002.05515
46. "The story of how Airbnb implemented neural networks to improve their search feature", Analytics India Magazine, year not given. https://analyticsindiamag.com/the-

story-of-how-airbnb-implemented-neural-networks-to-improve-their-search-feature/
47. "Distilled: Rental Search Ranking", aman.ai, year not given. https://aman.ai/sysdes/airbnb/
48. Abdool, Haldar et al., "Learning To Rank Diversely At Airbnb", arXiv, 2022 (CIKM 2023). https://arxiv.org/abs/2210.07774
49. Tan et al., "Optimizing Airbnb Search Journey with Multi-task Learning", arXiv / KDD, 2023. https://arxiv.org/abs/2305.18431
50. "Personalizing Airbnb search by learning from the guest journey" (summary), thenote.app, year not given. https://thenote.app/post/en/personalizing-airbnb-search-by-learning-from-the-guest-journey-c4zzcdn3wz
51. Ye et al., "Customized Regression Model for Airbnb Dynamic Pricing", KDD, 2018. https://dl.acm.org/doi/10.1145/3219819.3219830
52. "Academic publications: Airbnb Tech 2025 year in review" (includes BiListing and pairwise learning-to-rank), airbnb.tech, 2025. https://airbnb.tech/infrastructure/academic-publications-airbnb-tech-2025-year-in-review/
53. Johari, Li, Liskovich and Weintraub, "Experimental Design in Two-Sided Platforms: An Analysis of Bias", arXiv, 2020. https://arxiv.org/pdf/2002.05670v3
54. Holtz and Aral, "Limiting Bias from Test-Control Interference in Online Marketplace Experiments", arXiv, 2020. https://arxiv.org/pdf/2004.12162
55. "Price Experimentation and Interference", arXiv, 2023. https://arxiv.org/pdf/2310.17165
56. Airbnb CIKM paper on lead-day bias in pricing experiments for expiring goods, airbnb.tech (PDF), 2023. https://airbnb.tech/wp-content/uploads/sites/19/2023/12/CIKM.pdf
57. "Experimentation" notes (pitfalls; lead-day bias in pricing tests), Notion, year not given. https://citrine-straw-ba5.notion.site/Experimentation-0990e9ed61db4d2299bea641c6ab7ffb
58. "Beyond A/B Test: Speeding up Airbnb Search Ranking Experimentation through Interleaving", Airbnb Tech Blog (Medium), 2022. https://medium.com/airbnb-engineering/beyond-a-b-test-speeding-up-airbnb-search-ranking-experimentation-through-interleaving-7087afa09c8e
59. "Experiments at Airbnb", Airbnb Engineering and Data Science (Medium), 2014. https://medium.com/airbnb-engineering/experiments-at-airbnb-e2db3abf39e7
60. "Senior Staff Machine Learning Engineer, Relevance and Personalization", Airbnb Careers, 2025-2026. https://careers.airbnb.com/positions/7381973/
61. "Senior Machine Learning Engineer, Relevance and Personalization (Query Intelligence)", Airbnb Careers, 2026. https://careers.airbnb.com/positions/8065789/
62. "Senior Machine Learning Engineer, Customer Support Engineering", Airbnb Careers, 2026. https://careers.airbnb.com/positions/8014904/
63. "Staff Machine Learning Engineer, Support Products, Airbnb", Working Nomads, 2026. https://www.workingnomads.com/jobs/staff-machine-learning-engineer-support-products-airbnb-460586
64. "Machine Learning Engineer, Customer Support Engineering", Accel jobs board, 2026. https://jobs.accel.com/companies/hoteltonight-2/jobs/75694130-machine-learning-engineer-customer-support-engineering
65. "Senior Staff Machine Learning Engineer, Community Support Engineering (GenAI)", Built In, 2025. https://builtin.com/job/senior-staff-machine-learning-engineer-community-support-engineering/6560520

66. "Senior Staff Machine Learning Engineer, Communication & Connectivity", Built In, 2025. https://builtin.com/job/senior-staff-machine-learning-engineer-communication-connectivity/6495497
67. Paper on integrating LLMs into Airbnb customer support, arXiv, 2025. https://arxiv.org/abs/2510.10331
68. "LLM integration for customer support automation and enhancement" (Airbnb case study), ZenML LLMOps database, 2022. https://zenml.io/llmops-database/llm-integration-for-customer-support-automation-and-enhancement
69. "Machine Learning Engineering Intern (PhD), Airbnb", Working Nomads, 2026. https://www.workingnomads.com/jobs/machine-learning-engineering-intern-phd-airbnb
70. "Building Airbnb Categories with ML and Human-in-the-Loop", Airbnb Engineering (Medium), 2022. https://medium.com/airbnb-engineering/building-airbnb-categories-with-ml-and-human-in-the-loop-e97988e70ebb